

FROM C TO PYTHON · PRODUCTION-READY
REFERENCE

Python 使用大全

面向 C 语言学习者的 14 讲系统进阶

覆盖 14 大主题:数据类型 → 运算符 → 流程控制 → 函数 → 内存管理 → 面向对象
文件I/O → 网络 → 并发异常 → 泛型与元编程 → 反射 → 注解装饰器 → 模块系统 → 类型系统进阶

第1章 数据类型与变量 · 第2章 运算符与表达式 · 第3章 流程控制 · 第4章 函数与方法 · 第5章 内存管理 · 第6章 面向
对象 · 第7章 文件I/O

第8章 网络通信 · 第9章 并发与异常 · 第10章 泛型与元编程 · 第11章 反射与动态特性 · 第12章 注解与装饰器 · 第13
章 模块系统 · 第14章 类型系统进阶

知识导图 × C 语言对照 × 例题精讲 × 章末实验

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 建立**动态类型**世界观:变量是"名字绑定",类型属于对象而非名字
- 把 C 的 `int` / `char` / `double` 与 Python 内建类型对上号: `int` 无上限、`float` 就是 C 的 `double`
- 掌握全部字面量:进制前缀、下划线分隔符、`raw` 字符串、`f-string`、字节字面量
- 理解"常量只是约定":Python 没有 `const`,用大写命名约定
- 掌握显式类型转换与经典坑(`int("3.14")` 报错、`bool("False")` 为真)
- 会用字符串全套 API 与 `list` / `dict` / `set` 基础操作

💡 从 C 到 Python:本章主题如何迁移

在 C 里, `int x;` 意味着"在栈上分配 4 字节,类型在编译期锁死"。这就是静态类型:类型约束内存布局。Python 完全不同:执行 `x = 42` 时,Python 在**堆上**创建整数对象,让名字 `x` 指向它;再执行 `x = "hello"`,又创建字符串对象并让 `x` 指向新对象——旧对象无人引用,稍后被垃圾回收器回收。

所以 Python 的变量是**名字绑定**:名字只持有"指向对象的引用",不持有数据。类型是对象的属性,不是名字的属性。直接后果:**赋值永远只做"换标签",从不拷贝数据**;任何变量可装任何类型。这既是灵活性,也是大工程 bug 的来源——现代工程用类型标注 + `mypy` 补救(第14章)。

本章只要求两个直觉:①变量 = 名字绑定;②类型挂在对象上。两个直觉通了,后面 13 章的语法困惑会消散大半。

📦 前置知识自查(你已会的 C 基础)

- `int` / `unsigned int` / `long long` 的位数与取值范围; `sizeof`
- `char[]` 与字符串字面量、`\0` 结尾; `printf` 的 `%d` / `%s`
- `struct`、`typedef`、`const` 限定符
- 隐式转换规则(整型提升、`int`→`double`)

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 §1.1–§1.2,在 REPL 里用 `type()` / `id()` 验证"名字绑定"
2. **第二阶段(1 小时)**:读 §1.3–§1.5,把 §1.5 的转换坑逐一复现
3. **第三阶段(1.5 小时)**:读 §1.6–§1.8,把 C 的字符串操作改写成 Python 版
4. **第四阶段(实验,1 小时)**:完成章末实验"学生信息登记与成绩统计"
5. **验收标准**:能说清"为什么 `int x;` 与 `x = 42` 本质不同"

本章知识导图

- §1.1 内建类型总览:与 C 的类型对照
- §1.2 变量:名字绑定与动态类型(赋值语义、id、del、命名规范)
- §1.3 字面量:整型/浮点/字符串/raw/f-string/字节/布尔/None
- §1.4 常量:约定优于强制
- §1.5 类型转换:显式转换函数与六大坑
- §1.6 字符串:不可变、切片、常用 API、编码
- §1.7 复合类型入门:list/tuple/dict/set
- §1.8 枚举与类型别名

§1.1 内建类型总览

Python 解释器自带一组内建类型,无需包含任何头文件。下表把它们与 C 类型——对照——这是本章最重要的一张表。

Python 类型	对应的 C 概念	说明与差异
<code>int</code>	<code>int / long long</code>	无上限大整数,不会溢出
<code>float</code>	<code>double</code>	就是 C 的 <code>double</code> ,没有 32 位 <code>float</code>
<code>complex</code>	—	复数,写作 <code>1+2j</code>
<code>bool</code>	<code>_Bool</code>	只有 <code>True / False</code> ,是 <code>int</code> 子类
<code>str</code>	<code>char[]</code>	Unicode 字符串,不可变
<code>bytes</code>	<code>unsigned char[]</code>	字节序列,不可变;bytearray 可变
<code>list</code>	动态数组	可变、自动扩容、元素任意类型
<code>tuple</code>	只读数组	不可变,常用于多返回值
<code>dict</code>	哈希表	键值映射,3.7+ 保持插入顺序
<code>set</code>	哈希集合	去重与集合运算
<code>NoneType</code>	<code>NULL</code> 指针	唯一实例 <code>None</code> ,表示"没有值"

★ 重点:类型是对象的属性

在 C 中"变量有类型";在 Python 中,"对象有类型,变量只是名字"。用 `type(x)` 查询 `x` 当前绑定对象的类型;同一个名字可以先后绑定不同类型的对象。`isinstance(x, T)` 判断"`x` 是否为 `T` 或其子类"——注意它在运行时检查,生产代码不要滥用。

1.1.1 用 `type()` 验证一切

拿到 Python 交互环境后第一件事:用 `type()` 验证每个字面量的真实类型。注意 `issubclass(bool, int)` 为 `True`——布尔就是整数的子类, `True + 1` 得 2,与 C 的 `_Bool` 提升一致。

示例 1-1 类型一览

```
print(type(42))          # <class 'int'>
print(type(3.14))        # <class 'float'> — 就是 double!
print(type(True))        # <class 'bool'>
print(type("hi"))        # <class 'str'>
print(type(b"hi"))       # <class 'bytes'>
print(type(None))        # <class 'NoneType'>
print(type([1, 2]))      # <class 'list'>
print(type((1, 2)))      # <class 'tuple'>
print(type({"k": 1}))     # <class 'dict'>
print(type({1, 2}))      # <class 'set'>
print(issubclass(bool, int)) # True
print(True + 1)          # 2
```

1.1.2 int 不会溢出

C 的 `int` 加 1 就溢出(有符号溢出还是 UB);Python 的 `int` 是任意精度的, `10**100` 得到完整精确的整数。代价:每个整数都是堆上对象,运算比机器整数慢一个数量级——这就是动态类型的取舍。整数写法:二进制 `0b1010`、八进制 `0o12`、十六进制 `0xFF`;没有 `unsigned`, 因为大整数天然无符号概念。

§1.2 变量:名字绑定与动态类型

C 的声明与初始化是两件事,且未初始化局部变量是 UB。Python 只有一种动作: `x = 5` ——不存在“声明”,赋值即创建。读一个从未赋值的名字抛 `NameError`,而不是像 C 一样给你随机值——错误被显式暴露,这正是 Python 好调试的原因。

C 的写法	Python 的写法
<code>int x = 5;</code>	<code>x = 5 # 赋值即"出生"</code>
<code>int y; /* 未初始化→UB */</code>	<code># 不赋值就没有这个名字</code>
<code>x = 3.14; /* 编译错误 */</code>	<code>x = 3.14 # 合法:重新绑定</code>
<code>printf("%d", x);</code>	<code>print(x) # 类型随对象走</code>

1.2.1 赋值语义:绑定引用,不拷贝数据

执行 `b = a` 时,C 拷贝值;Python 拷贝“引用”。若对象可变(如 `list`),两个名字共享同一对象,修改其一,另一个也变——这是 C 程序员踩的第一个 Python 坑。想拷贝用切片 `a[:]` 或 `list(a)` (浅拷贝,深拷贝见第14章)。生产中最常见的例子是函数传参:把列表传给函数,函数里 `append`,调用方的列表也跟着变——第4章会系统讲透这个语义。

示例 1-2 名字绑定 vs 拷贝


```

a = [1, 2, 3]
b = a                # 不拷贝!b 与 a 指向同一个对象
b.append(4)
print(a)             # [1, 2, 3, 4] — a 也"变"了

print(id(a) == id(b)) # True — id 是对象标识(类比地址)

c = a[:]             # 浅拷贝:新列表
c.append(5)
print(a)             # [1, 2, 3, 4] — a 不受影响

# 不可变对象没有这个问题:任何"修改"都产生新对象
s = "hi"
t = s + "!"          # 创建新字符串,t 重新绑定
print(s)             # hi(原对象未动)

```

⚠ 误区 1:以为 `b = a` 之后两者独立

C 里 `int b = a;` 后两者互不影响;Python 里 `b = a` 只是把 `b` 绑到 `a` 的对象上。对**可变对象**(list/dict/set/自定义对象), `b.append(...)` 会反映到 `a`;对**不可变对象**(int/float/str/tuple)则安全。记不住就问自己:这个对象能 `append` 吗?能,就共享了。

1.2.2 命名规范与 `del`

PEP 8 约定:变量与函数用 `snake_case`,类名 `PascalCase`,常量 `ALL_CAPS`。这些是规范而非语法,但违反规范 = 制造混乱。`del x` 解除名字绑定(不是 `free`,对象等垃圾回收);`del lst[0]` 删元素;`del d["k"]` 删键值对。

§1.3 字面量

字面量是"直接写在源码里的值"。Python 的字面量家族比 C 丰富得多,一个示例全部覆盖:

示例 1-4 字面量全家桶

```
# ---- 整数 ----
n1 = 42                # 十进制
n2 = 0x2A              # 十六进制(与 C 一致)
n3 = 0o52              # 八进制(C 写作 052)
n4 = 0b101010         # 二进制(C 无原生支持)
n5 = 1_000_000         # 下划线分隔符(3.6+, 可读性利器)
print(n2, n3, n4, n5)  # 42 42 42 1000000

# ---- 浮点 ----
f1 = 3.14
f2 = 1.5e-3            # 科学计数法
f3 = float("inf")      # 正无穷(还有 -inf 与 nan)

# ---- 字符串 ----
s1 = "单引号也行"      # 两种引号等价
s2 = ""可以跨行""      # 三引号:多行字符串
s3 = r"C:\Users\new"   # raw 字符串:反斜杠不转义!
s4 = b"byte"           # 字节字面量,类型 bytes
s5 = f"1+1={1+1}"      # f-string:花括号内是表达式
s6 = f"{s1!r}"         # !r 用 repr 显示(带引号)

# ---- 特殊值 ----
t = True               # 布尔只有 True / False
z = None               # None:类比 NULL 指针
```

⚠ 误区 2:反斜杠与 C 的记忆冲突

在 C 里 "C:\new" 的 \n 是换行;Python 同样如此!所以 Windows 路径必须写 r"C:\new" (raw)或 "C:\\new" 或 "C:/new"。生产代码里写 Windows 路径,永远用 raw 字符串或 pathlib(第7章)。

1.3.1 f-string:现代格式化主力

C 用 printf("%d, %s", n, s) 占位;Python 3.6+ 首选 f-string:字符串前加 f,花括号里直接写表达式。旧式 % 格式化与 str.format() 只在兼容老代码时用。

示例 1-5 f-string 实战

```
name, score = "李雷", 92.5
print(f"{name} 的成绩是 {score:.1f} 分")  # 李雷 的成绩是 92.5 分

print(f"{name:<10}|")    # 左对齐占 10 格
print(f"{name:>10}|")    # 右对齐占 10 格
print(f"{score:06.1f}")  # 总宽 6 补零 1 位小数 → 0092.5

width, height = 3, 4
print(f"面积 = {width * height}")  # 花括号里能算
print(f"数名:{name!r}")           # 调试神器:带引号
```

§1.4 常量:约定优于强制

Python **没有** `const` / `#define`。社区约定:全大写命名,并约定"不修改"——解释器不拦你。这是刻意设计:Python 认为"约束靠自觉与审查,而非编译器"。防修改的三种手段:

- 用不可变对象: `tuple`、`frozenset` 无法就地修改
- 类型标注 `typing.Final` + `mypy` 静态检查(第14章)
- 模块级集中定义:外部 `import config` 只读使用

示例 1-6 常量的正确打开方式

```
# config.py — 常量集中管理,别散落各处
MAX_RETRY = 3
TIMEOUT_SEC = 30.0
DEFAULT_ENCODING = "utf-8"
SUPPORTED_EXT = frozenset({".py", ".md", ".txt"}) # 防误改

def load_file(path: str):
    if not path.endswith(tuple(SUPPORTED_EXT)):
        raise ValueError(f"不支持的扩展名:{path}")
    return open(path, encoding=DEFAULT_ENCODING)

# 可变 vs 不可变
a = (1, 2) # tuple 不可变
# a[0] = 9 # TypeError: 'tuple' object does not support item assignment
b = [1, 2] # list 可变
b[0] = 9 # 合法
```

§1.5 类型转换

C 有隐式转换(整型提升、`int`→`double`)和显式转换 `(double)x`。Python 的立场:数值间保留少量隐式转换,其余一律显式。隐式转换只发生在 `int` ↔ `float` ↔ `complex` 的算术里;`int` 与 `str` 之间**绝不**隐式转换——`"a" + 1` 直接抛 `TypeError`,而不是像 JS 那样偷偷变成 `"a1"`。这就是"强类型动态语言"。

C 的写法	Python 的写法
<code>(double)x / (int)y</code>	<code>float(x) / int(y)</code>
<code>sprintf(buf, "%d", n)</code>	<code>f"{n}"</code> 或 <code>str(n)</code>
<code>atoi("42")</code>	<code>int("42")</code> (非数字抛 <code>ValueError</code>)
字符 ↔ 数字	<code>ord("A")</code> → 65; <code>chr(65)</code> → "A"

1.5.1 转换的六大经典坑

示例 1-7 转换的坑与正确姿势

```
# 坑 1:int("3.14") 失败!int() 不接受小数点字符串
# n = int("3.14")          # ValueError: invalid literal
n = int(float("3.14"))     # 正确姿势:先 float 再截断 → 3

# 坑 2:int("42.0") 同样失败——先浮点再整
# int("42.0")              # ValueError

# 坑 3:进制字符串要带第二参
v = int("ff", 16)          # 255
v2 = int("1010", 2)        # 10

# 坑 4:bool("False") 是 True!非空字符串恒为真
print(bool("False"), bool(""), bool(0)) # True False False

# 坑 5:float 截断是向零截断,不是四舍五入
print(int(3.7), int(-3.7)) # 3 -3

# 坑 6:首尾空格被容忍,中间的不行
print(int(" 42 "))        # 42
# int("4 2")              # ValueError
```

⚠ 误区 3:以为 Python 会自动截断或拼接

7 / 2 得 3.5 (真除法,不截断!);要截断用 7 // 2。"1" + "2" 是 "12" 不是 3;数字与字符串拼接必须 f"{n}"。一切"自动"都没有,一切显式——这正是 Python 好调试的原因。

§1.6 字符串:从 char 数组到一等公民

C 的字符串是 `char[]`,拼剪全靠 `strcpy / strcat`,还提防缓冲区溢出。Python 的 `str` 是**不可变的 Unicode 字符串序列**:没有 `\0`、没有缓冲区边界。不可变带来直接好处:字符串对象可以被安全共享,不用担心某处修改破坏别处。

1.6.1 索引、切片与常用 API

索引从 0 开始,负数从末尾数(`s[-1]` 是末字符)。切片 `s[start:stop:step]` 是 Python 最强特性,口诀:左闭右开,负数倒数,步长可负。

示例 1-8 字符串 API 速览

```
s = "Hello, Python"
print(s[0], s[-1])          # H n
print(s[0:5])               # Hello(不含下标 5)
print(s[7:])                 # Python(到末尾)
print(s[::-1])              # nohtyP ,olleH(逆序!)

print(s.find("Py"))         # 7;找不到返回 -1 不抛异常
print(s.replace("Python", "C")) # Hello, C
print("a,b,c".split(","))    # ['a', 'b', 'c']
print("-".join(["2026", "08", "26"])) # join 在分隔符上!
print(" hi ".strip())        # hi
print("abc123".isalpha())    # False

print(len("你好"))          # 2 — len 是字符数,不是字节数
```

1.6.2 编码:str 与 bytes 的鸿沟

C 的 `char` 是字节;Python 的 `str` 是 Unicode 码点。读写文件、发网络包时, `str` 必须编码成 `bytes`, 反之解码。`bytes` 只是字节,怎么解释成字符由编码决定;现代统一用 UTF-8。

示例 1-9 编码与解码

```
text = "中文Python"
data = text.encode("utf-8")      # str → bytes
print(data)                      # b'\xe4\xb8\xad\xe6\x96\x87Python'
print(len(text), len(data))      # 8 字符 vs 13 字节

back = data.decode("utf-8")      # bytes → str
print(back == text)              # True

# 编码不一致会乱码或报错;文件读写统一
# 指定 encoding="utf-8"(第7章详述),别依赖系统默认
```

§1.7 复合类型入门:list / tuple / dict / set

C 数组定长同型;Python 的 `list` 是变长、任意元素、自动扩容的动态数组——C 里手写 `realloc` 数组的成品版。`list` 的索引与切片规则和字符串完全一致:负数下标从尾部数起,切片左闭右开。这里给基础操作,第14章从内存视角深入。

示例 1-10 list/dict/set 基础操作

```
# ---- list ----
nums = [3, 1, 4, 1, 5]
nums.append(9)      # 尾部追加(自动扩容)
nums.insert(0, 0)   # 头部插入
nums.sort()         # 就地排序
print(nums)         # [0, 1, 1, 3, 4, 5, 9]
print(len(nums))    # 7

# ---- tuple:不可变,常用于打包 ----
point = (3, 4)
x, y = point        # 解包
single = (1,)       # 单元素元组必须带逗号!

# ---- dict ----
student = {"name": "韩梅梅", "age": 18, "scores": [88, 92, 95]}
print(student["name"])      # 韩梅梅;键不存在抛 KeyError
print(student.get("grade", "N/A")) # N/A:get 带默认值
student["grade"] = "大一"   # 新增键值对
for k, v in student.items(): # 遍历键值
    print(k, v)

# ---- set:去重 ----
tags = {"python", "c", "python"}
print(tags)      # {'c', 'python'}
print("c" in tags) # True(哈希查找)
```

⚠ 误区 4:dict 取键以为像数组越界一样安全

`d["k"]` 键不存在抛 `KeyError` (会显式报错,这是好事)。生产习惯:不确定键是否存在用 `d.get(key, default)`;遍历用 `for k, v in d.items()`。另外 dict 的键必须可哈希——list 不能做键,tuple 可以,因为 dict 底层就是哈希表。

§1.8 枚举与类型别名

C 的 `enum` 只是带名字的整数,可以把 42 直接赋给枚举变量,零检查。Python 的 `enum.Enum` 是真正类型:成员是对象,值必须合法,repr 清晰,还能带方法。`IntEnum` 可当整数无缝使用(如做下标)。

示例 1-11 枚举与类型别名

```
from enum import Enum, IntEnum

class Color(Enum):
    RED = 1
    GREEN = 2
    BLUE = 3

class Level(IntEnum):          # IntEnum 可以当 int 用
    LOW = 1
    MEDIUM = 2
    HIGH = 3

print(Color.RED.name, Color.RED.value)    # RED 1
print(Color(2))                          # Color.GREEN:按值反查
print(Level.HIGH > Level.LOW)             # True

# 类型别名:C 的 typedef 在 Python 里就是普通赋值
StudentId = int
sid: StudentId = 20260001                # 标注只是说明,不强制
print(type(sid))                         # <class 'int'> — 别名不产生新类型
```

🚀 工程建议:枚举优先于魔法数字

生产代码里,状态、等级、类型字段一律用 `Enum` 而非裸整数:`order.status = OrderStatus.PAID` 比 `order.status = 3` 可读得多,还能防越界值。这是"可读性投资"里回报率最高的一项;`StrEnum` (3.11+)可直接当字符串用。

本章速查表

主题	写法 / 结论
类型查询	<code>type(x)</code> ; <code>isinstance(x, T)</code>
整数	任意精度; <code>0x</code> / <code>0o</code> / <code>0b</code> 前缀; <code>1_000_000</code>
浮点	只有 <code>double</code> ; <code>1.5e-3</code> 、 <code>float("inf")</code>
None	判空写 <code>x is None</code> 而非 <code>==</code>
变量	赋值即创建; 名字可重新绑定; 无未初始化
拷贝	<code>b = a</code> 只绑引用; <code>a[:]</code> 浅拷贝
常量	无 <code>const</code> ; ALL_CAPS + 集中定义 + 自觉
字符串	不可变; 负索引; <code>s[1:4]</code> 左闭右开; <code>len</code> 数字符
f-string	<code>f"{x:.2f}"</code> 、 <code>f"{x:>10}"</code> 、 <code>f"{x!r}"</code>
转换	<code>int(x)</code> / <code>float(x)</code> / <code>str(x)</code> ; <code>int(s, 16)</code>
编码	<code>s.encode("utf-8")</code> ↔ <code>b.decode("utf-8")</code>
list/dict	<code>append</code> / <code>sort</code> ; <code>d.get(k, 默认)</code> 安全
枚举	<code>Enum</code> 类型安全; <code>IntEnum</code> 当 <code>int</code> 用

最小必会清单

- 能说出: 变量是"名字绑定", 类型属于对象, 赋值不拷贝数据
- 能写出 `int/float/bool/str/bytes/None/list/dict/set` 各一个字面量
- 能用 `type()/id()` 验证"同一个对象"与"同一个值"的区别
- 能解释 `int("3.14")` 为什么报错、正确写法是什么
- 能写出 `f"{name:>10}"` 右对齐与 `f"{x:.2f}"` 保留小数
- 能说出切片左闭右开规则, 写出逆序切片 `s[::-1]`
- 能区分 `str`(字符)与 `bytes`(字节), 知道 `encode/decode`
- 能用 `d.get(key, default)` 安全读字典
- 能解释 `bool("False")` 为什么是 `True`

自测题

Q 1. 执行 `a = [1, 2]; b = a; b.append(3)` 后 `a` 是什么? 若 `a = "hi"; b = a; b = b + "!"`, `a` 呢? 为什么不同?

✅ 参考答案

`[1, 2, 3]`: list 可变, `b = a` 共享对象。字符串场景 `a` 仍是 `"hi"`: str 不可变, `b + "!"` 创建新对象让 `b` 重新绑定。核心: 可变对象共享修改, 不可变对象"修改"即新建。

Q 2. 下列哪个抛异常? ① `int("3.14")` ② `float("3.14")` ③ `int("0xff", 16)` ④ `int("1e3")` ⑤ `bool("False")`

✓ 参考答案

① 和 ④ 抛 `ValueError: int()` 不接受小数点与指数记法;② 得 3.14;③ 得 255;⑤ 得 True。

Q 3. `s = "Hello, World"` ,写出取 `"World"` 的切片、逆序切片、最后一个字符的表达式。

✓ 参考答案

`s[7:]` (或 `s[-5:]`)、`s[::-1]`、`s[-1]`。切片左闭右开,省略 stop 到末尾。

Q 4. `7 / 2`、`7 // 2`、`-7 // 2`、`-7 % 2` 各是多少?与 C 有何不同?

✓ 参考答案

3.5、3、-4、1。Python 的 `//` 与 `%` 向下取整(符号随除数),恒有 `a == (a//b)*b + a%b`;C 向零截断(`-7/2 == -3`)。

Q 5. 为什么 `len("中文")` 是 2 而 `len("中文".encode("utf-8"))` 是 6?这说明了什么?

✓ 参考答案

`str` 的 `len` 数 Unicode 字符(码点),`bytes` 的 `len` 数字节。中文在 UTF-8 中每字 3 字节,2 字符 = 6 字节。`str` 面向人类可读字符,`bytes` 面向存储传输,必须显式编码/解码。

Q 6. `d = {}` ,依次执行 `d["a"] = 1`、`d["b"] = 2`、`d.setdefault("a", 100)`、`d["a"] = d.get("a", 0) + 1` ,最终 `d` 是什么?

✓ 参考答案

`{'a': 2, 'b': 2}`。`setdefault` 只在键不存在时写入, "a" 保持 1; `d.get("a", 0)` 得 1,加 1 回写为 2。这类场景 `collections.defaultdict` 是更优解。

章末实验题:学生信息登记与成绩统计系统

实验 1:学生信息登记与成绩统计

实验目标:用全章所有数据类型(整型/浮点/布尔/字符串/list/dict/枚举/常量/转换/格式化)实现一个命令行学生管理系统。

实验要求:

- 任务 1:用 `Enum` 定义年级枚举(FRESHMAN/SOPHOMORE/JUNIOR/SENIOR)(知识点:枚举)
- 任务 2:用常量模块定义配置:最大学生数、及格线 60.0、编码 utf-8(知识点:常量约定)
- 任务 3:单个学生用 dict 存储{学号、姓名、年龄、年级、是否在校 bool、成绩 list},总表用 list(知识点:复合类型)
- 任务 4:录入时用 `input()` 读字符串, `int()` / `float()` 转换并捕获 `ValueError` 重输(知识点:类型转换与异常)
- 任务 5:统计平均分、最高分、及格率,f-string 格式化输出(知识点:格式化与算术)
- 任务 6:按学号查找学生并打印完整信息(知识点:dict 查询)

实验环境:Python 3.10+,标准库即可。

第一步 写 `config.py` 与 `student.py` (常量与枚举),再写主程序,保持"数据定义在前、逻辑在后"

第二步 入函数用 `while True + try/except` 循环处理非法输入——生产最常见的输入校验模式

第三步 计用 `sum()/max()` 或手动循环;平均分用真除法 `/` 得到 float

第四步 出报告用 `f"{avg:.1f}"` 控制小数位

✅ 参考答案要点(代码分两段)

第一段:配置、枚举与录入。关键设计:枚举隔离魔法数字、常量集中配置、录入函数返回形状统一的 dict。

```
# config.py — 常量集中定义
MAX_STUDENTS = 100
PASS_LINE = 60.0
ENCODING = "utf-8"

# student.py — 枚举定义
from enum import Enum

class Grade(Enum):
    FRESHMAN = 1      # 大一
    SOPHOMORE = 2     # 大二
    JUNIOR = 3         # 大三
    SENIOR = 4         # 大四

# main.py — 主程序
from config import MAX_STUDENTS, PASS_LINE, ENCODING
from student import Grade

def make_student(sid: str, name: str, age: int,
                 grade: Grade, active: bool, scores: list) -> dict:
    """构造一个学生 dict(形状统一,便于后续处理)"""
    return {
        "sid": sid, "name": name, "age": age,
        "grade": grade, "active": active, "scores": scores,
    }

def input_score(prompt: str) -> float:
    """读入成绩,非法输入反复要求重输"""
    while True:
        raw = input(prompt).strip()
        try:
            value = float(raw)
            if 0.0 <= value <= 100.0:
                return value
            print("成绩必须在 0~100 之间")
        except ValueError:
            print("请输入数字")
```

第二段:录入组装、统计报告与主流程。注意学号查找用了 dict 存储,查找 O(1)。

```

def input_student() -> dict:
    """录入一个学生:覆盖类型转换、枚举反查"""
    sid = input("学号: ").strip()
    name = input("姓名: ").strip()
    age = int(input("年龄: "))          # 非法整数抛 ValueError
    grade_no = int(input("年级(1~4): "))
    grade = Grade(grade_no)           # 按值反查枚举成员
    active = input("是否在校(y/n): ").strip().lower() == "y"
    scores = [input_score(f"第{i}门成绩: ") for i in range(1, 4)]
    return make_student(sid, name, age, grade, active, scores)

def report(students: list) -> None:
    """统计并格式化输出"""
    total = [sum(s["scores"]) / len(s["scores"]) for s in students]
    avg = sum(total) / len(total)
    top = max(total)
    rate = sum(1 for t in total if t >= PASS_LINE) / len(total) * 100
    print("=" * 40)
    print(f"学生人数: {len(students)} | 平均: {avg:.1f} | 最高: {top:.1f}")
    print(f"及格率: {rate:.1f}%")
    print("=" * 40)
    for s in students:
        avg_s = sum(s["scores"]) / len(s["scores"])
        tag = "及格" if avg_s >= PASS_LINE else "不及格"
        print(f"{s['sid']} {s['name']} 年级={s['grade'].name} "
              f"平均={avg_s:.1f} [{tag}]")

def find_student(students: list, sid: str) -> dict | None:
    """按学号查找;找不到返回 None(True 值判定)"""
    for s in students:
        if s["sid"] == sid:
            return s
    return None

def main() -> None:
    students = []
    while input("继续录入? [y/n]: ").strip().lower() == "y":
        try:
            students.append(input_student())
        except (ValueError, TypeError) as exc:
            print(f"录入失败,已跳过:{exc}")
    report(students)
    sid = input("查询学号: ").strip()
    stu = find_student(students, sid)
    if stu is not None:          # 判 None 用 is
        print(f"{stu['name']} 平均分 "
              f"{sum(stu['scores']) / len(stu['scores']):.1f}")
    else:
        print("未找到该学号")
    with open("students.txt", "w", encoding=ENCODING) as fh:
        for s in students:
            fh.write(f"{s['sid']},{s['name']},{s['age']},")

```

```
f"{s['grade'].value},{int(s['active'])}\n")
```

```
if __name__ == "__main__":  
    main()
```

设计说明:① 录入函数把"读字符串→校验→转类型→组装"拆成独立步骤,任何一步失败都能单独测试;② 枚举成员用 `.name` 显示、`.value` 持久化,体现"面向人的表示"与"面向机器的表示"分离;③ 保存文件用 `with open` 保证资源释放(第5、7章细讲),显式 `encoding="utf-8"` 避免 Windows 默认 GBK 乱码——这条在真实生产里救过无数人。

下一章预告:第2章 运算符与表达式——`//` 向下取整、`**` 幂运算、短路返回操作数、链式比较、`walrus` 操作符,Python 的运算符世界与 C 有 8 处本质不同。

第2章 运算符与表达式

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 `/`、`//`、`%` 三兄弟的精确语义,尤其是**负数取模向下取整**与 C 的差异
- 理解 Python **没有** `++` / `--` 的原因,并写出等价写法
- 掌握 `and` / `or` / `not` 的短路求值与"返回操作数"特性,能写出惯用的默认值表达式
- 掌握链式比较、`==` vs `is` 的本质区别、浮点比较的正确姿势
- 掌握解构赋值、多变量交换、`:=` walrus 操作符(3.8+)
- 熟记优先级与结合性,能读懂任何复杂表达式,并会主动用括号消歧义
- 会用位运算处理标志位与掩码,知道何时该用(权限系统)何时不该用(可读性优先)

💡 从 C 到 Python:本章主题如何迁移

C 的运算符你全会——Python 保留了绝大部分,但语义上有几处**本质性差异**,每一处都是 C 程序员的第一天翻车现场。第一, `/` 永远是真除法: `7 / 2` 在 C 里是 3,在 Python 里是 `3.5`,C 的整数除法语义被拆给了 `//`。第二,取模符号不同: `-7 % 2` 在 C 里是 -1,在 Python 里是 1。第三, `++` / `--` 不存在——设计者认为 `x++` 隐藏了"既读又写"的复杂性,而且 `++x` 会被解析成 `+(+x)`,这是新手最常见的槽点。

第四,逻辑运算符不返回布尔,返回操作数本身: `"a" or "b"` 得到 `"a"` ——这是生产里最常用的技巧(默认值、链式查找)。第五, `==` 比较"值", `is` 比较"是不是同一个对象"——C 程序员容易把两者用反。

好消息:Python 把 C 的未定义行为全部消灭了——没有求值顺序 UB、没有符号溢出、没有序列点纠缠,表达式严格从左到右求值,任何疑问都有明确答案。读这一章带着一个问题: "同样的符号,Python 和 C 的行为哪里不一样?" ——全章所有坑都藏在答案里。

📦 前置知识自查(你已会的 C 基础)

- C 运算符的优先级与结合性(尤其 `*` 与 `++` 的纠缠、移位优先级陷阱)
- 短路求值: `&&` 与 `||` 从左到右、提前停止
- 整数除法与取模的向零截断; `%` 对负数时的符号规则
- 位运算:与/或/异或/非/左移/右移;掩码与标志位的用法
- `?:` 三目运算符; `++i` 与 `i++` 的区别

🚀 本章学习路线

1. **第一阶段(约 1 小时,算术与取模)**:读 §2.1–§2.2,在 REPL 里把负数除法、取模全部算一遍,建立新直觉
2. **第二阶段(约 1 小时,位运算与逻辑)**:读 §2.3–§2.4,重点练习 `and` / `or` 的返回值技巧
3. **第三阶段(约 1.5 小时,比较与赋值)**:读 §2.5–§2.7,做浮点比较与解构练习
4. **第四阶段(实验,约 1 小时)**:完成章末实验"表达式求值计算器",它是全章运算符的综合训练场
5. **验收标准**:能脱口而出 `-7 // 2`、`-7 % 2`、`0.1 + 0.2 == 0.3` 的答案与原因

本章知识导图

- §2.1 算术运算:/ 真除法、// 整除、% 取模、** 幂
- §2.2 自增自减的缺失与替代写法
- §2.3 位运算:全套运算符、标志位与掩码
- §2.4 逻辑运算:短路求值与"返回操作数"
- §2.5 比较运算:链式比较、== vs is、浮点比较
- §2.6 赋值:复合赋值、解构、walrus :=
- §2.7 条件表达式(三目的 Python 版)
- §2.8 优先级与结合性、特色运算符总表

§2.1 算术运算

Python 的算术运算符与 C 同名的只有 `+` `-` `*`,其余三个都有讲究。先看总表:

C 运算符	Python 运算符	差异要点
<code>/</code> (整数除法截断)	<code>/</code> (真除法,永远得 float)	<code>7/2 == 3.5</code> ;C 要 <code>7.0/2</code> 才行
<code>—</code>	<code>//</code> (向下取整除法)	<code>-7//2 == -4</code> ,向负无穷取整
<code>%</code> (余数符号同被除数)	<code>%</code> (模,符号同除数)	<code>-7%2 == 1</code> ,恒有 <code>a == (a//b)*b + a%b</code>
<code>—</code>	<code>**</code> (幂)	<code>2**10 == 1024</code> ;C 要 <code>pow()</code>
<code>int</code> 溢出(UB / 回绕)	大整数,永不溢出	<code>2**200</code> 完全精确

2.1.1 除法三兄弟与 divmod

三个除法运算符必须背熟: `/` 真除法(结果为 float)、`//` 向下取整(结果为整)、`%` 模运算(符号随除数)。Python 保证 `a == (a // b) * b + a % b` 恒成立——这是与 C"向零截断"的本质分歧:Python 选"向负无穷取整 + 模恒非负",因为这样 `%` 才能实现"循环索引"等需求(`(-1) % 7 == 6`,把 -1 安全映射到一周第 6 天)。

示例 2-1 除法三兄弟

```
print(7 / 2)      # 3.5    真除法:永远返回 float
print(7 // 2)     # 3      向下取整
print(-7 // 2)    # -4     C 给 -3,Python 向负无穷取整!
print(7 % 2)      # 1
print(-7 % 2)     # 1      C 给 -1,Python 符号随除数(2)
print(7 % -2)     # -1     除数 -2,余数符号随除数
print(-7 % -2)    # -1

# 恒等式:整除 + 模 可以互相推导
a, b = -7, 2
print(a == (a // b) * b + a % b)  # True

# divmod:一次拿到 (商, 余)
q, r = divmod(17, 5)
print(q, r)        # 3 2
```

2.1.2 幂运算与浮点警告

`2 ** 10` 是幂; `2 ** 0.5` 是开方; 注意 `-2 ** 2` 是 `-(2**2)` 得 -4, 因为 `**` 优先级高于一元负号——教科书级坑。

浮点算术必须牢记: `0.1 + 0.2` 得 `0.30000000000000004`。这不是 Python 的 bug, 是 IEEE 754 与十进制小数的固有误差, C 里一模一样。生产对策: 金额用 `decimal.Decimal`, 科学计算用容差比较, 普通场景接受误差。

示例 2-2 幂与浮点

```
print(2 ** 10)          # 1024
print(2 ** 0.5)         # 1.4142135623730951(开方)
print(-2 ** 2)          # -4 !! (先算 2**2 再取负)
print((-2) ** 2)        # 4

# 浮点误差: 人尽皆知的 0.1+0.2
print(0.1 + 0.2)        # 0.30000000000000004
print(0.1 + 0.2 == 0.3) # False

# 金额计算用 Decimal
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2")) # 0.3 精确
```

⚠ 误区 1: 把 `//` 当作"C 的整数除法"直接搬

`7 // 2 == 3` 和 C 一致, 但 `-7 // 2 == -4` 与 C 的 -3 不同。C 的整除向零截断, Python 的 `//` 向负无穷取整。生产场景: 计算分页页码、环形缓冲索引时, `(-1) // 7` 在 Python 里是 -1, 若按 C 的直觉写成 `(-1) / 7` 再取整会差 1。只要涉及负数, 先想清楚再写。

§2.2 没有 ++ 与 --

Python 没有自增自减运算符。设计者认为 `i++` 既是有值的表达式又是带副作用的语句, 这种"双重身份"是 C 求值顺序 UB 与无数 bug 的源头。Python 的立场: 修改变量就是赋值语句, 写 `i += 1`。注意 `++i` 在 Python 里**不报错**——它被解析成 `+(+i)` (正号运算符), C 程序员写顺手了会得到"看起来没错其实没自增"的代码, 这是本章第二著名的坑。

示例 2-3 自增的替代写法

```
i = 0
i += 1          # 唯一正确的自增写法
print(i)        # 1

# 危险: ++i 不报错, 但只是两个正号!
j = 5
print(++j)      # 5 (不是 6, 也不是语法错误)

# C 的 for(i=0; i<n; i++) 在 Python 里:
n = 10
for i in range(n):
    pass
i = 0
while i < n:    # 或等价 while
    i += 1
```

§2.3 位运算

Python 的位运算符与 C 完全同名: `& | ^ ~ << >>`。差异只有一点:右移是**算术右移**(负数高位补 1),因为 int 任意精度,"符号位"天然存在, `-8 >> 1 == -4`,与 C 的带符号整数一致。

位运算生产用途:①标志位组合(权限系统);②掩码提取(协议解析)。不推荐用位运算做快速乘除 2 的幂——写 `x // 16` 比 `x >> 4` 好读,解释器里性能差异可忽略。

示例 2-4 位运算与权限标志

```
# 权限标志:每一位代表一种权限
READ = 1          # 0b001
WRITE = 2         # 0b010
EXEC = 4          # 0b100

perm = READ | EXEC    # 组合 → 5
print(bin(perm))      # 0b101
print(perm & READ)     # 1(非 0 即有该权限)
print(bool(perm & WRITE)) # False(无写权限)

# 掩码提取:从协议字段中取比特
packet = 0b11010100
low4 = packet & 0b1111    # 取低 4 位
high4 = packet >> 4       # 取高 4 位
print(low4, high4)       # 4 13

# 负数右移是算术右移,保持符号
print(-8 >> 1)           # -4
print(-8 << 1)           # -16

# 位运算查权限:非 0 即拥有(与 C 的 if (perm & FLAG) 同构)
print(bool(perm & WRITE)) # False
```

§2.4 逻辑运算:短路与返回值

Python 的逻辑运算符是 `and / or / not` (不是 `&& / || / !`),与 C 的对应关系如下:

C 的写法	Python 的写法	差异
<code>a && b</code>	<code>a and b</code>	C 返回 0/1;Python 返回操作数本身
<code>a b</code>	<code>a or b</code>	同上,短路规则一致
<code>!x</code>	<code>not x</code>	not 返回布尔 True/False
<code>x>=0 && x<100</code>	<code>0 <= x < 100</code>	链式比较一步到位

语义有三条,第二条是 C 程序员必须重构直觉的:

1. **短路求值**与 C 完全一致:从左到右,能定则停; `False and 任何` 不再求右边
2. **返回值是操作数本身**,不是布尔! `a and b` 返回"使表达式定值的那一侧"的值: `a` 为假返回 `a`,否则返回 `b`; `a or b` 为真返回 `a`,否则返回 `b`
3. 真值判定:非零数、非空容器、`True` 为真;0、空字符串/列表/字典、`None`、`False` 为假

示例 2-5 短路与“返回操作数”

```
# 短路:右侧根本不会执行
def side_effect():
    print("被调用了")
    return True

print(False and side_effect())    # False,不打印(短路)
print(True or side_effect())      # True,不打印(短路)
print(True and side_effect())     # 打印,返回 True

# 返回操作数:默认值写法
name = "" or "匿名用户"          # "" 为假 → 取 "匿名用户"
count = 0 or 10                  # 0 为假 → 10
print(name, count)               # 匿名用户 10

# 链式取第一个非空值(等价于 C 里一串 if)
a, b, c = "", None, "最终值"
print(a or b or c)               # 最终值

# 注意:or 不能用来判断"是否为 None"(0 和 "" 也被吃掉)
level = 0
print(level or 5)                # 5 — 但 0 可能合法!
print(level if level is not None else 5)  # 0:显式判 None
```

⚠ 误区 2: or 默认值把合法假值吃掉了

`x or 默认` 在 `x` 是 0、空串、空列表时也返回默认值。若这些“假值”业务上合法(分数 0、空名单),就用 `x if x is not None else 默认`。真实事故:统计系统用 `avg or 0` 处理“无数据”,平均分为 0 的班级全部被污染——区分“没有值(None)”与“值为假”是生产必修课。

§2.5 比较运算

比较运算符 `==` `!=` `<` `>` `<=` `>=` 都在,但有两个 Python 特性:一是**链式比较** `0 <= x < 100` 等价于 C 的 `x >= 0 && x < 100`;二是 `==` 与 `is` 的分工: `==` 比较“值相等”(递归比较容器内容), `is` 比较“同一个对象”。判 `None` 必须用 `is None` (PEP 8 强约定):因为 `None` 是单例,“是不是 `None`”本质是身份问题,用 `==` 在自定义类型上可能被 `__eq__` 欺骗。

示例 2-6 链式比较与 `==` vs `is`

```

# 链式比较:C 要写两遍变量,Python 一次搞定
x = 42
print(0 <= x < 100)          # True
print(100 > x >= 40)         # True(反向链式也行)

# == 比较值,is 比较身份
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)                # True:内容逐元素相等
print(a is b)                # False:两个不同对象

# 小整数缓存陷阱:解释器对 -5~256 做了缓存
p, q = 256, 256
print(p is q)                # True(缓存!)
m, n = 257, 257
print(m is n)                # 可能是 False(未缓存)
# 结论:比数值用 ==,is 只用于 None 与单例

value = None
print(value is None)         # True(生产标准写法)

```

2.5.1 浮点比较的正确姿势

`0.1 + 0.2 == 0.3` 是 `False`,所以浮点"相等"要用容差。 `math.isclose` 处理相对误差与绝对误差,生产直接用它。

示例 2-7 浮点比较

```

import math

print(0.1 + 0.2 == 0.3)      # False(二进制浮点误差)
print(math.isclose(0.1 + 0.2, 0.3)) # True(默认容差 1e-09)

# 小数值场景要显式指定绝对容差
print(math.isclose(1e-10, 0.0, abs_tol=1e-9)) # True

# 逐元素比较
scores = [99.9, 100.0, 98.5]
hits = sum(1 for s in scores if math.isclose(s, 99.9, rel_tol=1e-6))
print(hits)                  # 1

```

§2.6 赋值:复合赋值、解构与 walrus

2.6.1 复合赋值与多变量解构

`+=` `-=` `*=` `/=` `//=` `%=` `**=` `&=` `|=` `^=` `<<=` `>>=` 全都在,语义与 C 一致(先求右侧再重新绑定)。Python 用**元组解构**实现更强的多变量赋值:

C 的写法	Python 的写法
<code>tmp=a; a=b; b=tmp;</code>	<code>a, b = b, a</code>
<code>int x, y; x=1; y=2;</code>	<code>x, y = 1, 2</code>
<code>sscanf(s, "%d %d", &x, &y)</code>	<code>x, y = map(int, s.split())</code>
<code>a[0]=a[1]=a[2]=0;</code>	<code>a[:3] = [0, 0, 0] (切片赋值)</code>

示例 2-8 解构赋值

```
# 多变量同时赋值
a, b, c = 1, 2, 3
print(a, b, c)           # 1 2 3

# 经典交换(不需要临时变量)
x, y = 10, 20
x, y = y, x
print(x, y)              # 20 10

# 从列表/元组/字符串解构
name, age = ("张三", 18)    # 等价于 name, age = "张三", 18
first, *rest = [1, 2, 3, 4] # *rest 收走其余
print(first, rest)         # 1 [2, 3, 4]
head, *middle, tail = "hello"
print(head, middle, tail)  # h ['e', 'l', 'l'] o

# 个数不匹配会报错
# a, b = [1, 2, 3]          # ValueError: too many values to unpack
```

2.6.2 walrus 操作符 := (3.8+)

`:=` 叫“海象操作符”,在**表达式内部**完成赋值,返回被赋的值。最典型的用途:避免“先调用再判断”写两遍;在推导式里复用计算结果。

示例 2-9 walrus 操作符

```
import re

# 场景 1:正则匹配,只写一次调用
text = "订单号:ORD-2026-0001"
if (m := re.search(r"ORD-\d+", text)):
    print("找到:", m.group(0))    # 找到: ORD-2026-0001

# 场景 2:逐行读取直到空(while 里最常用)
with open("data.txt", encoding="utf-8") as fh:
    while (line := fh.readline()):
        print(line.strip()[:20])

# 场景 3:推导式内复用计算结果
nums = [1, 4, 9, 16, 25]
squares = [y for x in nums if (y := x ** 0.5) > 3]
print(squares)                  # [4.0, 5.0]
```

工程建议:walrus 是好工具,别当玩具

`:=` 的价值是“消除重复计算”,过度使用会伤可读性。判据:去掉它后必须写两遍同一个昂贵调用(正则匹配、读文件、查库),就用;只为少写一行,别用。`while (line := fh.readline())` 是官方惯用法,放心用。

§2.7 条件表达式(三目的替代)

C 的 `?:` 在 Python 里写作 真值分支 `if 条件 else 假值分支`——顺序反直觉,但只此一种。它是**表达式**(有值),可嵌在任何需要值的地方;没有 C 的“语句版三目”。Python 的 `if` 是语句不能当表达式用,函数内“按条件返回不同值”时,条件表达式是首选。

示例 2-10 条件表达式

```
age = 20
stage = "成年" if age >= 18 else "未成年"
print(stage)                # 成年

# 嵌套:不推荐(可读性差),最多两层
score = 85
level = "优" if score >= 90 else ("良" if score >= 80 else "及格")
print(level)                # 良

# 在列表推导里当过滤器用
nums = [i if i % 2 == 0 else -i for i in range(6)]
print(nums)                 # [0, -1, 2, -3, 4, -5]

# 生产更常用:函数内提前返回代替嵌套
def desc(age):
    if age < 0:
        return "非法"
    return "成年人" if age >= 18 else "未成年人" # 返回条件表达式
```

§2.8 优先级与特色运算符

Python 的优先级从高到低(常用部分): `**` → 一元 `+` `-` `~` → `*` `/` `//` `%` → `+` `-` → `<<` `>>` → `&` → `^` → `|` → 比较 → `not` → `and` → `or` → 赋值/条件表达式。与 C 的关键差异: `**` 高于一元负号(前面已见),且**没有逗号表达式**、没有指针运算符、没有 `sizeof` 陷阱。

Python 独有运算符: `in` (成员判断)、`is` (身份判断)、`:=` (赋值表达式)、`@` (矩阵乘,科学计算用;也用于装饰器,见第12章)。

示例 2-11 优先级实战

```
# and/or 低于比较:下面的写法与直觉一致
x = 5
print(x > 0 and x < 10)      # True(先比较,再 and)

# 但布尔运算混合时要小心
a, b, c = 1, 0, 2
print(a or b and c)          # 1(and 优先于 or,先算 b and c)
print((a or b) and c)        # 2(加括号改变语义)

# in 成员判断:list 是线性查找,set/dict 是哈希查找
names = ["张三", "李四"]
print("张三" in names)       # True
big = {i: i for i in range(10000)}
print(9999 in big)           # True(dict 查键,O(1))

# 生产建议:复杂表达式一律加括号,别赌优先级
# 反例:if a and b or c and d  — 没人能一眼读懂
# 正例:if (a and b) or (c and d)
```

⚠ 误区 3:把 in 当 strstr 用,忘了它的复杂度

`x in list` 是 $O(n)$ 线性查找;在大列表里做十万次 `in` 就是 $O(n \cdot m)$ 灾难。生产做法:查找频繁的集合用 `set` 或 `dict` (哈希, $O(1)$ 平均)。改一行数据结构,性能提升几个数量级——这是 Python 优化里性价比最高的一招,务必养成"凡是要查重/查找,先想 `set`"的肌肉记忆。

本章速查表

主题	写法 / 结论
真除法	<code>a / b</code> 恒为 <code>float(7/2 == 3.5)</code>
整除	<code>a // b</code> 向下取整(<code>-7//2 == -4</code>)
取模	<code>a % b</code> 符号随除数(<code>-7%2 == 1</code>)
自增	没有 <code>++</code> ; <code>i += 1</code> ; <code>++i</code> 是正号不报错
位运算	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code> 同名同义;右移为算术右移
逻辑运算	<code>and</code> / <code>or</code> / <code>not</code> ;短路;返回操作数
默认值	<code>a or b</code> ;判 None 用 <code>x if x is not None else b</code>
链式比较	<code>0 <= x < 100</code> 等价 C 的 <code>x>=0 && x<100</code>
相等判断	<code>==</code> 比值; <code>is</code> 比身份; None 用 <code>is None</code>
浮点比较	用 <code>math.isclose(a, b)</code> 而非 <code>==</code>
walrus	<code>x, y = y, x</code> ; <code>while (line := f.readline())</code> (3.8+)
三目	<code>a if 条件 else b</code> (注意顺序!)
优先级	<code>**</code> > 一元 > 乘除 > 加减 > 移位 > 位与 > 比较 > <code>not</code> > <code>and</code> > <code>or</code>
成员判断	<code>x in 容器</code> ;set/dict 是哈希查找, $O(1)$

最小必会清单

- 能脱口而出 `7/2`、`7//2`、`-7//2`、`-7%2` 四个值
- 能解释为什么 `-2 ** 2` 是 -4,以及怎么写才是 4
- 能写出 `++i` 在 Python 里的真实含义与正确写法
- 能利用 `a or b` 写默认值,并说清与 `if a is None` 的适用边界
- 能说出 `==` 与 `is` 的区别,以及小整数缓存现象
- 能写出链式比较 `0 <= x < 100` 与 C 的等价形式
- 能用 `math.isclose` 做浮点相等比较
- 能写出 `x, y = y, x` 与 `first, *rest = lst`
- 能解释 walrus 解决什么问题,并写一个 `while (line := ...) 示例`
- 能写出三目表达式并注意顺序;知道查重用 `set` 而非 `list`

自测题

Q 1. 计算下列各式的值(先不运行,手算): `10 / 4`、`10 // 4`、`-10 // 4`、`-10 % 4`、`10 % -4`。

✓ 参考答案

2.5、2、-3、2、-2。验证:前四个满足 `-10 == (-3)*4 + 2`;第五个 `10 == (-3)*(-4) + (-2)`。模的符号永远与除数一致。

Q 2. `a = 3; b = 0; c = 5`,求 `a and b or c` 的值,并解释求值顺序。

✓ 参考答案

结果为 5。and 优先于 or:a and b 得 0 (a 真则继续,返回 b), 0 or c 得 5。(a and b) or c 结果不变;复杂布尔表达式必须靠括号控制。

Q 3. 为什么 `x == None` 应该写成 `x is None`? 举一个 `==` 会给出错误结果的例子。

✓ 参考答案

None 是单例,"是否为 None"是身份问题, is 的语义更精确;且自定义类型可能重载 `__eq__` 把 `x == None` 变成任意结果。例子:定义一个 `__eq__` 恒返回 True 的类,则 `obj == None` 为 True 而 `obj is None` 为 False。PEP 8 明确要求判 None 用 `is`。

Q 4. 用一行表达式把字符串 `"hello world"` 中的元音字母收集成列表(提示:列表推导 + `in`)。

✓ 参考答案

`[ch for ch in "hello world" if ch in "aeiou"]` → `['e', 'o', 'o']`。要点: `ch in "aeiou"` 是字符串成员判断($O(n)$ 但 n 只有 5);生产可用 `set("aeiou")` 提速。

Q 5. 写出与 C 代码 `result = (x > 0) ? x * 2 : -x;` 等价的 Python 表达式。

✅ 参考答案

`result = x * 2 if x > 0 else -x`。注意"真分支"写在 `if` 前;它是表达式,可用于赋值、传参、推导式。

Q 6. 下面的代码有什么问题? `count = get_count() or 0`,其中 `get_count()` 返回数据库记录条数。

✅ 参考答案

当业务要求区分"查询失败(None)"与"确实 0 条"时, `None or 0` 把两种情况合并了。正确: `c = get_count(); count = 0 if c is None else c`。教训:假值与 `None` 是不同的语义。

章末实验题:表达式求值计算器

实验 2:表达式求值计算器

实验目标:实现一个命令行计算器,读入"数字 运算符 数字"表达式,支持 `+` `-` `*` `/` `//` `%` `**`,正确处理负数、优先级(幂高于乘除高于加减)、除零与非法输入,并把 `and` `or` `is` `in` `walrus`/条件表达式等本章知识点融进实现。

实验要求:

- 任务 1:读入形如 `7 / 2`、`-7 // 2`、`2 ** 10`、`17 % 5` 的三段式表达式并正确求值(知识点:真除法/整除/取模/幂、负数语义、`strip` 清理)
- 任务 2:用 `in` 校验运算符合法性,非法运算符提示后重新输入(知识点:成员判断、短路)
- 任务 3:对 `//` 与 `%` 的除数为 0 给出友好报错;对非数字输入用 `float()` 转换并捕获 `ValueError` (知识点:类型转换、异常捕获)
- 任务 4:用条件表达式区分"结果是否为整数",整数显示为整、浮点保留 4 位小数(知识点:三目、格式化)
- 任务 5:统计已计算次数与最近一次结果,用 `walrus` 在循环条件里读取输入;支持 `quit` 退出(知识点:`walrus`、链式比较、`while`)
- 任务 6:每次运算后打印一行历史摘要,格式: [第N次] `7 / 2 = 3.5` (知识点:f-string 与解构)

实验环境:Python 3.10+,单文件 `calc.py` 即可,命令行运行。

第一步 循环先写骨架: `while (expr := input(...)) != "quit"`,体会 `walrus` 消除重复输入调用

第二步 析用 `expr.split()` 得到三个部分,长度不对时提示"格式:数字 运算符 数字"

第三步 算分发用 `dict` 映射运算符到 `lambda`(第4章会细讲函数与 `dict` 的结合,这里先用 `if/elif` 链即可)

第四步 出统一走一个 `fmt_result` 函数,内部用条件表达式判断整数显示

✅ 参考答案要点(代码分两段)

第一段:运算符分发表与解析。分发表用 `dict + lambda` ——这是 Python 替代 C 的 `switch` 语句的惯用法;解析与计算分离,非法输入在解析层被过滤。

```
# calc.py — 表达式求值计算器(覆盖第2章全部知识点)
import math

# 运算符分发表:键是运算符,值是"两个数的计算函数"
OPS = {
    "+": lambda a, b: a + b,
    "-": lambda a, b: a - b,
    "*": lambda a, b: a * b,
    "/": lambda a, b: a / b,          # 真除法,永远 float
    "//": lambda a, b: a // b,       # 向下取整
    "%": lambda a, b: a % b,         # 模:符号随除数
    "**": lambda a, b: a ** b,        # 幂
}

def parse_expr(line: str):
    """解析 '数字 运算符 数字' 为 (a, op, b);非法返回 None"""
    parts = line.split()
    if len(parts) != 3:
        print("格式应为: 数字 运算符 数字(如 7 / 2),运算符支持:",
              " ".join(OPS))
        return None
    a_str, op, b_str = parts          # 解构赋值
    if op not in OPS:                  # in 成员判断
        print(f"不支持的运算符: {op!r}")
        return None
    try:
        a = float(a_str)              # 类型转换
        b = float(b_str)
    except ValueError:
        print("请输入合法数字")
        return None
    return a, op, b

def fmt_result(value) -> str:
    """整数显示为整,浮点保留 4 位(条件表达式)"""
    return f"{value:.0f}" if value == int(value) else f"{value:.4f}"
```

第二段:主循环。walrus 在循环条件里完成读入与判断;链式比较做结果区间提示;短路保护除零。


```

def main() -> None:
    print("表达式计算器:输入 '数字 运算符 数字',quit 退出")
    count = 0
    last = None                                # 最近一次结果
    # walrus:循环条件里完成读入与判断,不再写两遍 input()
    while (line := input("> ").strip()) != "quit":
        parsed = parse_expr(line)
        if parsed is None:
            continue                            # 非法输入,重新来
        a, op, b = parsed
        if op in ("/", "%") and b == 0:          # 短路保护除法
            print("除数不能为 0")
            continue
        result = OPS[op](a, b)                  # 分发执行
        count += 1
        last = result
        if (a < 0 or b < 0) and op in ("/", "%"):
            print(f"    [提示] 负数{op}:向下取整,余数符号随除数")
        print(f"[第{count}次] {a:g} {op} {b:g} = {fmt_result(result)}")
        # 链式比较:结果范围提示
        print("    结果在 [0, 100) 区间内" if 0 <= result < 100
              else "    结果超出 [0, 100) 区间")
    if last is not None:
        ok = math.isclose(last, float(int(last)))
        print(f"共计算 {count} 次,最后结果 {last}(整数?{ok})")

if __name__ == "__main__":
    main()

```

设计说明:① 运算符分发表 OPS 让"加一个新运算符 = 加一行",这是 C 的 switch 在 Python 里的正统替代;② 解析与计算分离,非法输入在解析层被过滤,计算层只需专注数学;③ 负数整除提示语帮助学生现场巩固 `//` / `%` 的符号规则;④ 全部输出走 `fmt_result`,保证"3.0"显示成"3"——这是生产日志里最常见的格式化细节。

下一章预告:第3章 流程控制——`if` 的真值判定、`match` 结构匹配、`for` 遍历万物、`for-else` 与 `while-else` 的奇技、没有 `goto` 的世界。

第3章 流程控制

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 `if / elif / else` 的真值判定规则,与 C 的"非零即真"对比找差异
- 学会 `match` 语句(Python 3.10+)的结构匹配:值匹配、解构、守卫,并理解它为什么不是 C 的 `switch`
- 彻底告别 `for(i=0; i<n; i++)`,改用 `for x in iterable` 遍历 `list/dict/str/文件`,并掌握 `range / enumerate / zip` 三件套
- 掌握 `while` 的用法与 Python 没有 `do-while` 的替代写法
- 掌握 `break / continue` 与 Python 独门的 `for-else / while-else`
- 理解"没有 `goto`"的世界如何用提前返回与异常替代
- 写出符合 Pythonic 风格的循环:列表推导、生成器、`for-else` 查找惯用法

💡 从 C 到 Python:本章主题如何迁移

C 的流程控制由 `if/else`、`switch`、`for`、`while`、`goto` 统治;Python 对应 `if/elif/else`、`match` (3.10+)、`for...in`、`while`,**没有** `goto`,也没有 `do-while`。表面是语法换皮,底层是两种哲学。

第一,**条件判定从"非零"变成"真值"**。C 的 `if (x)` 问"x 是否非零";Python 的 `if x:` 问真值——0、`None`、空字符串、空列表、空字典、`False` 都是假,其余为真。这让代码更简洁(`if not items:` 直接判空),但也要求记住"哪些对象是假的",否则会写出 `if not scores:` 误杀"成绩为 0"的 bug(第2章预告过)。

第二,**循环从"计数"变成"遍历"**。C 的 `for(i=0; i<n; i++)` 是"用下标访问数组";Python 的 `for x in xs` 是"让迭代器逐个交出元素"。下标、边界、步长被迭代器协议封装(第6章细讲);代价是 `i += 2` 式跳跃遍历要显式写 `range(0, n, 2)`。

第三,**switch 进化成结构匹配**。C 的 `switch` 只能匹配整型常量、还有 `fallthrough` 陷阱;Python 的 `match` 能匹配任意对象、解构元组/列表/字典、带守卫条件,与 Rust 的 `match` 同源。它是新范式,初学先用它的 80% 能力(值匹配 + 解构)。

最后是 `goto`:C 用它做错误处理与跳出多重循环;Python 用 `try/except` (第9章)与"提前 `return` + 封装函数"替代。写 Python 时请忘掉 `goto`——你需要的一切都有更清晰的替代。

📦 前置知识自查(你会的 C 基础)

- `if/else if/else` 的嵌套与悬挂 `else` 问题
- `switch` 的 `fallthrough` 行为与 `break` 的必要性
- `for(i=0; i<n; i++)` 三表达式循环与边界条件
- `while` 循环与 `do-while` 的"先执行后判断"语义
- `break / continue`; `goto` 的错误处理模式

🚀 本章学习路线

1. **第一阶段(约 1 小时,分支)**:读 §3.1–§3.2,把真值表和 match 的几种模式在 REPL 里各敲一遍
2. **第二阶段(约 1 小时,循环)**:读 §3.3–§3.4,练习 range/enumerate/zip 组合遍历
3. **第三阶段(约 1 小时,控制转移)**:读 §3.5–§3.6,做查找类的 for-else 练习
4. **第四阶段(实验,约 1.5 小时)**:完成章末实验"菜单驱动系统 + 素数筛选",覆盖全章
5. **验收标准**:不看资料把 C 的 `for(i=0;i<n;i++) printf("%d",a[i]);` 改写为 Pythonic 版本,并能说出 match 与 switch 的三个差异

本章知识导图

- §3.1 if/elif/else 与真值判定(真值表、悬挂 else、if 表达式)
- §3.2 match 结构匹配(值匹配、通配、解构、守卫)
- §3.3 for 循环:遍历万物(range/enumerate/zip、可迭代对象)
- §3.4 while 循环与 do-while 的替代写法
- §3.5 break/continue/else:循环的"完成标志"
- §3.6 没有 goto:提前返回、标志位、异常替代
- §3.7 循环惯用法:推导式、生成器、无限循环对照

§3.1 if/elif/else 与真值判定

语法上 Python 的 if 与 C 有三个差异:①条件后是**冒号**且不写括号;②分支体靠**缩进**而非花括号;③多分支用 `elif`。缩进即块——初学时会被吐槽,但它消灭了 C 里花括号错位引发的一整类 bug,团队里也不再有"这个 else 属于哪个 if"的争论。

C 的写法	Python 的写法
<code>if (x > 0) { ... }</code>	<code>if x > 0: + 缩进块</code>
<code>else if (y > 0)</code>	<code>elif y > 0:</code>
<code>if (ptr) /* 非空指针 */</code>	<code>if ptr is not None:</code>
<code>if (arr_len > 0)</code>	<code>if items: (非空容器为真)</code>
<code>if (flag == 0)</code>	<code>if not flag:</code>

3.1.1 真值判定表

下面这张表决定了一切 `if` 的行为,背熟它比背语法重要得多:

为假(False 值)	为真(True 值)
False 、 None	True
0、0.0、0j	任何非零数
空字符串 ""	任何非空字符串
空容器: []、()、{}、set()	任何非空容器
自定义对象的 __bool__ 返回 False	其余一切对象

示例 3-1 if/elif/else 与真值

```
def grade_of(score: float) -> str:
    if score >= 90:          # 条件后冒号, 不写括号
        return "优"
    elif score >= 80:        # elif 取代 C 的 else if
        return "良"
    elif score >= 60:
        return "及格"
    else:
        return "不及格"

# 真值判定: 判空容器直接 if
items = []
if items:                   # 空列表为假
    print("有内容")
else:
    print("空列表")

name = input("姓名: ").strip()
if not name:                # 空输入为假 → not 为真
    print("姓名不能为空")
```

⚠ 误区 1: 把 C 的“非零即真”直觉带进 Python

C 里 `if (x)` 只问“x 是不是 0”; Python 问“x 的真值”。两个经典翻车: ① `if not scores:` 想表达“没有成绩”, 但 `scores = [0]` (一份 0 分) 时列表非空, 为真, 走不进分支——这其实是对的, 但很多 C 程序员会误写成 `if scores == []:` (可以但啰嗦); ② 判断“是否 None”必须写 `if x is None:`, 写 `if x:` 会把 0、空串一起判掉。规则: 判 None 用 `is None`, 判空用 `not`, 判零用 `== 0`。

§3.2 match 结构匹配(Python 3.10+)

`match` 语句在 3.10 引入, 基本语法:

示例 3-2 match 值匹配

```
def describe(status: str) -> str:
    match status:
        case "ready":
            return "就绪"
        case "running":
            return "运行中"
        case "done":
            return "已完成"
        case _:
            # 通配:等价于 default
            return f"未知状态: {status!r}"
```

```
print(describe("ready"))      # 就绪
print(describe("crashed"))    # 未知状态: 'crashed'
```

match 是"语句"不是表达式;C 的 switch 要求整型常量,
match 可以匹配任意对象与类型!

3.2.1 解构匹配与守卫

match 的真正威力在于**解构**:把"形状"写出来,Python 自动拆包并匹配。配合 `if` 守卫(guard)可以做条件分支。它是处理"按类型/结构分派"的正规军,替代 C 里"switch 枚举 + 手写结构体解析"的组合。

示例 3-3 match 解构与守卫

```
def handle(event):
    match event:
        case ("click", x, y):
            # 元组解构
            print(f"点击位置 ({x}, {y})")
        case ["move", *path]:
            # 列表解构:首元素 + 其余
            print(f"移动路径: {path}")
        case {"type": "login", "user": u}:
            # 字典解构
            print(f"用户 {u} 登录")
        case {"type": "input", "value": v} if len(v) > 0:
            print(f"输入: {v}")
            # 守卫:if 进一步约束
        case _:
            # 通配兜底
            print(f"未知事件: {event!r}")

handle(("click", 10, 20))      # 点击位置 (10, 20)
handle(["move", (0, 0), (5, 5)]) # 移动路径: [(0, 0), (5, 5)]
handle({"type": "login", "user": "alice"}) # 用户 alice 登录
handle({"type": "input", "value": "hi"})    # 输入: hi
```

⚠ 误区 2:把 match 当 switch,问"为什么 case 里不能放表达式"

C 的 `case 1+1:` 是常量表达式;Python 的 `case` 是**模式**——写 `case x:` 会无条件匹配并把 `x` 绑定为被匹配对象(等于通配!),这是新手最懵的行为。想匹配"计算出来的值",用守卫: `case _ if v == threshold:`。另外 `case 1 | 2:` 表示"1 或 2"(或模式),不是位或。3.10 之前的旧代码库没有 match,仍用 if/elif 链,新项目放心用。

§3.3 for 循环:遍历万物

Python 的 `for` 只能遍历可迭代对象(list、str、dict、set、文件、range、生成器.....)。C 的三种 for 用法里,"遍历数组"和"遍历字符串"被统一成 `for x in xs` ;"计数循环"交给 `range()` 。下表是本章最实用的对照:

C 的写法	Python 的写法
<code>for (i=0; i<n; i++) printf("%d", a[i]);</code>	<code>for v in a: (直接遍历值)</code>
<code>for (i=0; i<n; i++) printf("%d", a[i]); (需要下标)</code>	<code>for i, v in enumerate(a):</code>
<code>for (i=0; i<n; i+=2)</code>	<code>for i in range(0, n, 2):</code>
<code>for (i=n-1; i>=0; i--)</code>	<code>for i in range(n-1, -1, -1): 或 for v in reversed(a):</code>
遍历两个数组	<code>for x, y in zip(a, b):</code>

3.3.1 range / enumerate / zip 三件套

这三个函数覆盖了 C 程序员在 for 循环里 90% 的下标操作:`range(start, stop, step)` 生成整数序列(左闭右开); `enumerate(xs)` 边遍历边给下标; `zip(a, b)` 并行遍历多个序列,长度不等时按最短的截断(3.10+ 加 `strict=True` 可在长度不等时抛错)。它们都返回"惰性"的迭代器而不是真列表,内存占用恒定——这是与 C 的数组下标循环在资源模型上的又一差异。

示例 3-4 range/enumerate/zip

```
# range:左闭右开,步长可正可负
print(list(range(5)))           # [0, 1, 2, 3, 4]
print(list(range(2, 10, 3)))    # [2, 5, 8]
print(list(range(10, 0, -2)))   # [10, 8, 6, 4, 2]

# enumerate:需要下标时
names = ["张三", "李四", "王五"]
for i, name in enumerate(names, start=1): # 从 1 开始编号
    print(f"{i}. {name}")

# zip:并行遍历
subjects = ["语文", "数学", "英语"]
scores = [88, 95, 91]
for subj, score in zip(subjects, scores):
    print(f"{subj}: {score}")

# zip(strict=True)(3.10+):长度不一立刻报错,防静默截断
# for x, y in zip([1, 2, 3], [10, 20], strict=True):
#     pass                       # ValueError: zip() argument 2 is shorter

# 转字典:zip 两序列合成 dict,一行搞定
grade = dict(zip(subjects, scores))
print(grade)                     # {'语文': 88, '数学': 95, '英语': 91}
```

3.3.2 遍历 dict / set / 文件

示例 3-5 遍历各种容器

```
# 遍历 dict:默认遍历键;键值用 .items()
student = {"name": "韩梅梅", "age": 18, "active": True}
for key in student:
    print(key)                # name age active
for key, value in student.items():
    print(f"{key} = {value}")

# 遍历 set:顺序无保证
tags = {"py", "c", "py", "go"}
for tag in tags:
    print(tag)                # 无序输出

# 遍历字符串:逐字符
for ch in "Python":
    print(ch.upper(), end=" ") # P Y T H O N

# 遍历文件:按行流式,大文件也安全(第7章细讲)
with open("data.txt", encoding="utf-8") as fh:
    for line in fh:           # 文件本身可迭代!
        print(line.rstrip())
```

⚠ 误区 3:遍历时修改容器(迭代器失效)

C++ 里遍历 vector 时 erase 会迭代器失效;Python 同样:遍历 list 时 remove / append 会跳过元素或报 RuntimeError。正确姿势是**遍历副本**: for x in lst[:] 或 for x in list(lst);筛选新集合用列表推导 [x for x in lst if ok(x)] 或原地重写: lst[:] = [x for x in lst if ok(x)]。删除 dict 的键也一样: for k in list(d): 再删。

§3.4 while 循环与 do-while 的替代

while 与 C 完全同义:条件为真就继续。Python **没有** do-while (先执行一次再判断)——这是有意为之,因为 while True: + 末尾 break 表达力相同且意图更清晰。输入校验循环是经典场景:

示例 3-6 while 与输入校验

```
# do-while 的替代:先执行,再判断
def read_age() -> int:
    while True:                # 模拟 do-while 的骨架
        raw = input("年龄: ").strip()
        try:
            age = int(raw)
            if 0 < age < 150:    # 链式比较
                return age
            print("年龄须在 0~150 之间")
        except ValueError:
            print("请输入整数")

n = 10
while n > 0:
    n -= 2                    # 没有 n--, 写 n -= 2
    print(n, end=" ")        # 8 6 4 2 0

# 哨兵循环:读到特定值退出(walrus, 见第2章)
while (line := input("> ").strip()) != "quit":
    print(f"收到: {line}")
```

§3.5 break / continue / else

break 立即跳出本层循环; **continue** 跳到下一次迭代——与 C 一致。Python 的独门武器是 **for-else / while-else** : 循环**没有被 break 打断而自然结束**时执行 else 块。它专治"查找类"问题:C 里要设 **found** 标志位:

示例 3-7 for-else:查找惯用法

```
# 场景:找第一个满足条件的元素,找不到要报告
def find_prime(limit: int) -> int:
    """返回大于 limit 的最小素数;不存在时打印提示"""
    for n in range(limit + 1, limit + 1000):
        for d in range(2, int(n ** 0.5) + 1):
            if n % d == 0:
                break          # 内层 break:不是素数
        else:                  # 内层没被 break → n 是素数
            return n
    else:                      # 外层跑完没 return(实际到不了)
        print("范围内没有素数")
    return -1

print(find_prime(100))        # 101
print(find_prime(90))        # 97

# 等价于 C 的标志位写法:found=0; for(...){...; found=1; break;}
# for-else 就是内置了这个标志位
```


🚀 工程建议:for-else 别炫技,见注释三分

for-else 是 Python 独有的语法,C 程序员第一次见一定懵。团队协作时,写上"# 循环完整跑完才进 else"这类注释,或者干脆封装成函数 + 提前 return(上面的 `find_prime` 就是好例子:else 只在表达"没找到"时出现一次)。生产代码的黄金法则是 循环体越短越好、退出路径越显式越好。

§3.6 没有 goto 的世界

Python 没有 `goto`。C 里 `goto` 的三个经典用途,Python 各有更清晰的替代:

C 的 goto 用法	Python 的替代
错误处理: <code>goto cleanup;</code>	<code>try/except/finally</code> (第9章)+ <code>with</code> (第5章)
跳出多层嵌套循环	封装成函数, <code>return</code> 直接退出;或标志位
重试逻辑	<code>while True:</code> + <code>continue / break</code>

示例 3-8 跳出多层循环的正规姿势

```
# 需求:在二维表里找第一个负数
table = [
    [1, 2, -3],
    [4, 5, 6],
    [7, 8, 9],
]

# 姿势一:函数 + 提前 return(最推荐)
def find_neg(mat) -> tuple | None:
    for i, row in enumerate(mat):
        for j, v in enumerate(row):
            if v < 0:
                return (i, j)      # 多层循环一次跳出
    return None                    # 找不到

print(find_neg(table))            # (0, 2)
```

§3.7 循环惯用法:推导式与生成器

列表推导 [表达式 for 变量 in 可迭代对象 if 条件] 是 Pythonic 风格的代表作:C 里"建新数组 + for 循环 + if 判断 + append"四步,推导式一行完成。它读起来像数学集合记号 $\{x^2 \mid x \in S, x > 0\}$ 。注意它**返回新列表**(副本),不修改原对象——所以它天然规避了遍历修改的坑。

示例 3-9 列表推导与生成器

```

nums = [1, 2, 3, 4, 5, 6]

# C 风格(反例,别这么写):
squares = []
for n in nums:
    if n % 2 == 0:
        squares.append(n * n)

# Pythonic:一行
squares = [n * n for n in nums if n % 2 == 0]
print(squares)                # [4, 16, 36]

# 字典推导 / 集合推导
square_map = {n: n * n for n in nums}
print(square_map[3])          # 9

# 生成器表达式:不一次性建列表,内存友好
total = sum(n * n for n in range(1_000_000)) # 括号内不建列表
print(total)                   # 333332833333500000

# 双层循环推导:拍平二维结构(先外层后内层)
matrix = [[1, 2], [3, 4]]
print([v for row in matrix for v in row])    # [1, 2, 3, 4]

```

⚠ 误区 4:推导式里写复杂逻辑

推导式适合"一个表达式 + 一个条件"。一旦出现多个 if、嵌套循环超过两层、或需要 `if/else` 分支,就拆回普通循环。判断标准:别人读你代码时,能 3 秒内说出推导式在干什么吗?不能,就拆。这是可读性优先于简洁的工程原则,Python 官方社区对"过度推导"同样持反对态度。

本章速查表

主题	写法 / 结论
if 语法	if 条件: 冒号 + 缩进; elif 多分支; else 兜底
真值	假值: 0、None、False、空容器、空串; if items: 判非空
判 None	if x is None: ,绝不写 if not x: 判 None
match	3.10+: case _: 通配; case 1 2: 或; if 守卫
计数循环	range(n)、range(a, b, step) 左闭右开
下标+值	for i, v in enumerate(xs, start=1):
并行遍历	for x, y in zip(a, b, strict=True):
倒序遍历	reversed(xs) 或 range(n-1, -1, -1)
遍历 dict	d.items() / d.values(); for k in d 是遍历键
遍历文件	for line in fh: 流式按行
do-while	无; 用 while True: + 末尾 break
break/continue	与 C 一致, 只影响本层循环
for-else	循环未被 break 时执行 else; 查找惯用法
goto	没有; try/except、return、标志位替代
推导式	[f(x) for x in xs if 条件]; 字典/集合推导
遍历修改	禁止; 遍历副本 xs[:] 或改用推导式

最小必会清单

- 能默写真值表: 6 种假值分别是什么
- 能写出 if / elif / else 完整结构, 并说明为什么不用 else if
- 能写出 match 的值匹配、通配 case _: 、守卫 if 、或模式 1 | 2
- 能解释 "match 的 case 写变量名是无条件匹配"
- 能把 for(i=0; i<n; i++) 三种变形(需要下标/步长/逆序)改写成 Python
- 能用 enumerate 带起始编号、zip 并行遍历、dict(zip(...)) 构造字典
- 能写出 do-while 的替代模式与哨兵循环(while (line := ...))
- 能写出 for-else 的查找惯用法, 并解释 else 何时执行
- 能说出 Python 里 "跳出多层循环" 的两种姿势
- 能写出列表推导与字典推导, 并判断何时该拆回普通循环
- 能说出遍历 list/dict 时修改容器的后果与正确做法

自测题

Q 1. 下列表达式哪些为真? `bool(0.0)`、`bool("0")`、`bool([])`、`bool([None])`、`bool({})`、`bool(0j)`。

✓ 参考答案

`False`、`True` (非空字符串,哪怕内容是字符 "0")、`False`、`True` (有一个元素的列表)、`False`、`False`。规则:空容器/零值/`None`/`False` 为假,其余为真。

Q 2. 用 `for-else` 写一个函数,判断整数 `n` 是否为素数(不用内建模块),并解释 `else` 的位置。

✓ 参考答案

```
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            break          # 找到因子,跳出
    else:
        return True        # 完整跑完 → 没有因子 → 素数
    return False
```

`else` 挂在 `for` 上:循环没有被 `break` 打断时执行。注意 `range` 上限取到根号 `n` 即可,这是素数判定的经典优化。

Q 3. 写出与 C 代码等价的 Python: `for (i = 0; i < 10; i += 2) printf("%d ", i);`。

✓ 参考答案

`for i in range(0, 10, 2): print(i, end=" ")`, 输出 `0 2 4 6 8`。`range` 三参数分别对应起始、终止(不含)、步长。

Q 4. `match` 与 C 的 `switch` 有哪三个关键差异?

✓ 参考答案

① 匹配对象:`switch` 只能整型/枚举,`match` 可匹配任意对象、类型与结构;② 模式能力:`match` 支持解构、或模式 `|`、守卫 `if`;③ 无 `fallthrough`:每个 `case` 自动结束,不需要 `break`。另外 `case` 里写变量名是无条件绑定,不是常量比较。

Q 5. 用一行列表推导,从 `words = ["python", "c", "go", "java", "rust"]` 中选出长度大于 3 的单词,并转为大写。

✓ 参考答案

`[w.upper() for w in words if len(w) > 3]` → `['PYTHON', 'JAVA', 'RUST']`。结构:表达式 → `for` → `if`(过滤)。

Q 6. 遍历 `dict` 时删除部分键,直接 `for k in d: del d[k]` 会怎样?正确写法是什么?

✅ 参考答案

抛 `RuntimeError: dictionary changed size during iteration` (或行为不可靠)。正确写法:先取键副本——`for k in list(d):` 再删;或推导式重建 `d = {k: v for k, v in d.items() if 保留条件}`。列表同理,遍历 `lst[:]` 副本。

章末实验题:菜单驱动系统 + 素数筛选统计

📄 实验 3:文本菜单 + 素数筛选 + 数组遍历统计

实验目标:实现一个菜单驱动的命令程序,内置"素数筛选、数组统计、字符串分析"三个功能,强制用到 `if/elif`、`match`、三种循环、`break/continue`、`for-else`,并处理非法输入。

实验要求:

- 任务 1:主循环用 `while True` 打印菜单(1.素数筛选 2.数组统计 3.字符串分析 0.退出),用 `match` 分发功能(知识点:`while`、`match` 值匹配、通配)
- 任务 2:素数筛选功能:输入上限 `n`,用 `for...else` 找出 `2~n` 之间全部素数,每行输出 10 个(知识点:`for-else`、`range`、`break/continue`、格式化)
- 任务 3:数组统计功能:读入一串以逗号分隔的数字,统计个数、总和、平均值、最大值、最小值,负数与小数都要正确处理(知识点:类型转换、真值、内建函数 `sum/max/min`)
- 任务 4:字符串分析功能:输入一句话,统计其中字母、数字、空格、其他字符的个数,并用 `enumerate` 输出每个非空格字符的下标(知识点:遍历字符串、`enumerate`、真值)
- 任务 5:功能 3 用 `for i, v in enumerate(...)` 输出前三个元素的下标与值,展示"带下标的遍历"(知识点:`enumerate`)
- 任务 6:所有输入非法时给出提示并 `continue` 回主菜单;退出走 `break` 并用 `else` 输出"感谢使用"(知识点:`break/continue/while-else`)

实验环境:Python 3.10+(需要 `match`),单文件 `menu.py`。

第一步 写主循环骨架:`while True + match`,让 0 退出先通

第二步 素数筛选单独成函数,返回 `list`;输出时用 `enumerate` 或计数变量控制换行(每 10 个一行)

第三步 数组统计用 `split(",")` 拆分,列表推导 + `float()` 转换,非法数字用 `try/except` 捕获后返回空列表,主菜单根据"结果是否为空列表"判定是否打印统计(真值判定)

第四步 字符串分析用 `for ch in s:` 配合 `ch.isalpha() / ch.isdigit() / ch.isspace()`

✅ 参考答案要点(代码分两段)

参考实现约 100 行,分两段展示。第一段:素数筛选与数组统计。重点:for-else 判素数、推导式解析输入、enumerate 带编号输出、真值判定拦截空结果。

menu.py — 文本菜单驱动系统(覆盖第3章全部知识点)

```
def is_prime(n: int) -> bool:
    """for-else 判素数:循环没被 break 即为素数"""
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            break
    else:
        return True
    return False

def find_primes(limit: int) -> list:
    """返回 2~limit 内全部素数"""
    return [n for n in range(2, limit + 1) if is_prime(n)]

def parse_numbers(text: str) -> list:
    """'1,2.5,-3' -> [1.0, 2.5, -3.0];含非法项返回空列表"""
    parts = [p.strip() for p in text.split(",") if p.strip()]
    nums = []
    for p in parts:
        try:
            nums.append(float(p))      # 类型转换 + 异常捕获
        except ValueError:
            return []                  # 非法输入,整体作废
    return nums

def menu_sieve() -> None:
    raw = input("请输入上限 n: ").strip()
    if not raw.isdigit():              # 真值 + isdigit 校验
        print("请输入正整数")
        return
    primes = find_primes(int(raw))
    print(f"2~{raw} 之间共 {len(primes)} 个素数:")
    for i, p in enumerate(primes, start=1):  # enumerate 带编号
        print(f"{p:>4}", end="\n" if i % 10 == 0 else " ")
    if len(primes) % 10 != 0:
        print()

def menu_stats() -> None:
    text = input("请输入逗号分隔的数字: ")
    nums = parse_numbers(text)
    if not nums:                       # 空列表为假 -> 提示
        print("输入无效(需要数字,如: 1,2.5,-3)")
        return
    total = sum(nums)                  # 内建聚合函数
    print(f"个数: {len(nums)} | 总和: {total:.2f} "
          f"| 平均: {total / len(nums):.2f} "
          f"| 最大: {max(nums)} | 最小: {min(nums)}")
```

```
for i, v in enumerate(nums[:3]):    # 前三个元素的下标与值
    print(f" 第{i + 1}个元素 = {v}")
```

第二段:字符串分析与主菜单。match 分发替代 switch, case _ + continue 处理非法选项,while-else 收尾。

```
def menu_text() -> None:
    s = input("请输入一句话: ")
    alpha = digit = space = other = 0
    for ch in s:                    # 遍历字符串
        if ch.isalpha():
            alpha += 1
        elif ch.isdigit():
            digit += 1
        elif ch.isspace():
            space += 1
        else:
            other += 1
    print(f"字母 {alpha} | 数字 {digit} | 空格 {space} | 其他 {other}")
    for i, ch in enumerate(s):
        if not ch.isspace():
            print(f" 下标 {i}: {ch!r}")

def main() -> None:
    while True:                    # do-while 式主循环
        print("\n==== 工具箱 =====")
        print("1. 素数筛选    2. 数组统计")
        print("3. 字符串分析  0. 退出")
        choice = input("请选择: ").strip()
        match choice:              # match 分发(match 是语句)
            case "1":
                menu_sieve()
            case "2":
                menu_stats()
            case "3":
                menu_text()
            case "0":
                break                # 唯一出口
            case _:
                # 通配兜底
                print("无效选项,请重新输入")
                continue
    else:
        # while-else:循环没被 break 打断才执行;
        # 本程序 break 必被触发,else 不执行(演示语法)
        print("感谢使用,再见!")

if __name__ == "__main__":
    main()
```

设计说明:① 每个菜单功能独立成函数,主循环只做分发——这是"函数即模块"思想的雏形(第4章深入);② match 分发清晰表达意图,非法选项走 case _ 通配 + continue ;③ 解析函数对非法输入返回空列表,调用方用真值判定

拦截,数据流简单;④ 素数输出用 `enumerate` 的编号判断换行,避免维护独立计数变量;⑤ 注意 `match` 是语句不能"返回"值,所以分发函数各自 `return`,主循环只管调用——这是 Python 流程控制组织的通用模式。

下一章预告:第4章 函数与方法——"传对象引用"的传参语义、默认参数可变对象的经典坑、`*args/**kwargs`、闭包与晚期绑定、`lambda` 与高阶函数,函数是 Python 世界的第一公民。

第4章 函数与方法

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 `def` 定义、参数与返回值的完整语法,以及 `docstring` 规范
- **讲透"传对象引用"**:可变对象在函数内被修改会反映到外部,不可变对象不会;知道 `a = ...` 与 `a.append(...)` 的本质区别
- 掌握默认参数的经典坑:可变对象做默认值只初始化一次,正确姿势是 `None` 哨兵
- 掌握 `*args` / `**kwargs` 与关键字参数、`/` 与 `*` 分隔符(PEP 570/3102)
- 掌握多返回值(元组解构)、`lambda`、高阶函数(`map` / `filter` / `sorted` 的 `key`)
- 理解闭包与**晚期绑定**坑,知道循环变量捕获问题与修复
- 会用 `if __name__ == "__main__":` 组织可执行脚本

💡 从 C 到 Python:本章主题如何迁移

C 里函数是"函数",Python 里函数是**对象**——它可以被赋值给变量、塞进列表、作为参数传递、作为返回值,甚至可以给函数"附加属性"。C 用函数指针实现的回调,在 Python 里是日常语法;C 里"把函数传给函数"要小心翼翼写 `int (*cmp)(const void*, const void*)`,Python 直接写 `sorted(data, key=func)`。

最大的认知跨越在**参数传递**。C 是传值;想改外部变量,必须传指针。Python 是"传对象引用":参数拿到的是对象引用的**拷贝**——所以"重新给形参赋值"不影响外部(和传值一样),但"通过形参就地修改可变对象"会反映到外部(和传指针一样)。C 程序员需要同时丢弃两个直觉:①Python 没有"传引用给普通变量"这回事——函数内 `x = 新值` 只是让形参重新绑定;②Python 也不存在"值拷贝"——函数内 `list 参数.append(...)` 改的就是外面的那个对象。

第三个迁移点是**重载**。C 没有重载(除非 C11 `_Generic`);C++ 有重载。Python 没有重载:一个名字一个函数。替代方案是默认参数、`*args`、类型分派(`isinstance`)或干脆不同名字——实践中默认参数 + `None` 哨兵覆盖了 90% 的重载需求。

最后一个迁移点是闭包。C 的函数指针只能指向"静态的全局函数";Python 的函数可以**捕获创建它的作用域里的变量**,形成闭包。这让"工厂函数"成为可能:调用一次 `make_counter()` 得到一个带私有状态的函数。代价是晚期绑定坑:闭包捕获的是变量**本身**而非创建时的值——循环里生成的闭包全部共享同一个循环变量,本章会给出生产级修复。

📦 前置知识自查(你已会的 C 基础)

- C 函数的声明/定义/调用、参数与返回类型; `main(int argc, char *argv[])`
- 传值 vs 传指针: `void f(int x)` 与 `void f(int *p)` 的行为差异
- 函数指针与回调: `qsort` 的 `cmp` 参数
- 可变参数: `va_list` / `va_arg` (`printf` 的实现)
- `static` 函数与文件作用域; `errno` 错误返回约定

本章学习路线

1. **第一阶段(1.5 小时,语法):**读 §4.1–§4.3,重点做 §4.2 的"函数内修改是否影响外部"判断练习
2. **第二阶段(1.5 小时,参数与返回):**读 §4.4–§4.5,练 *args/**kwargs 与多返回值
3. **第三阶段(1.5 小时,一等函数与闭包):**读 §4.6–§4.8,复现晚期绑定坑并修复
4. **第四阶段(实验,1.5 小时):**完成章末实验"可配置统计管线"
5. **验收标准:**能不看资料回答"函数内 `lst.append(1)` 与 `lst = [1]` 对外部的影响分别是什么"

本章知识导图

- §4.1 函数定义与调用(def、docstring、类型标注)
- §4.2 参数传递:传对象引用(可变/不可变行为表)
- §4.3 默认参数与可变默认值经典坑
- §4.4 关键字参数、*args/**kwargs、参数分隔符
- §4.5 返回值:多返回值与 None 的语义
- §4.6 一等函数:lambda 与高阶函数
- §4.7 闭包与晚期绑定坑
- §4.8 递归与深度限制
- §4.9 入口与结构: `__name__ == "__main__"`

§4.1 函数定义与调用

语法要点: `def` 关键字、函数名、括号参数、冒号、缩进体。**没有返回类型声明**(返回值类型用标注表示,仅文档意义);没有函数原型/声明分离——定义即生效,但**调用必须发生在定义之后**(模块从上到下执行)。docstring(三引号字符串)是函数文档, `help(f)` 会显示它,生产代码必写。

示例 4-1 定义、docstring 与标注

```
def calc_avg(scores: list) -> float:
    """计算平均分。

    参数:
        scores: 数字列表(非空)
    返回:
        平均分;空列表返回 0.0
    """
    if not scores:
        return 0.0
    return sum(scores) / len(scores)

print(calc_avg([80, 90, 95]))      # 88.33333333333333
print(calc_avg([]))                # 0.0
# help(calc_avg)                  # 查看 docstring
# 注意:函数必须在调用前定义—模块按从上到下顺序执行
```

⚠ 误区 1:以为函数返回类型"强制"了返回值

-> float 只是标注,解释器不检查。返回 None 或字符串都"合法",直到运行时才可能炸。这与 C 的编译期类型检查完全不同——所以生产项目要么加 mypy(第14章),要么在函数开头用 assert /异常做防御性校验。标注的价值在文档与静态分析,不在运行时。

§4.2 参数传递:传对象引用(本章最重要)

精确语义只有一条:参数传递 = 把实参对象引用赋值给形参(等价于 param = arg)。由此推出全部行为:

函数内的操作	C 的等价	对外部的影响
x = 新对象 (给形参赋值)	传值: f(int x) 修改 x	无影响(形参重新绑定)
lst.append(1) (就地修改)	传指针: f(int *p) 修改 *p	有影响(对象被改)
s = s + "!" (str 拼接)	传值	无影响(不可变,产生新对象)
d["k"] = v (dict 写键)	传指针	有影响
n += 1 (int 运算)	传值	无影响(int 不可变)

示例 4-2 可变与不可变的行为对照

```
def demo(lst, n, s, d):
    lst.append(100)          # 就地修改 → 外部可见
    lst = [0, 0]             # 重新绑定 → 外部不可见(重要!)
    n = n + 1                # int 不可变 → 外部不可见
    s = s + "!"              # str 不可变 → 外部不可见
    d["new"] = 1             # dict 就地修改 → 外部可见

a, num, text, data = [1, 2], 10, "hi", {"k": 0}
demo(a, num, text, data)
print(a)                    # [1, 2, 100]      append 生效
print(num, text)            # 10 hi          不受影响
print(data)                 # {'k': 0, 'new': 1} 写键生效

# 一句话记忆:能就地改的(可变对象)就影响外部;
# 产生新对象的(不可变对象或重新赋值)就不影响外部
```

⚠ 误区 2:想"传引用"修改变量本身

C 程序员遇到"函数内改不了外部 int"时,第一反应是找引用语法。Python **没有**这种能力,也不该有:函数修改外部变量意味着隐藏耦合,是 bug 温床。正确姿势:①返回新值由调用方赋值;def f(x): return x + 1,然后 x = f(x);②把要改的东西装进可变容器(dict/list/对象属性);③或直接用一个类。生产代码里,函数要么返回结果,要么就地修改传入的可变对象,二选一,绝不混着来。

§4.3 默认参数与经典坑

默认参数语法 def f(a, b=10) 与 C++ 相同,但有一个 C++ 没有的坑:默认值在定义时求值一次,且被所有调用共享。用可变对象做默认值,等于让所有调用共用同一个列表——这是 Python 界排名前三的经典 bug。

示例 4-3 可变默认参数坑与修复

```
# 反例:全世界的 Python 初学者都会踩的坑
def add_item(item, bag=[]):          # 默认列表只创建一次!
    bag.append(item)
    return bag

print(add_item("苹果"))              # ['苹果']
print(add_item("香蕉"))              # ['苹果', '香蕉'] ← 上一次的残留!
# 不同调用者共享了同一个默认列表

# 正例:None 哨兵 + 函数内创建
def add_item_ok(item, bag=None):
    if bag is None:                  # 每次调用都新建列表
        bag = []
    bag.append(item)
    return bag

print(add_item_ok("苹果"))            # ['苹果']
print(add_item_ok("香蕉"))            # ['香蕉']

# 不可变默认值(数字/字符串/None)没有此问题,放心用
def greet(name, greeting="你好"):    # "你好" 是不可变 str
    return f"{greeting}, {name}"
```

⚠ 误区 3:以为默认参数“每次调用重新求值”

默认参数在 `def` 执行时求值一次(模块加载时),之后是同一个对象。可变的默认值(列表/字典/集合)被修改后,污染会保留给下一次调用。生产纪律:默认参数只允许不可变对象;需要可变就用 `None` 哨兵 + 函数内创建。这条规则比背一万个 API 都值钱。

§4.4 关键字参数、*args 与 **kwargs

Python 函数调用时可以用**关键字参数**按名字传参: `f(a=1, b=2)` ——调用顺序可以与定义顺序不同,这是 C 没有的能力。参数定义侧还可以用 `*args` (收位置参数的元组)与 `**kwargs` (收关键字参数的字典);调用侧用 `*iterable` 展开序列、`**dict` 展开映射。3.8+/3.10+ 的分隔符语法: `/` 前只能按位置传, `*` 后必须按关键字传。

示例 4-4 参数全家桶

```
def connect(host, port, timeout=30, *, use_ssl=True):
    """* 之后的参数必须按关键字传"""
    print(host, port, timeout, use_ssl)

connect("db.example.com", 5432)           # 位置 + 默认
connect("db.example.com", 5432, timeout=5) # 关键字覆盖默认
connect(host="h", port=1, use_ssl=False)   # 全关键字,顺序随意
# connect("h", 1, True)                   # TypeError: use_ssl 必须关键字

def log(level, *messages, **extra):
    """*messages 收任意多个位置参数,**extra 收任意关键字"""
    print(f"[{level}]", " ".join(str(m) for m in messages))
    for k, v in extra.items():
        print(f"    {k} = {v}")

log("INFO", "启动完成", "耗时 0.3s", user="alice", env="prod")

# 调用侧展开
params = {"host": "h", "port": 1}
connect(**params)           # 等价 connect(host="h", port=1)
nums = [1, 2, 3]
print(*nums)                # 1 2 3(等价 print(1, 2, 3))
```



工程建议:签名设计三原则

- ① 参数多于 4 个时,考虑把强相关的几个收进 `**kwargs` 或数据类(第14章);
- ② 布尔开关参数一律用关键字调用,调用处写 `send(data, retry=True)` 而不是 `send(data, 1, 0)`,否则没人知道 `True` 是谁;
- ③ `*args` / `**kwargs` 主要用于"转发"(写框架、装饰器,第12章),业务函数别滥用——丢失了签名可读性。

§4.5 返回值:多返回值与 None

C 的函数只能返回一个值,多值靠出参指针或结构体;Python 返回元组即可: `return a, b` (打包),调用方 `x, y = f()` (解包)。没有显式 `return` 的函数返回 `None`——它就是 C 的 `void` 加上"无值"标记。C 的错误返回约定(返回 `-1/errno`) 在 Python 里由**异常**取代(第9章);返回 `None` 表示"没有结果"是合法的数据语义。

示例 4-5 多返回值

```
def stats(nums: list) -> tuple:
    """返回 (总和, 个数, 平均值);空列表平均值为 None"""
    total = sum(nums)
    count = len(nums)
    avg = total / count if count else None
    return total, count, avg          # 打包成元组返回

total, count, avg = stats([10, 20, 30])  # 解包接收
print(total, count, avg)                # 60 3 20.0

# 丢弃不需要的值
_, n, _ = stats([1, 2, 3])
print(n)                                # 3

# C 的"出参"风格(不推荐):刻意不这么写
# def fill(buf): buf["x"] = 1  — 除非是就地修改已有容器
```

§4.6 一等函数:lambda 与高阶函数

函数是对象: `f = len`; `f([1,2])` 合法;函数可以放进 list/dict,可以传参、返回。lambda 是单表达式匿名函数,等价于 C 的回调函数指针——但写法短得多。对照关系:

C 的做法	Python 的做法
<code>qsort(a, n, sz, cmp)</code> 传比较函数指针	<code>sorted(a, key=func)</code> 传取键函数
<code>void (*cb)(int);</code> <code>register_cb(cb);</code>	<code>cb = lambda x: print(x); register_cb(cb)</code>
函数只能全局定义	函数可嵌套、可捕获变量(闭包)
回调里传状态要 <code>void* ctx</code>	闭包直接捕获,无需额外参数

Python 官方推荐的高阶函数风格:

- `sorted(data, key=func)` — 按 key 排序,取代 C 的 `qsort` + 比较回调
- `map(func, it)` / `filter(func, it)` — 批量映射/过滤(常被列表推导替代,但 `map` 惰性)
- `functools.reduce` — 归约(求积等;可读性差,慎用)
- 把函数存进 dict 做"分发表"(第3章实验已见)

示例 4-6 高阶函数与 lambda

```

users = [
    {"name": "李雷", "age": 18},
    {"name": "韩梅梅", "age": 17},
    {"name": "张三", "age": 22},
]

# key 参数:排序的"取键函数"(取代 qsort 的比较回调)
by_age = sorted(users, key=lambda u: u["age"])
print([u["name"] for u in by_age])      # 韩梅梅 李雷 张三

by_name = sorted(users, key=lambda u: u["name"])  # 字符串排序

# map/filter:惰性迭代器
ages = map(lambda u: u["age"], users)      # 不立即执行
print(list(ages))                          # [18, 17, 22]

adults = filter(lambda u: u["age"] >= 18, users)
print([u["name"] for u in adults])        # 李雷 张三

# 函数作返回值:工厂
def make_multiplier(k):
    def mul(x):
        return x * k
    return mul

double = make_multiplier(2)
print(double(21))                        # 42

```

工程建议:lambda 用不用

lambda 体只能有一个表达式——逻辑超过一行,立刻改用 `def` 并起个名字。生产判据:给这个 lambda 起名后,名字能否说明意图?能(`key=lambda u: u["age"]`),用;不能,写 `def`。另外排序的 `key` 函数调用次数是 $O(n \log n)$, 昂贵操作(如解析时间戳)请先缓存(`users.sort(key=cache)` 或提前构造键值对)。

§4.7 闭包与晚期绑定坑

闭包 = 函数 + 它捕获的外部变量。上面的 `make_multiplier` 就是闭包: `mul` 捕获了 `k`。C 的函数指针做不到这点——被指向的函数没有“记忆”。但闭包有一个著名陷阱:**晚期绑定**——闭包捕获的是变量本身,不是创建时的值。循环里创建的一批闭包,全部共享循环结束后的最终值:

示例 4-7 晚期绑定坑与修复


```
# 坑:循环里生成 3 个闭包,记住 i=0,1,2
funcs = []
for i in range(3):
    funcs.append(lambda: i)          # 捕获的是变量 i 本身!

print([f() for f in funcs])          # [2, 2, 2] ← 全是 2!

# 修复一:默认参数立即绑定值(推荐,简单)
funcs = []
for i in range(3):
    funcs.append(lambda i=i: i)      # i=i:默认参数在定义时求值
print([f() for f in funcs])          # [0, 1, 2]

# 修复二:工厂函数(更明确)
def make_fn(i):
    return lambda: i

funcs = [make_fn(i) for i in range(3)]
print([f() for f in funcs])          # [0, 1, 2]
```

⚠ 误区 4:以为闭包"拍快照"了创建时的值

闭包捕获的是**变量单元**而非值。在循环/回调场景(定时器、事件、线程池提交任务)里,所有闭包会看到循环变量的最终值。修复首选"默认参数绑定" `lambda i=i: i`;语义上它把当前值固化到函数自己的局部变量里。生产事故经典场景:for 循环里给按钮绑定回调,点哪个都是最后一个——记住这个坑,第9章线程池还会再见它。

§4.8 递归与深度限制

Python 支持递归,但有**深度限制**(默认 1000, `sys.getrecursionlimit()` 可查)。C 的递归受栈大小限制(通常几万层);Python 的每个调用帧开销大,1000 层就会抛 `RecursionError`。没有尾递归优化(PEP 622 之类一直未落地),不要指望编译器帮你。生产结论:深度可能超过几百的递归(树/图遍历通常没问题,链表求长等线性递归要小心),改写成迭代或用显式栈。

示例 4-8 递归与迭代改写

```

import sys
print(sys.getrecursionlimit())      # 1000(默认)

# 经典递归:目录树节点求和(深度可控,放心用)
def tree_sum(node):
    total = node.get("value", 0)
    for child in node.get("children", []):
        total += tree_sum(child)
    return total

# 线性递归:深度 = n,大 n 会炸
def fact(n):
    return 1 if n <= 1 else n * fact(n - 1)

# 改迭代:显式栈(深度任意)
def fact_iter(n):
    result = 1
    while n > 1:
        result *= n
        n -= 1
    return result

print(fact(10), fact_iter(10))      # 3628800 3628800
# sys.setrecursionlimit(10000)     # 可调,但只是推迟问题

```

§4.9 入口与结构: `__name__ == "__main__"`

C 程序的入口是 `main()`;Python 没有固定入口——模块顶层代码直接执行。习惯上把"运行时动作"放进 `main()`,并用 `if __name__ == "__main__":` 保护:

C 程序	Python 脚本
<code>int main(int argc, char **argv)</code>	<code>def main(): ...</code> (无强制签名)
运行即从 <code>main</code> 开始	顶层代码从上到下执行
—(无"导入"概念)	<code>if __name__ == "__main__": main()</code> 防导入副作用
<code>argc / argv</code>	<code>sys.argv</code> (第7章)

直接运行本文件时 `__name__` 是 `"__main__"`,被 `import` 时是模块名——于是同一个文件既能当脚本跑,又能被导入复用函数而不会误触发副作用。这是 Python 脚本组织的第一纪律。

示例 4-9 `main` 守卫

```
# tools.py
def fetch_data(url: str) -> str:
    return f"模拟数据:{url}"

def process(text: str) -> int:
    return len(text)

def main() -> None:
    data = fetch_data("https://example.com")
    print(f"处理结果:{process(data)}")

if __name__ == "__main__":
    main()          # python tools.py 时执行
# 被 import tools 时,main() 不会执行—这是关键
```

本章速查表

主题	写法 / 结论
定义	<code>def f(a, b=10) -> int:</code> 冒号 + 缩进;docstring 三引号
调用	位置参数、关键字参数 <code>f(a=1)</code> 、混用(位置在前)
传参语义	传对象引用:就地修改可变对象影响外部;重新赋值不影响
改外部变量	没有引用语法;用返回值或就地修改可变对象
默认参数	定义时求值一次;可变默认值用 <code>None</code> 哨兵
<code>*args</code>	收位置参数为元组; <code>print(*nums)</code> 调用侧展开
<code>**kwargs</code>	收关键字参数为字典; <code>connect(**params)</code> 展开
分隔符	<code>/</code> 前只能位置; <code>*</code> 后必须关键字(3.8+/3.10+)
多返回值	<code>return a, b + x, y = f(); _</code> 丢弃
无返回	返回 <code>None</code> ,等价 C 的 <code>void</code>
lambda	单表达式; <code>sorted(data, key=lambda u: u["age"])</code>
map/filter	惰性迭代器;多与推导式二选一
闭包	函数捕获外层变量;循环闭包有晚期绑定坑
晚期绑定修复	<code>lambda i:i: i</code> 或工厂函数
递归	默认上限 1000;线性递归改迭代或显式栈
入口	<code>if __name__ == "__main__": main()</code>

最小必会清单

- 能说出"传对象引用"的精确语义,并判断任意一段"函数内修改代码"是否影响外部

- 能解释可变默认参数 bug 的机制,并写出 None 哨兵修复版
- 能写出 *args/**kwargs 的定义与调用侧展开
- 能解释 / 与 * 分隔符的作用(PEP 570/3102)
- 能写出多返回值函数与解包调用
- 能用 key 参数实现按字典字段排序
- 能解释 lambda 捕获循环变量的坑,并给出两种修复
- 能写出闭包计数器/乘法器工厂
- 能说出递归上限默认值与 RecursionError 的应对
- 能写出 __name__ 守卫并解释 import 与直接运行的区别

自测题

Q 1. 写出下列代码的最终输出: `def f(a, lst=[]): lst.append(a); return lst`,依次调用 `f(1)`、`f(2)`、`f(3, [9])`。

✓ 参考答案

输出 `[1]`、`[1, 2]`、`[9, 3]`。前两次共享同一个默认列表(定义时创建),第二次看到第一次的残留;第三次显式传了新列表。修复: `def f(a, lst=None): lst = [] if lst is None else lst`。

Q 2. 函数 `def g(x): x.append(1); x = [9]; return x`,调用 `a = [0]; r = g(a)` 后, `a` 与 `r` 分别是什么?

✓ 参考答案

`a == [0, 1]` (append 就地修改,外部可见); `r == [9]` (重新绑定只影响形参)。这题是传参语义的照妖镜:append 改对象,赋值换标签。

Q 3. 用 lambda + sorted 按字符串长度从短到长排序 `words = ["python", "c", "rust", "java"]`。

✓ 参考答案

`sorted(words, key=lambda w: len(w))` → `['c', 'rust', 'java', 'python']`。要点:key 是"取键函数",不是比较函数——这是与 `qsort` 比较回调的本质区别。

Q 4. 为什么下面的代码输出 `[2, 2, 2]`?给出修复。 `funcs = [lambda: i for i in range(3)]; print([f() for f in funcs])`

✓ 参考答案

晚期绑定:lambda 捕获的是循环变量 `i` 本身,调用时 `i` 已是最终值 2。修复: `[lambda i=i: i for i in range(3)]` 或 `[make(i) for i in range(3)]` 其中 `def make(i): return lambda: i`。

Q 5. 用 *args 写一个函数 `join_all(sep, *parts)`,把 `parts` 用 `sep` 连接后返回。

✅ 参考答案

```
def join_all(sep, *parts):
    """用 sep 连接任意多个 parts; sep 是普通位置参数"""
    return sep.join(parts)

print(join_all("-", "a", "b", "c"))    # a-b-c
print(join_all(" ", "Python", "3.10")) # Python 3.10
print(join_all("+", *[1, 2, 3]))        # 1+2+3(调用侧展开)
```

Q 6. 一个函数既有可变默认参数又有递归,深度 2000 的递归会怎样?如何验证与应对?

✅ 参考答案

抛 `RecursionError: maximum recursion depth exceeded` (默认上限 1000)。验证: `sys.getrecursionlimit()`; 应对: 改写为迭代/显式栈, 或万不得已 `sys.setrecursionlimit(5000)` (只是推迟, 且可能触发 C 栈溢出崩溃, 生产慎用)。

章末实验题: 可配置统计管线

📁 实验 4: 可配置的数组统计/过滤管线

实验目标: 用函数式风格(回调、闭包、高阶函数、多返回值全覆盖)实现一个数据统计管线: 一段数字经过“过滤 → 变换 → 统计”三步, 每一步都可配置。

实验要求:

- 任务 1: 写 `filter_data(data, keep)`, `keep` 是谓词回调(如“只保留正数”), 返回新列表且不修改原列表(知识点: 传对象引用、回调、列表推导)
- 任务 2: 写 `map_data(data, transform)`, `transform` 是变换回调(如平方、放大倍数)(知识点: 高阶函数)
- 任务 3: 写 `make_range_checker(lo, hi)` 闭包工厂, 返回“判断 `x` 是否在 `[lo, hi]`”的谓词; 验证不同工厂实例互不干扰(知识点: 闭包、工厂)
- 任务 4: 写 `make_scaler(k)` 闭包工厂, 返回乘以 `k` 的变换函数; 同时演示晚期绑定坑, 用 `lambda k=k`: 修复后给三个缩放器 `k=1, 2, 3` 分别映射(知识点: 晚期绑定与修复)
- 任务 5: 写 `statistics(data)` 返回 (总和, 个数, 平均值, 最大, 最小) 五元组, 空列表返回 `None` 平均(知识点: 多返回值)
- 任务 6: 主流程用 `if __name__ == "__main__":` 驱动: 过滤 → 变换 → 统计 三步, 每步打印输入/输出; 默认参数全部用 `None` 哨兵风格设计(知识点: `main` 守卫、默认参数)

实验环境: Python 3.10+, 单文件 `pipeline.py`。

第一步 写三个工厂函数(`make_range_checker`/`make_scaler`), 它们返回闭包——先跑通“闭包有记忆”的直觉

第二步 `filter_data` 与 `map_data` 内部用列表推导实现, 保持“不修改入参”的纯函数风格

第三步 统计函数返回五元组, 调用处用 `total, count, avg, mx, mn = ...` 解包

第四步 后用守卫把三阶段串起来, 每个阶段打印前后数据, 肉眼验证

✅ 参考答案要点(代码分两段)

第一段:纯函数与闭包工厂。核心设计:数据流单向(入参只读、返回新列表),工厂闭包携带私有状态——这是函数式风格的骨架。

```
# pipeline.py — 可配置统计管线(覆盖第4章全部知识点)

def filter_data(data: list, keep) -> list:
    """按谓词 keep 过滤,返回新列表,不修改入参"""
    return [x for x in data if keep(x)]

def map_data(data: list, transform) -> list:
    """按变换 transform 映射,返回新列表"""
    return [transform(x) for x in data]

def make_range_checker(lo, hi):
    """闭包工厂:返回"判断 x 是否在 [lo, hi]"的谓词"""
    def keep(x):
        return lo <= x <= hi      # 链式比较
    return keep

def make_scaler(k):
    """闭包工厂:返回乘以 k 的变换函数"""
    def scale(x):
        return x * k
    return scale

def statistics(data: list) -> tuple:
    """多返回值:返回 (总和, 个数, 平均值, 最大, 最小)"""
    total = sum(data)
    count = len(data)
    avg = total / count if count else None
    return total, count, avg, max(data, default=None), min(data, default=None)
```

第二段:主流程。演示晚期绑定坑与修复——循环里生成三个缩放器,不修复则全部按最后一个 k 缩放。

```

def main() -> None:
    raw = [3, -1, 7, 0, 4, -5, 9, 2]

    # 1. 过滤:闭包工厂产生谓词
    keep_positive = make_range_checker(0, 10**9)
    step1 = filter_data(raw, keep_positive)
    print(f"过滤后: {step1}")

    # 2. 变换:演示晚期绑定坑
    # 反例:三个缩放器共享循环变量 k,全变成 x3
    bad = [make_scaler(k) for k in (1, 2, 3)]
    print(f"坑: {[f(10) for f in bad]}")          # [30, 30, 30]

    # 修复:默认参数立即绑定
    def make_scaler_fixed(k, _k=k):
        return lambda x: x * _k

    ok = [make_scaler_fixed(k) for k in (1, 2, 3)]
    print(f"修复: {[f(10) for f in ok]}")        # [10, 20, 30]

    # 3. 变换:平方后取正数区
    step2 = map_data(step1, lambda x: x * x)
    step3 = filter_data(step2, keep_positive)
    print(f"平方后: {step2}")

    # 4. 统计:多返回值解包
    total, count, avg, mx, mn = statistics(step3)
    print(f"个数={count} 总和={total} 平均={avg} "
          f"最大={mx} 最小={mn}")

    # 5. 空列表语义:平均值为 None
    print(statistics([]))                      # (0, 0, None, None, None)

if __name__ == "__main__":
    main()

```

设计说明:① 管线各阶段是纯函数(只读入参、返回新列表),可以任意组合、单独测试——这是函数式风格的工程收益;② 晚期绑定修复用了"默认参数立即绑定"技巧,并在实验里同时演示"坑"与"修复",读代码的人一眼看懂两种行为;③ 统计函数返回五元组,主流程一次性解包,避免了 5 个全局变量;④ main 守卫保证本文件既可 `python pipeline.py` 运行,也可被导入复用 `filter_data` 等函数。

下一章预告:第5章 内存管理——引用计数与分代 GC 的工作原理、gc 模块、循环引用与 `__del__` 的坑、with 上下文管理器,以及"在 GC 语言里内存泄漏怎么发生"。

第5章 内存管理

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 Python 的"一切皆堆对象":没有 C 的栈上变量,所有对象都在堆上由引用计数管理
- 掌握**引用计数**的工作原理:绑定/解绑/函数传参/容器操作如何改变计数
- 掌握**分代 GC**:什么时候触发、能做什么(循环引用)、不能做什么(及时释放)
- 掌握循环引用与 `__del__` 的经典坑,学会用 `weakref` 打破循环
- 掌握 `with` 上下文管理器:内置用法与自定义 `__enter__` / `__exit__`、`contextlib`
- 理解"GC 语言里内存泄漏怎么发生":全局容器、缓存、闭包、循环引用
- 会用 `tracemalloc` 定位内存增长,养成资源释放的生产纪律

💡 从 C 到 Python:本章主题如何迁移

C 的内存模型是"栈 + 堆 + 静态区",程序员用 `malloc` / `free` 手动管理堆,用 `calloc` / `realloc` 调整,还要小心悬垂指针、重复释放、内存泄漏。Python 把这一切拿走:解释器为每个对象在**堆上**分配内存,用**引用计数**跟踪"还有谁在用它",计数归零立即回收(`__del__` 与内存释放都在此刻发生);堆内存不足时,分代垃圾回收器清扫"互相引用但没有外部引用"的对象环。这就是 CPython 的"引用计数 + 分代 GC"双轨制。

这套设计的直接后果:**你几乎不会"泄漏"内存,也几乎不能"控制"释放时机**。C 里 `free(ptr)` 是精确的;Python 里"什么时候释放"取决于引用什么时候消失——可能立即(计数归零),可能延迟(在 GC 环里),可能永久(还被全局引用)。所以生产调优的姿势完全不同:C 是"找漏 free",Python 是"确保引用图不膨胀"。

另一个大迁移是**资源释放**。C 里文件句柄、锁、网络连接靠手动 `fclose` / `unlock` / `close`,漏一次就泄漏一次。Python 的 `with` 语句是 RAII 的 Python 版:进入时 `__enter__`,退出时(无论正常、`return`、还是异常) `__exit__` 必然执行。C++ 程序员会心一笑,这是 RAII 的思想;C 程序员需要记住:打开资源的地方,立刻用 `with` 包起来,没有例外。

最后,GC 语言也有泄漏——只是形态不同:全局容器只增不减、缓存永不淘汰、闭包意外捕获大对象、回调列表不断累积。本章实验会复现并修复一个真实的"泄漏"。记住结论:在 Python 里排查内存问题,先画引用图,再谈优化。

📦 前置知识自查(你已会的 C 基础)

- `malloc` / `calloc` / `realloc` / `free` 的契约与内存泄漏、悬垂指针、双重释放
- 栈帧与局部变量的生命周期;函数返回后栈上变量失效
- `fopen` / `fclose` 等资源句柄的手动管理
- 静态/全局变量的生命周期(程序全程存活)
- 指针别名:两个指针指向同一块内存

🚀 本章学习路线

1. **第一阶段(1.5 小时,引用计数)**:读 §5.1–§5.2,用 `sys.getrefcount` 亲手验证计数变化
2. **第二阶段(1 小时,GC)**:读 §5.3–§5.4,复现循环引用与 `__del__` 坑,用 `weakref` 修复
3. **第三阶段(1.5 小时,资源管理)**:读 §5.5,写自定义上下文管理器
4. **第四阶段(实验,1.5 小时)**:完成章末实验"泄漏复现与修复"
5. **验收标准**:能解释"为什么 `del x` 之后内存不一定立刻归还"与"弱引用是什么"

本章知识导图

- §5.1 内存模型:栈/堆/静态区 vs 一切皆堆对象
- §5.2 引用计数:`getrefcount`、绑定/解绑、`del`
- §5.3 分代 GC:循环引用、`gc` 模块、阈值
- §5.4 `__del__` 的坑与 `weakref` 弱引用
- §5.5 `with` 上下文管理器与 `contextlib`
- §5.6 泄漏的四种形态与 `tracemalloc` 排查
- §5.7 生产实践:拼接性能、对象复用、内存纪律

§5.1 内存模型:一切皆堆对象

C 里 `int x = 5`; 把 5 放在栈上; `int *p = malloc(...)` 在堆上。Python 里**所有对象都在堆上**——包括整数、字符串这些"小东西"。名字(变量)只是指向堆对象的引用,它们存在于模块/函数的命名空间中。下表是内存模型对照:

C	Python
栈上局部变量,函数返回自动失效	名字绑定到堆对象;作用域结束,名字消失,引用计数减一
<code>malloc / free</code> 手动管理堆	引用计数归零立即回收;循环引用由 GC 回收
静态/全局变量全程存活	模块级名字全程存活——它们指向的对象"永不释放"
指针别名需小心	引用即别名,处处都是(第1章 <code>b = a</code>)
未初始化内存是垃圾值	没有未初始化对象;对象创建即完整初始化

★ 重点:名字与对象分离

名字 = 引用,对象 = 堆上的实体。内存的"生死"由对象的引用计数决定,不由名字决定。`del x` 只是删除名字;对象是否被回收,取决于还有没有别的名字/容器引用它。

§5.2 引用计数

CPython 对每个对象维护一个计数:谁引用它,计数加一;引用消失,计数减一;归零,立即释放内存。哪些操作会改变计数?

- 赋值 `y = x`、传入函数、放进 list/dict、作为返回值——计数 +1
- `del x`、函数返回、从容器删除、作用域结束——计数 -1
- 注意: `sys.getrefcount(x)` 本身会临时 +1,所以结果比直觉多 1

这与 C 的堆管理形成最鲜明的对照——C 靠人,Python 靠机制:

C(malloc/free)	Python(引用计数)
申请/释放时机由程序员决定	由"最后一个引用消失"自动决定
漏 free 或重复 free 都是 bug	没有"重复释放";漏释放表现为引用还活着
悬垂指针:free 后仍持有地址	无悬垂指针:对象没了名字/引用也不存在
free 立即归还,时机精确	计数归零立即归还;循环引用等 GC,时机不定
崩溃通常发生在"错误的地点和时间"	错误通常表现为"内存曲线缓慢上升"

示例 5-1 观察引用计数

```
import sys

x = [1, 2, 3]          # 对象创建,1 个引用(x)
print(sys.getrefcount(x)) # 2 = 真实 1 + getrefcount 自己临时 +1

y = x                  # y 也引用 → 计数 2
print(sys.getrefcount(x)) # 3

lst = [x]              # 列表持有引用 → 3
print(sys.getrefcount(x)) # 4

del y                  # 少一个引用 → 2
print(sys.getrefcount(x)) # 3

lst.clear()            # 列表不再引用 → 1
print(sys.getrefcount(x)) # 2

del x                  # 最后一个引用消失 → 对象立即释放
# print(sys.getrefcount(x)) # NameError: 名字都不存在了

# 小整数和短字符串有缓存,别拿它们测计数(会误导)
```

⚠ 误区 1:以为 del 立即释放内存

del x 只删名字。对象还活着,直到**最后一个引用**消失(或被 GC 从循环中回收)。函数里的局部变量在函数返回时统一减引用;闭包捕获的变量活得比函数长(第4章);全局变量活到进程结束。判断"何时释放"的唯一方法:数引用——这也是 weakref (§5.4)存在的原因。

§5.3 分代 GC:专治循环引用

引用计数解决不了**循环引用**:A 引用 B,B 引用 A,但外面没人引用它们——计数永远 ≥ 1 ,永不归零。分代 GC(标记-清除 + 分代)定期扫描,找出这种"孤岛"并回收。要点:

- **什么时候触发**:分配次数超过阈值时(默认 700/10/10,三代); gc.collect() 手动触发
- **能做什么**:回收循环引用;unreachable 对象若定义了 `__del__`,会被移入 `gc.garbage`
- **不能做什么**:不能按你的时间表回收——它不是引用计数那种"立即"语义
- 只有**容器**(list/dict/set/对象实例)参与 GC;int/str 不参与(它们造不出循环)

示例 5-2 循环引用与手动回收

```
import gc

class Node:
    def __init__(self, name):
        self.name = name
        self.other = None

def make_cycle() -> None:
    a = Node("A")
    b = Node("B")
    a.other = b          # A → B
    b.other = a          # B → A:形成环,函数返回后无人引用

gc.disable()            # 先关掉自动 GC, 观察
make_cycle()
print("回收前:", len(gc.get_objects()))    # 环还在
n = gc.collect()        # 手动回收
print(f"回收了 {n} 个对象")    # 环被标记-清除回收
print("回收后:", len(gc.get_objects()))
gc.enable()              # 生产代码千万别留 gc.disable()

# 生产注意:gc.get_objects() 开销大,只用于调试
```

⚠ 误区 2:依赖 GC 来"及时"释放资源

文件句柄、锁、连接这类资源**不能等 GC**:循环引用里的对象可能要等很多轮 GC 才被清扫,句柄就泄漏了。而且 `__del__` 方法在 GC 回收时的调用时机不可预测。生产铁律:资源释放永远用 `with / try-finally` 显式控制 (§5.5),GC 只负责内存,不负责资源。另一个极端:不要没事调 `gc.collect()`,它是全停顿操作,在服务里会引发卡顿。

§5.4 `__del__` 的坑与 `weakref` 弱引用

`__del__` 是"对象被回收前"的钩子,但有三宗罪:①时机不可预测(引用计数归零时立即,但循环引用里要等 GC);②循环引用里它可能导致对象永远留在 `gc.garbage`;③解释器退出时全局变量不保证调用。C 程序员会把它当析构函数用——**不要**。析构逻辑的正确位置是上下文管理器 (§5.5)或显式 `close()` 方法。

弱引用(`weakref.ref`)不增加引用计数:目标存活时,弱引用可访问;目标被回收,弱引用返回 `None`。它专治两件事:①打破循环引用;②缓存(C 的"缓存但允许淘汰")。

示例 5-3 `__del__` 的坑与 `weakref`

```

import gc
import weakref

class Resource:
    def __init__(self, name):
        self.name = name
        self.peer = None

    def __del__(self):          # 坑:依赖它的时机就是错的
        print(f"{self.name} 被回收")

r1, r2 = Resource("r1"), Resource("r2")
r1.peer, r2.peer = r2, r1      # 循环引用
del r1, r2                    # 两个对象计数都 >= 1
print("--- 手动 GC ---")
gc.collect()                  # 这里才回收;若 __del__ 有副作用,顺序不定

# weakref:不增加计数,目标死了就是 None
class Cache:
    pass

obj = Cache()
ref = weakref.ref(obj)        # 弱引用
print(ref() is obj)           # True:目标活着
del obj
print(ref())                   # None:目标已回收

# 弱引用打破循环:one 只弱引用 two
class Partner:
    pass

one, two = Partner(), Partner()
one.other = weakref.ref(two)   # 弱引用 → 不构成环
two.other = one
del two
print(one.other())             # None:two 已被回收(计数归零)

```

工程建议:缓存的正确姿势

全局缓存 (CACHE = {}) 会让所有被缓存对象永久存活——这本身是一种“泄漏”。两个对策：① `weakref.WeakValueDictionary`：对象被别处释放后自动从缓存消失，缓存永不膨胀；② `functools.lru_cache` (第12章)自带大小上限。生产里，“缓存必须有限”是内存纪律的第一条。

§5.5 with 上下文管理器

`with` 语句 = C++ RAII 的 Python 版：**进入执行 `__enter__`，退出(正常/异常/return)必然执行 `__exit__`**。文件、锁、连接、临时目录……凡是有“用完必须关”语义的对象，都用 `with`。它等价于：

```
# 下面的 with 语句:
with open("f.txt", encoding="utf-8") as fh:
    data = fh.read()

# 等价于 try/finally(别自己写,with 更清晰):
fh = open("f.txt", encoding="utf-8")
try:
    data = fh.read()
finally:
    fh.close()
```

资源释放的 C/Python 对照——C 靠人肉记住,Python 靠语法保证:

C(手动释放)	Python(with 上下文)
<code>FILE *fp = fopen(...); ... fclose(fp);</code>	<code>with open(...) as fp: ...</code>
pthread_mutex_lock/unlock 成对	<code>with lock:</code> 解锁必然执行
中间 return/异常 → 漏释放	return/异常 → <code>__exit__</code> 依然执行
资源计数全靠审查纪律	资源计数可由代码评审强制("一律 with")
close 后再次使用是 UB	退出 with 后访问资源对象同样报错(closed)

5.5.1 自定义上下文管理器

自己写资源类时,实现 `__enter__` / `__exit__`;只想包一层逻辑(计时、加锁)时,用 `@contextmanager` 装饰器把生成器函数变成上下文管理器——`yield` 前后分别是进入/退出逻辑。

示例 5-4 自定义上下文管理器

```
# 方式一:类实现 __enter__/___exit__
class Timer:
    """测量 with 块执行耗时"""

    def __enter__(self):
        import time
        self.start = time.perf_counter()
        return self          # as 子句拿到它

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        cost = time.perf_counter() - self.start
        print(f"耗时 {cost * 1000:.2f} ms")
        return False         # False:异常继续向外抛

with Timer():
    total = sum(range(1_000_000))

# 方式二:@contextmanager 生成器版
from contextlib import contextmanager

@contextmanager
def timed(name):
    import time
    start = time.perf_counter()
    try:
        yield                # with 块体在这里执行
    finally:
        cost = time.perf_counter() - start
        print(f"[{name}] 耗时 {cost * 1000:.2f} ms")

with timed("排序"):
    data = list(range(1000, 0, -1))
    data.sort()

# 内置常用:contextlib.suppress / closing / ExitStack
from contextlib import suppress, closing, ExitStack
```

⚠ 误区 3:忘记 with 或手动 open/close

`fh = open(...); ...; fh.close()` 是 C 时代思维——中间任何一行抛异常,close 就不执行,句柄泄漏。规矩: **open 的一瞬间就用 with 包起来**;多资源用 `with A() as a, B() as b:` 或 `ExitStack`。同理,锁、连接、临时文件全部用 with。代码评审看到"裸 open 后 5 行再 close",直接打回。

§5.6 泄漏的四种形态与 tracemalloc

GC 语言也会"泄漏",但形态不同——本质都是"引用图只增不减"。四种典型:

- **全局容器累积**:日志列表、请求记录 append 后从不清理

- **缓存无上限**:字典缓存键不断增加
- **闭包捕获**:回调闭包意外捕获大对象,生命周期被拉长
- **事件/回调注册未注销**:观察者列表只加不减

tracemalloc 是标准库内存追踪器,可快照对比定位增长来源:

示例 5-5 tracemalloc 定位内存增长

```
import tracemalloc

tracemalloc.start()          # 开始追踪(生产调试时才开)

# 模拟泄漏:全局列表只增不减
leaked = []
for i in range(1000):
    leaked.append({"id": i, "payload": "x" * 1000})

snapshot = tracemalloc.take_snapshot()
top = snapshot.statistics("lineno")    # 按代码行统计
for stat in top[:3]:
    print(stat)                  # 显示"哪一行分配了多少"

# 输出示例(行号视实际文件):
# /path/demo.py:10: size=1000 KiB, count=1000, average=1.0 KiB
# → 定位到第 10 行,就是泄漏源头

tracemalloc.stop()

# 对比两次快照更实用:
# s1 = take_snapshot(); 运行可疑代码; s2 = take_snapshot()
# s2.compare_to(s1, 'lineno') 列出增长最大的行
```

§5.7 生产实践:性能与纪律

三个高频内存/性能话题:字符串拼接、对象复用、内存视图。

示例 5-6 拼接性能与对象复用

```
# 1) 字符串拼接:+= 是  $O(n^2)$ ,join 是  $O(n)$ 
parts = [f"行{i}" for i in range(10000)]
text = ""
for p in parts:
    # 反例:每次 + 都新建字符串并拷贝
    text += p + "\n"

text = "\n".join(parts)    # 正例:一次性拼接(生产标准)

# 2) 大量小对象复用:数据类/元组优先于 dict
import timeit
t_dict = timeit.timeit(
    '{"x": 1, "y": 2}', number=100000)
t_tup = timeit.timeit(
    '(1, 2)', number=100000)
print(f"dict 构造 {t_dict:.3f}s vs 元组 {t_tup:.3f}s")
# 元组显著更快;百万级循环里,打包用元组

# 3) 大文件不整读:逐行/分块处理,避免一次性内存峰值
# (第7章详述;这里记住:能流式就不要全量)
```



工程建议:内存纪律清单

① 资源用 with,不用裸 open;② 缓存必须有限(WeakValueDictionary/lru_cache 上限);③ 全局容器谨慎 append——考虑 deque(maxlen=) 或定期清理;④ 大数据用流式而非全量;⑤ 上线前用 tracemalloc/psutil 做一次"长跑内存"压测,观察内存曲线是否收敛。C 程序员要改的直觉:不再问"这块内存谁释放",改问"这个对象的引用图会不会无限膨胀"。

本章速查表

主题	结论 / 写法
内存模型	一切皆堆对象;名字只是引用
释放时机	引用计数归零立即;循环引用等 GC;全局引用永不
查看计数	<code>sys.getrefcount(x)</code> (自带 +1)
删除名字	<code>del x</code> 只减引用,不保证释放
循环引用	list/dict/对象环; <code>gc.collect()</code> 手动回收
gc 阈值	默认 700/10/10;别随意 disable
<code>__del__</code>	时机不可预测;析构逻辑放 <code>with/close()</code>
弱引用	<code>weakref.ref(obj)</code> 不增计数;目标死了返回 <code>None</code>
弱缓存	<code>weakref.WeakValueDictionary</code> 自动淘汰
with	进入 <code>__enter__</code> ,退出必然 <code>__exit__</code>
自定义上下文	类实现 <code>__enter__</code> / <code>__exit__</code> 或 <code>@contextmanager</code> + <code>yield</code>
多资源	<code>with A() as a, B() as b:</code> 或 <code>ExitStack</code>
泄漏形态	全局容器、无界缓存、闭包捕获、注册不注销
内存排查	<code>tracemalloc</code> 快照 + <code>compare_to</code>
拼接	<code>"\n".join(parts)</code> 优于 <code>+=</code>
小对象	元组/数据类优于 dict;大文件流式处理

最小必会清单

- 能说出引用计数归零、GC、全局引用三种释放时机
- 能解释 `sys.getrefcount` 为什么多 1
- 能构造循环引用并用 `gc.collect` 回收,且不依赖 `__del__`
- 能说出 `__del__` 三宗罪与正确替代
- 能写出 `weakref` 打破循环、`WeakValueDictionary` 做有限缓存
- 能写出自定义上下文管理器(类版与 `@contextmanager` 版)
- 能说出 `with` 与 `try/finally` 的等价关系
- 能说出 GC 语言内存泄漏的四种形态
- 能用 `tracemalloc` 定位内存增长行
- 能写出 `join` 拼接与元组打包的优化写法

自测题

Q 1. `x = [1,2,3]; y = x; del x` 之后,列表对象被释放了吗?为什么?

✅ 参考答案

没有。 `del x` 只删除名字 `x`, 对象仍被 `y` 引用, 计数为 1。只有 `del y` 后计数归零才释放。判断依据是引用计数, 不是“名字是否还存在”。

Q 2. 循环引用为什么会让引用计数失效? Python 靠什么机制解决?

✅ 参考答案

$A \rightarrow B$ 、 $B \rightarrow A$ 时, 两者计数都 ≥ 1 , 永不归零, 即使外部无人引用。解决机制是分代 GC 的标记-清除: 从根(全局、栈、寄存器)出发标记可达对象, 不可达的环被回收。注意只有容器类对象参与 GC。

Q 3. 为什么“用 `__del__` 释放文件句柄”是反模式? 正确做法是什么?

✅ 参考答案

`__del__` 时机不可预测(等 GC、解释器退出不保证), 且循环引用里可能被推迟甚至跳过。正确做法: 实现 `close()` 方法 + 上下文管理器(`__enter__` / `__exit__`), 调用方用 `with` 保证退出时释放。

Q 4. 用 `@contextmanager` 写一个“打印 `with` 块前后”的上下文管理器, 支持 `as` 子句拿到开始时间。

✅ 参考答案

```
import time
from contextlib import contextmanager

@contextmanager
def log_block(name):
    print(f"[{name}] 开始")
    start = time.perf_counter()
    try:
        yield start          # as 子句拿到 start
    finally:
        cost = time.perf_counter() - start
        print(f"[{name}] 结束, 耗时 {cost * 1000:.2f} ms")

with log_block("任务") as t0:
    print(f" 已用 {time.perf_counter() - t0:.3f} s")
```

Q 5. 全局缓存字典为什么可能“泄漏”内存? 给出两种修复。

✅ 参考答案

缓存里的对象被永久强引用, 永不回收。修复: ① `weakref.WeakValueDictionary`, 对象被别处释放后自动出缓存; ② 带上限的缓存(`functools.lru_cache(maxsize=N)` 或手写 LRU)。核心原则: 缓存必须有限。

Q 6. `gc.disable()` 之后会发生什么? 什么情况下它还安全?

✅ 参考答案

循环引用不再被自动回收,长期运行的进程内存只增不减。仅当:①程序无循环引用(全部对象由引用计数管理);②配合显式 `gc.collect()` 定期清理。生产服务禁用 GC 前必须做内存压测,绝大多数情况不该碰这个开关。

章末实验题:内存行为观察与泄漏修复

实验 5:内存行为观察实验(泄漏复现与修复)

实验目标:复现一个"日志服务"的内存泄漏(全局日志列表 + 循环引用 + 无界缓存),用引用计数观察、tracemalloc 定位,最终修复并用上下文管理器管理资源。

实验要求:

- 任务 1:写 `observe_refcount()` :创建对象,观察赋值/入列表/入字典/删除四种操作后的 `sys.getrefcount` 变化(知识点:引用计数)
- 任务 2:定义 `LogEntry` 类(含 payload 大字符串、next/prev 双链引用),构造 1000 个且相互成环,删除外部引用后观察 `len(gc.get_objects())` 与 `gc.collect()` 的回收数(知识点:循环引用、gc 模块)
- 任务 3:给 `LogEntry` 加 `__del__` 打印回收,观察"先构造环再回收"时 `__del__` 的调用时机,体会其不可预测性(知识点: `__del__` 的坑)
- 任务 4:用 `weakref.WeakValueDictionary` 实现日志缓存,对比强引用字典在"对象被删后"的行为差异(知识点:weakref)
- 任务 5:用 `@contextmanager` 实现 `memory_watch(label)` :进入时打印 tracemalloc 当前快照大小,退出时打印增量,用它测量任务 2 的泄漏量(知识点:上下文管理器、tracemalloc)
- 任务 6:综合修复:日志列表改用 `collections.deque(maxlen=200)` ,缓存改 `WeakValueDictionary` ,全部文件操作放 with 里——修复后再跑 `memory_watch` ,确认增量收敛(知识点:生产内存纪律)

实验环境:Python 3.10+,单文件 `memlab.py` 。

第一步 写 `observe_refcount`,把计数变化打印成表格,确认自己的直觉

第二步 构造环时用一个函数 `make_chain(n)` 返回所有节点,外部 `del nodes` 后观察——注意先把 `gc.disable()` 再 `enable` 以便精确观察

第三步 `memory_watch` 用 `contextmanager` 包裹 `tracemalloc.start/stop` 或 `take_snapshot` 对比

第四步 后把修复前后的峰值打印出来,形成对比结论

✅ 参考答案要点(代码分两段)

第一段:引用计数观察与循环引用。设计要点:用 `get_total_size()` 统一报告内存,方便前后对比。

```

# memlab.py — 内存行为观察与泄漏修复(覆盖第5章全部知识点)
import gc
import sys
import time
import weakref
from collections import deque
from contextlib import contextmanager
import tracemalloc

class LogEntry:
    """日志条目:带大 payload 与双向环引用"""

    def __init__(self, i: int):
        self.i = i
        self.payload = f"log-content-{i}" + "x" * 2000
        self.next = None
        self.prev = None

    def __del__(self):
        print(f"[__del__] 条目 {self.i} 被回收")

def observe_refcount() -> None:
    """观察四种操作对引用计数的影响"""
    x = [1, 2, 3]
    print(f"创建后:      {sys.getrefcount(x) - 1}")
    y = x
    print(f"y = x 后:      {sys.getrefcount(x) - 1}")
    lst = [x]
    print(f"入列表后:      {sys.getrefcount(x) - 1}")
    d = {"k": x}
    print(f"入字典后:      {sys.getrefcount(x) - 1}")
    del y, lst, d
    print(f"删除后:      {sys.getrefcount(x) - 1}")
    del x

def make_cycle(n: int) -> list:
    """构造 n 个互相成环的 LogEntry, 返回外部引用列表"""
    nodes = [LogEntry(i) for i in range(n)]
    for i in range(n):
        nodes[i].next = nodes[(i + 1) % n]
        nodes[i].prev = nodes[(i - 1) % n]
    return nodes

@contextmanager
def memory_watch(label: str):
    """进入时记录快照,退出时打印增量(上下文管理器实战)"""
    before = tracemalloc.take_snapshot().statistics("lineno")
    start = sum(s.size for s in before)
    try:
        yield
    finally:

```

```
after = tracemalloc.take_snapshot().statistics("lineno")
end = sum(s.size for s in after)
print(f"[{label}] 内存增量: {(end - start) / 1024:.1f} KiB")
```

第二段:泄漏场景与修复。修复前:强引用缓存 + 无界日志;修复后:弱引用缓存 + 有界队列。

```

def leaky_service(rounds: int) -> None:
    """修复前:全局日志列表只增不减 + 强引用缓存"""
    log_store = []
    cache = {}
    for r in range(rounds):
        nodes = make_cycle(50)          # 环:等 GC
        log_store.append(len(nodes))    # 列表只增不减
        cache[f"key-{r}"] = nodes[0]   # 强引用缓存
    with memory_watch("泄漏版"):
        del nodes, cache, log_store
    gc.collect()

def fixed_service(rounds: int) -> None:
    """修复后:deque 有界日志 + 弱引用缓存 + 显式清环"""
    log_store = deque(maxlen=200)      # 有界:永不膨胀
    cache = weakref.WeakValueDictionary() # 弱引用:自动淘汰
    for r in range(rounds):
        nodes = make_cycle(50)
        log_store.append(len(nodes))
        for node in nodes:             # 断开环,立即回收
            node.next = node.prev = None
        cache[f"key-{r}"] = nodes[0]
    with memory_watch("修复版"):
        del nodes, cache, log_store
    gc.collect()

def main() -> None:
    tracemalloc.start()                # 追踪开始
    print("== 引用计数观察 ==")
    observe_refcount()

    print("\n== 循环引用回收(GC 关闭时) ==")
    gc.disable()
    before = len(gc.get_objects())
    nodes = make_cycle(200)
    del nodes                          # 外部引用消失,环仍在
    gc.collect()                       # 手动回收(有 __del__ 的会进 garbage)
    print(f"回收了 {before - len(gc.get_objects())} 个对象")
    print(f"gc.garbage 中的对象数: {len(gc.garbage)}")
    gc.garbage.clear()
    gc.enable()

    print("\n== 泄漏 vs 修复 ==")
    leaky_service(20)
    fixed_service(20)

if __name__ == "__main__":
    main()

```

设计说明:① 观察实验放在函数里,保证局部变量作用域可控;② 循环引用实验先 `gc.disable()` 再手动 `collect`,让学生看到"外部引用消失≠立即回收";③ 有 `__del__` 的环回收后会残留到 `gc.garbage`,这正是"__del__ 让 GC 失

效"的经典表现;④ 修复版通过断开环引用让引用计数立即生效,配合有界队列与弱引用缓存,内存增量收敛;⑤ memory_watch 上下文管理器同时复习了第5章 with 与第4章闭包/装饰器思想。

下一章预告:第6章 面向对象——类与 self、_ 私有约定、继承与 MRO、鸭子类型多态、运算符重载、@classmethod/@staticmethod、组合优于继承。

第6章 面向对象

类与对象 · 继承与 MRO · 鸭子类型 · 运算符重载

本章目标:用 C 的 struct 与函数指针为锚点,讲透 Python 的类、self、继承与方法解析顺序(MRO)、鸭子类型、运算符重载,以及生产中最实用的 classmethod/staticmethod/property 三件套。学完后你能独立设计出结构清晰、可测试、可扩展的业务类。

直觉起点:在 C 里,数据和操作它的函数是分离的:struct 存数据,函数接受 struct 指针。Python 把两者合二为一——对象自带方法,方法第一个参数就是"那个对象自己"。语法变了,但底层思想依然是"数据 + 行为"。

前置要求:第1章名字绑定(对象 vs 名字)、第4章函数(默认参数、闭包)、第5章引用计数(对象生命周期)。

本章路线:第一阶段(2 小时)读 §6.1–§6.4,动手写第一个类并理解继承;第二阶段(1.5 小时)读 §6.5–§6.8,掌握鸭子类型与运算符重载;第三阶段(2 小时)完成章末实验。

本章知识导图

- └─ 6.1 从 struct 到 class —— 数据与行为的合一
- └─ 6.2 self 与 __init__ —— 第一个类、名称改写
- └─ 6.3 属性控制 —— _ 约定 / @property / __slots__
- └─ 6.4 继承与 MRO —— super、C3 线性化
- └─ 6.5 鸭子类型 —— 多态的真面目
- └─ 6.6 运算符重载 —— __eq__ / __lt__ / __repr__ 及配套
- └─ 6.7 三种方法 —— 实例方法 / @classmethod / @staticmethod
- └─ 6.8 组合优于继承 —— 生产设计取向

§6.1 从 struct 到 class

C 程序员描述"一个学生"时,写一个 struct,再写一组接受该指针的函数;调用时手传 `&stu`。Python 的 class 把这两个动作合成一个声明:字段与方法长在同一个命名空间里,方法调用时解释器自动把对象本身作为第一个参数传进去。

C(struct + 函数)	Python(class)
<code>struct Student s; init(&s, ...);</code>	<code>s = Student(...)</code>
操作函数在 struct 外,靠命名前缀归类	方法在类体内,归属关系由语法保证
忘记初始化→未定义行为	<code>__init__</code> 保证创建即初始化
函数指针成员模拟"方法",绕且晦涩	方法就是普通函数,第一个参数是 self
struct 拷贝是值拷贝	赋值只是复制引用,对象只有一个

类(class)是蓝图,**实例(instance)**是对象。类本身也是对象(第11章会深入),因此"定义类"只是创建一个可调用的、负责生产实例的工厂。

6.1.1 从"结构体+函数"到"类"

C 写法是数据与逻辑分离的;Python 写法把它们收进一个类,数据不出类也能被类方法加工。两者解决的问题相同:把一组相关的数据与操作绑定。

从 C 迁移的开发者常纠结“为什么不保持纯数据”。实际上 Python 的对象模型恰是 C 结构化数据的自然延伸:struct 表达“数据长什么样”,class 再附加“这些数据能做什么”。合并之后,数据与操作的匹配由解释器强制,而不是靠命名规范提醒,漏调 init 之类的问题在语法层面就被堵死。

§6.2 self 与 __init__:你的第一个类

```
class Point:
    """二维点。__init__ 负责初始化,__repr__ 负责调试展示"""

    def __init__(self, x: float, y: float) -> None:
        self.x = x
        self.y = y

    def distance(self, other: "Point") -> float:
        """两点间欧氏距离。注意 self 是第一个参数"""
        dx = self.x - other.x
        dy = self.y - other.y
        return (dx * dx + dy * dy) ** 0.5

    def __repr__(self) -> str:
        return f"Point({self.x}, {self.y})"

p1 = Point(1.0, 2.0)          # 等价于 Point.__init__(p1, 1.0, 2.0)
p2 = Point(4.0, 6.0)
print(p1.distance(p2))       # 5.0
print(p1)                    # Point(1.0, 2.0)
```

注意 `Point(1.0, 2.0)` 这种构造语法之所以成立,正是因为 `__init__` 接收了实例本身并完成初始化——理解这一行,就理解了 Python 一切魔法方法的入口。

关键理解: `p1.distance(p2)` 只是 `Point.distance(p1, p2)` 的语法糖——Python 没有“this 隐式传入”这回事,self 就是一个普普通通的参数,因此**定义时必须写 self,调用时绝不能传 self**。

误区:忘写 self 或调用时多传了参数。定义 `def greet(): ...` 而调用 `obj.greet()` 会报“takes 0 positional arguments but 1 was given”——因为解释器把 `obj` 硬塞给了第一个参数。另外记住 `__init__` 必须返回 `None`,显式 `return` 一个值会抛 `TypeError`。

6.2.1 __init__ 不是构造器

真正的构造器是 `__new__` (负责分配内存), `__init__` 只负责“初始化已有对象”。日常开发只写 `__init__` 即可;只有实现单例、不可变对象或元类时才会碰 `__new__`。`__init__` 可以省略:没写时 Python 调用 `object` 的默认初始化。

6.2.2 名称改写:双下划线不是“私有”

Python 没有 `private` 关键字,只有约定与改名。`_x` 表示“内部实现,勿外部访问”(纯约定,照常可访问);`__x` 会触发名称改写,变成 `_ClassName__x`,主要用来避免子类不小心覆盖父类属性:

```

class BankAccount:
    def __init__(self, owner: str) -> None:
        self.owner = owner
        self.__balance = 0          # 改写为 _BankAccount__balance

    def deposit(self, amount: float) -> None:
        self.__balance += amount

acct = BankAccount("李雷")
acct.deposit(100)
print(acct.__balance)              # AttributeError!
print(acct._BankAccount__balance)  # 100 — 改写可查,但别这么写

```

生产规范:内部成员用单下划线 `_xxx`;双下划线仅用于"防止子类冲突"的场景(如混入类)。真正的访问控制交给 `@property`。

§6.3 属性控制:@property 与 `__slots__`

C 的 `struct` 字段谁都能读写;生产 Python 用 **@property** 把"读取"包装成方法调用,从而可以在赋值时做校验、在读取时做计算,调用方语法不变:

```

class Employee:
    def __init__(self, name: str, salary: float) -> None:
        self.name = name
        self._salary = 0.0
        self.salary = salary        # 走 setter 校验

    @property
    def salary(self) -> float:
        """只读属性与 setter 同名,调用方像访问字段"""
        return self._salary

    @salary.setter
    def salary(self, value: float) -> None:
        if value < 0:
            raise ValueError("工资不能为负")
        self._salary = value

    @property
    def annual(self) -> float:
        return self._salary * 12    # 计算属性,无 setter → 只读

e = Employee("韩梅梅", 8000)
e.salary = 9000
print(e.annual)                    # 108000
e.salary = -1                      # ValueError: 工资不能为负

```

判断该用 `property` 还是裸字段:先写裸字段,等"需要校验/计算/缓存"时再升级为 `property`——调用方代码一行不用改。这比 C 里改字段为宏/函数要平滑得多。

6.3.1 __slots__:省内存的字段白名单

```
class User:
    """默认每个实例都有 __dict__ 字典;__slots__ 改为固定描述符"""
    __slots__ = ("name", "age")

    def __init__(self, name: str, age: int) -> None:
        self.name = name
        self.age = age

u = User("王五", 30)
u.email = "w@example.com"          # AttributeError: 没有这个槽
```

百万级小对象场景用 __slots__ 能显著降内存(省掉每实例的 dict 与弱引用表);代价是失去动态加属性。先测量再优化,别默认使用。

§6.4 继承与方法解析顺序

继承让子类复用父类行为并覆盖差异。调用 `super().__init__(...)` 是子类初始化父类部分的唯一正道——忘记调用会让父类字段缺失,这是继承场景第一大坑。

```
class Shape:
    def __init__(self, name: str) -> None:
        self.name = name

    def area(self) -> float:
        raise NotImplementedError("子类必须实现 area")

    def describe(self) -> str:
        return f"{self.name} 面积 {self.area():.2f}"

class Rectangle(Shape):
    def __init__(self, w: float, h: float) -> None:
        super().__init__("矩形")      # 必须先初始化父类部分
        self.w = w
        self.h = h

    def area(self) -> float:
        return self.w * self.h

r = Rectangle(3.0, 4.0)
print(r.describe())                  # 矩形 面积 12.00
```

误区:多继承里忘了调用链。 `super()` 不是"调父类",而是"沿 MRO 找下一个合作者"。菱形继承(A 有 B、C 两个子类,D 同时继承 B、C)中,如果 B、C 都调 `super().__init__`,D 实例化时 A.__init__ 恰好执行一次;如果谁漏了,初始化链就断。
规则:多继承里每一层都要调用 `super().__init__()`。

6.4.1 MRO:C3 线性化

Python 用 C3 算法把继承关系拍平成一条唯一的方法查找顺序,存在 `ClassName.__mro__` 里。查找方法时按 MRO 从左到右找第一个命中的。这与 C++ 的虚函数表思路不同:

C++(虚函数表)	Python(MRO)
编译期生成虚表,运行期查表	运行期沿 <code>__mro__</code> 线性顺序查找
多继承二义性要 virtual 显式处理	C3 保证唯一顺序,super() 自动接力
覆盖发生在编译期绑定	方法在调用那一刻动态决定(鸭子类型)
虚表指针藏在对象里,内存开销固定	查找是运行期遍历,慢在"每次调用"

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass

# C3 保证:每个类只出现一次,且顺序全局一致
for cls in D.__mro__:
    print(cls.__name__, end=" ")
print()                                     # 输出: D B C A object
```

生产建议:单继承 + 混入(mixin)足以覆盖 95% 需求;三层以上的深继承或"歪菱形"会让 MRO 与直觉脱节,优先改为组合。

§6.5 鸭子类型:多态的真面目

C 的多态靠继承 + 虚函数;"长得像就用"是 Python 的多态。检查"是否支持某操作"时,不要先 `isinstance` 再分支,而是直接调用并让错误自然发生——这叫 EAFP(easier to ask forgiveness than permission):

```
def total_area(shapes: list) -> float:
    """不检查类型,只要求每个元素有 area() 方法"""
    return sum(s.area() for s in shapes)

class Circle:
    # 完全不继承 Shape!
    def __init__(self, r: float) -> None:
        self.r = r

    def area(self) -> float:
        return 3.14159 * self.r * self.r

items = [Rectangle(3.0, 4.0), Circle(2.0)]
print(total_area(items))                # 12.0 + 12.57 = 24.57
```

C(显式类型分支)	Python(EAFP)
<code>if (shape->type == CIRCLE) ...</code>	直接调用 <code>s.area()</code> ,没有就抛 <code>AttributeError</code>
新增类型要改 <code>switch</code> 与头文件	新增类型只要方法名对上,调用方零改动
编译器在编译期抓住类型错误	错误推迟到运行期——用测试与静态检查工具兜底

误区:到处 `isinstance` 检查。 `isinstance` 分支写多了,代码会退化成 C 式 `switch`,每加一个类型改一个函数。正确姿势:接口约定 + 文档写明"需要 `area()` 方法",必要时用第10章的 `Protocol` 做静态检查,运行期直接调用。

§6.6 运算符重载:让对象参与表达式

C 的运算符只对内置类型生效;Python 允许类定义运算符行为。生产中最常用的是排序与比较: `__lt__` 让对象能被 `sorted/max/min` 使用, `__eq__` 让对象可比较可哈希:

```
from functools import total_ordering

@total_ordering          # 只实现 __eq__ 和 __lt__,其余自动补全
class Score:
    def __init__(self, name: str, value: int) -> None:
        self.name = name
        self.value = value

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Score):
            return NotImplemented
        return self.value == other.value

    def __lt__(self, other: "Score") -> bool:
        return self.value < other.value

    def __repr__(self) -> str:
        return f"{self.name}:{self.value}"

scores = [Score("甲", 88), Score("乙", 95), Score("丙", 79)]
print(sorted(scores))          # [丙:79, 甲:88, 乙:95]
print(max(scores))             # 乙:95
```

误区:实现 `__eq__` 却忘了 `__hash__`。 Python 规定:定义 `__eq__` 的类自动失去默认哈希(变为不可哈希),放进 `set/dict` 的 `key` 会直接 `TypeError`。要么同时实现 `__hash__` (基于同一组字段),要么让对象明确不可哈希(当作"不该进集合"的信号)。

另一个高频重载是 `__repr__` (给开发者看的调试信息)与 `__str__` (给用户看的展示)。只写一个就写 `__repr__`,因为 `str()` 会退回 `repr`,而 f-string 优先用 `__str__`。

§6.7 三种方法:实例方法 / `@classmethod` / `@staticmethod`

类体里可以出现三类函数,用途截然不同:

```

class Config:
    env = "prod" # 类属性,所有实例共享

    def __init__(self, key: str) -> None:
        self.key = key # 实例方法:操作单个实例

    def display(self) -> str:
        return f"{self.key}={Config.env}" # 实例方法第一参是 self

    @classmethod
    def set_env(cls, env: str) -> None:
        """类方法:第一参是类本身,常用于工厂与全局配置"""
        cls.env = env

    @staticmethod
    def is_valid(key: str) -> bool:
        """静态方法:不接 self/cls,只是逻辑上归属本类"""
        return bool(key) and key.isascii()

Config.set_env("staging") # 通过类调用
dev = Config("api_token")
print(dev.display()) # api_token=staging
print(Config.is_valid("ok")) # True

```

选用口诀:需要操作实例数据→实例方法;需要访问或修改类属性/写工厂→@classmethod;与实例、类都无关、只是收纳→@staticmethod。@staticmethod 在新代码里频率很低,多数"工具方法"放模块级函数更自然。

6.7.1 classmethod 工厂

经典生产用法:一个类有多个合法构造路径,用 classmethod 提供带名字的工厂:

```

from datetime import date

class Person:
    def __init__(self, name: str, birth: date) -> None:
        self.name = name
        self.birth = birth

    @classmethod
    def from_iso(cls, name: str, iso: str) -> "Person":
        """从 ISO 字符串构造,失败时抛 ValueError,职责集中"""
        return cls(name, date.fromisoformat(iso))

    @property
    def age(self) -> int:
        today = date.today()
        return today.year - self.birth.year - (
            (today.month, today.day) < (self.birth.month, self.birth.day))

p = Person.from_iso("赵六", "2000-03-15")
print(p.age)

```

§6.8 组合优于继承:生产设计取向

继承表达"is-a",组合表达"has-a"。继承树深了会僵硬:改父类影响所有子孙、子类被迫继承不需要的的方法。生产代码里,优先用组合:把变化的行为塞进一个对象,通过构造注入:

```
class Logger:                                # 策略:可替换的行为
    def log(self, msg: str) -> None:
        print(f"[log] {msg}")

class SilentLogger:                          # 另一策略,同一接口
    def log(self, msg: str) -> None:
        pass

class Service:
    def __init__(self, logger: Logger) -> None:
        self.logger = logger                # 组合:行为注入,而非继承

    def run(self) -> None:
        self.logger.log("开始工作")

Service(Logger()).run()                      # 打印
Service(SilentLogger()).run()               # 静默—调用方代码不变
```

判断何时继承:①严格 is-a(经理是员工);②需要复用实现且差异有限;③多态入口(接口/协议)。出现"只想借用两个方法""父类一半方法用不上""为了调 super()"时,改组合。

工程建议 1:给类写 `__repr__` 是免费的调试投资——异常信息、日志、pytest 失败输出都会打印它。宁可少写文档字符串,也别省 `__repr__`。

工程建议 2:类内部不直接碰"环境"细节(打印、读环境变量、连数据库),通过构造注入依赖,这样单测时替换一个假的实现即可,这也是"组合优于继承"的直接收益。

本章速查表

场景	写法
定义类	<code>class Point: ...</code>
初始化	<code>def __init__(self, x): self.x = x</code>
方法第一参	定义写 <code>self</code> , 调用绝不传
内部成员	单下划线 <code>_x</code> (约定)
防子类覆盖	<code>__x</code> → 改写为 <code>_Class__x</code>
只读/校验字段	<code>@property</code> + <code>@x.setter</code>
省内存白名单	<code>__slots__ = ("a", "b")</code>
子类初始化父类	<code>super().__init__(...)</code>
查看查找顺序	<code>D.__mro__</code>
比较排序	<code>__eq__</code> + <code>__lt__</code> , 配 <code>@total_ordering</code>
哈希配套	实现 <code>__eq__</code> 必须同时实现 <code>__hash__</code>
调试展示	<code>__repr__</code> (str 会退回它)
工厂构造	<code>@classmethod</code> 返回 <code>cls(...)</code>
纯工具函数	<code>@staticmethod</code> 或模块级函数
多态检查	直接调用, EAFP, 少用 <code>isinstance</code>
扩展方式	组合注入优先于加深继承

最小必会清单

1. 能写一个带 `__init__`、实例方法、`__repr__` 的类, 并解释 `p1.distance(p2)` 的完整调用过程
2. 说出 `_x` 与 `__x` 的区别, 并正确使用 `@property` 实现只读字段
3. 会用 `super().__init__` 初始化继承链, 并画出单继承与一次混入的 MRO
4. 能让自定义类直接参与 `sorted()`: 实现 `__lt__` 并配齐 `__eq__`/`__hash__`
5. 能区分实例方法、`@classmethod`、`@staticmethod` 三种场景, 并写出一个 `classmethod` 工厂
6. 能说明"组合优于继承"的理由, 并把一个"为复用继承"的类重构为注入

自测题

Q

Q1. `p1.distance(p2)` 与 `Point.distance(p1, p2)` 是什么关系? 定义方法时漏写 `self` 会怎样?

答案: 完全等价, 前者是后者的语法糖。漏写 `self` 后, 调用 `p1.distance(p2)` 会把 `p1` 当作第一个实参, 抛 `"takes 1 positional argument but 2 were given"`; 若只有 `p1.distance()` 这种调用, 则 `p1` 被当作参数, 方法体内访问属性时抛 `AttributeError`。

Q

Q2. 类里定义 `__x = 1` 后,外部访问 `obj.__x` 报 `AttributeError`,为什么?如何访问?

答案:双下划线开头触发名称改写, `__x` 在类体内被改成 `_类名__x`。外部应访问 `obj._类名__x`——但这是破坏封装的行为,生产代码不这么做;需要"私有"语义时用单下划线 + `@property`。

Q

Q3. 为什么定义 `__eq__` 之后对象会变得不可哈希?如何让对象既能比较又能进 `set`?

答案:Python 默认哈希基于 `id`;一旦定义 `__eq__`,默认哈希被停用以保持"相等对象哈希必相同"的不变量。同时实现 `__hash__` (用 `__eq__` 比较的同一组字段,如 `return hash((self.value,))`),对象即可安全进入 `set/dict`。

Q

Q4. 菱形继承 `D(B, C)` 中,B、C 都继承 A 且各自 `__init__` 调用了 `super().__init__()`,D 实例化时 A. `__init__` 执行几次?如果 B 漏调 `super()` 会怎样?

答案:一次。MRO 把每个类在线性化列表里只放一次, `super()` 沿 MRO 接力,A 只被初始化一次。B 漏调则初始化链在 B 处断裂, `super().__init__` 之后的合作者(A 与 C 的初始化)全部被跳过,对象处于部分初始化状态。

Q

Q5. 一段代码里频繁出现 `if isinstance(x, A): ... elif isinstance(x, B): ...`,它的问题是什么?怎么改?

答案:它把"类型分支"硬编码进调用方,新增类型要改调用方,违背开闭原则,与鸭子类型哲学相悖。改法:定义统一接口(同名方法或 Protocol),调用方直接调用;若分支本质是算法族,用策略对象注入(\$6.8 的组合模式)。

Q

Q6. `@property` 与裸字段相比多了函数调用开销,为什么生产代码仍然大量使用?

答案:因为收益大于开销:①可以在赋值点集中校验,错误提前到"写入时"而非"使用时";②可以做成计算属性,调用方无感;③可以只读或按条件隐藏底层字段,把"如何存"与"如何暴露"解耦;④升级成本低——先裸字段,需要时再包 `property`,调用方代码零改动。性能敏感的热路径才需要避开它。

章末实验题:员工薪酬系统

实验 6.1 员工薪酬系统(覆盖:\$6.2–\$6.8)

需求:① 基类 `Employee` 含姓名、基本工资, `pay()` 返回月薪, `describe()` 输出简介;② 子类 `Manager` 按级别给系数,子类 `Salesman` 按业绩提成;③ 工资为 `property`,负值报错;④ 员工支持 `==` (同名同类型)与 `<` (按月薪),可直接 `sorted`;⑤ 用 `classmethod` 工厂从字典构造;⑥ 组合一个 `Department` 管理多名员工,提供 `total_pay()` 与 `top_pay()`;⑦ 写 `main` 演示排序与多态统计。

覆盖知识点:类定义与 `__init__` (\$6.2)/名称改写与 `property` (\$6.2、\$6.3)/继承与 `super` (\$6.4)/鸭子类型与多态 (\$6.5)/运算符重载与 `total_ordering` (\$6.6)/`classmethod` 工厂与三种方法 (\$6.7)/组合 (\$6.8)。

实验环境:Python 3.10+,单文件 `payroll.py`。

```

# payroll.py — 员工薪酬系统(覆盖第6章全部知识点)
from functools import total_ordering

@total_ordering
class Employee:
    """员工基类:月薪属性带校验,支持比较与排序"""

    def __init__(self, name: str, salary: float) -> None:
        self.name = name
        self.salary = salary          # 走 setter 校验

    @property
    def salary(self) -> float:
        return self._salary

    @salary.setter
    def salary(self, value: float) -> None:
        if value < 0:
            raise ValueError(f"工资不能为负: {value}")
        self._salary = value

    def pay(self) -> float:
        return self._salary

    def describe(self) -> str:
        return f"{self.__class__.__name__} {self.name} 月薪 {self.pay():.0f}"

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Employee):
            return NotImplemented
        return (self.name, type(self)) == (other.name, type(other))

    def __lt__(self, other: "Employee") -> bool:
        return self.pay() < other.pay()

    def __repr__(self) -> str:
        return self.describe()

    @classmethod
    def from_dict(cls, data: dict) -> "Employee":
        """工厂:按 kind 分派到子类,职责集中在类方法里"""
        if data["kind"] == "manager":
            return Manager(data["name"], data["salary"], data["level"])
        if kind == "sales":
            return Salesman(data["name"], data["salary"], data["sales"],
                             data.get("rate", 0.05))
        raise ValueError(f"未知员工类型: {kind}")

class Manager(Employee):
    """经理:级别越高,月薪系数越大"""

    def __init__(self, name: str, salary: float, level: int) -> None:
        super().__init__(name, salary) # 先初始化父类部分

```

```
self.level = level
```

```
def pay(self) -> float:  
    return self.salary * (1 + 0.2 * self.level)    # 多态覆盖
```

```
class Salesman(Employee):  
    """销售:底薪 + 提成"""  
  
    def __init__(self, name: str, salary: float,  
                  sales: float, rate: float = 0.05) -> None:  
        super().__init__(name, salary)  
        self.sales = sales  
        self.rate = rate  
  
    def pay(self) -> float:  
        return self.salary + self.sales * self.rate  
  
class Department:  
    """组合:部门拥有多名员工,对外提供统计能力"""  
  
    def __init__(self, name: str, members: list | None = None) -> None:  
        self.name = name  
        self.members = list(members or [])  
  
    def add(self, emp: Employee) -> None:  
        self.members.append(emp)  
  
    def total_pay(self) -> float:  
        return sum(e.pay() for e in self.members)  
  
    def top_pay(self) -> Employee:  
        return max(self.members)    # 依赖 __lt__  
  
def main() -> None:  
    raw = [  
        {"kind": "manager", "name": "李雷", "salary": 20000, "level": 2},  
        {"kind": "sales", "name": "韩梅梅", "salary": 8000,  
         "sales": 60000, "rate": 0.06},  
        {"kind": "sales", "name": "王五", "salary": 8000, "sales": 30000},  
    ]  
    dept = Department("销售部")  
    for item in raw:  
        dept.add(Employee.from_dict(item))    # 工厂 + 多态  
  
    for emp in sorted(dept.members):    # 排序走 __lt__  
        print(emp.describe())  
    print(f"部门总薪酬: {dept.total_pay():.0f}")  
    print(f"薪酬最高: {dept.top_pay()}")  
  
if __name__ == "__main__":  
    main()
```

设计说明:① from_dict 工厂把"反序列化分派"收进类体系,新增工种只改工厂一个函数;② pay() 在子类覆盖、describe() 复用多态,体现"继承用于复用实现、多态用于扩展行为";③ __lt__ 让 max/sorted 直接可用,total_ordering 自动补齐 >、<=、>=;④ Department 用组合容纳任意 Employee,若将来换成接口或 Protocol,这里不用动;⑤ 校验集中在 property setter,非法数据在构造时就失败,而不是运行到 pay 时才爆。

下一章预告:第7章 文件与数据交换——pathlib 路径操作、open 的编码世界、JSON/CSV 结构化数据、按行流式处理大文件。学完面向对象,让数据在磁盘与内存之间安全流动。

第7章 文件与数据交换

pathlib · 编码与 open · 流式处理 · JSON / CSV · 标准输入输出

本章目标:讲透 Python 文件 I/O 的完整图景;pathlib 路径对象、open 的模式与编码、按行流式处理大文件、二进制与 struct 打包、JSON/CSV 结构化交换,以及 sys.argv/stdin/stdout 命令行管线。学完你能安全、高效、编码正确地处理生产中的数据文件。

直觉起点:C 里文件是 FILE* 加一串函数;Python 里文件是对象,路径也是对象。C 的核心痛苦——路径拼接、编码、缓冲区——在 Python 里都有语法级解药,但"模式选错清空文件"、"编码没指定乱码"这类坑依然存在,只是换了形态。

前置要求:第1章字符串与字节、第5章 with 上下文管理器(文件必须 with 打开)。

本章路线:第一阶段(1.5 小时)读 §7.1–§7.3,掌握 pathlib 与文本流式读取;第二阶段(1.5 小时)读 §7.4–§7.6,掌握二进制与 JSON/CSV;第三阶段(2 小时)读 §7.7–§7.8 并完成章末实验。

本章知识导图

- └─ 7.1 pathlib —— 路径是对象,不是字符串
- └─ 7.2 open 与编码 —— 模式、encoding、BOM、换行
- └─ 7.3 流式读大文件 —— 按行迭代,内存恒定
- └─ 7.4 二进制文件 —— struct 打包与解包
- └─ 7.5 JSON —— 序列化、反序列化与中文
- └─ 7.6 CSV —— DictReader、写行、编码陷阱
- └─ 7.7 argv 与标准流 —— 命令行管线
- └─ 7.8 生产实践 —— 临时文件与错误处理

§7.1 pathlib:路径是对象,不是字符串

C 的路径处理是 char 数组拼接:strcpy、strcat、检查末尾斜杠,一不留神缓冲区溢出或平台分隔符错误。Python 3.4+ 的 **pathlib** 把路径变成对象,拼接、判断、遍历都是方法调用,跨平台且不可变:

```
from pathlib import Path

base = Path("data") / "2026" / "logs"      # 运算符拼接,自动处理分隔符
print(base)                                # data\2026\logs (Windows)

p = Path("data/2026/logs/app.log")
print(p.name)          # app.log
print(p.stem)          # app
print(p.suffix)         # .log
print(p.parent)        # data\2026\logs
print(p.exists())       # False — 当前目录没有这个文件

for child in Path("data").iterdir():        # 遍历目录
    print(child.name)
```

C(字符串拼接)	Python(pathlib)
<code>strcat(buf, "/"); strcat(buf, name);</code>	<code>Path("data") / name</code>
平台分隔符要手动处理 / 转义	Path 自动使用当前平台分隔符
判存在 <code>stat() + errno</code> 组合拳	<code>p.exists()</code> / <code>p.is_dir()</code> / <code>p.is_file()</code>
目录遍历要 <code>opendir/readdir/closedir</code>	<code>p.iterdir()</code> / <code>p.glob("*.log")</code>
拼错路径只能靠运行期报错	Path 对象先行验证合法性,错误更早

工程建议 1:新代码一律用 `pathlib`,不用 `os.path` 拼接字符串;函数参数类型标注 `path: Path`,同时用 `pathlib.Path` 作为唯一入口,避免"字符串和 Path 混用"的隐式转换错误。

7.1.1 常用操作

读文件: `Path.read_text(encoding=...)`、`read_bytes()`; 写文件: `write_text()`、`write_bytes()`; 建目录: `makedirs(parents=True, exist_ok=True)`; 通配: `glob("**/*.log")` 递归找日志。注意 `read_text/write_text` 是"一把梭"API,适合小文件;大文件仍用 `open` 流式处理。

Windows 下拼路径还有一坑:字符串里反斜杠是转义符, `"C:\data"` 必须写成双反斜杠;换成 Path 之后这类问题彻底消失。迁移期混用 `os.path` 与 `pathlib` 会让类型标注失真,团队约定:函数签名一律 Path,内部统一 Path 方法。

§7.2 open 与编码:模式、encoding、换行

`open` 的完整调用是 `open(path, mode, encoding=...)`。模式的首字符决定核心行为:

```
with open("a.txt", "w", encoding="utf-8") as f:
    f.write("第一行\n")          # "w" 覆盖写,文件不存在则创建

with open("a.txt", "a", encoding="utf-8") as f:
    f.write("追加行\n")          # "a" 追加写,不覆盖已有内容

with open("a.txt", "r", encoding="utf-8") as f:
    print(f.read())              # "r" 默认模式,读取全部文本
```

模式	含义	C 对应
"r"	读文本(默认),文件必须存在	"r"
"w"	写文本,立即清空原内容	"w"
"a"	追加写,不清空,从末尾开始	"a"
"x"	独占新建,文件已存在则报错	无(要手写 O_EXCL)
"r+"	读写,不清空,光标在开头	"r+"
"rb" / "wb"	二进制读/写,不做编码解码	"rb" / "wb"

写文件还有一个"看不见的延迟":内容先落在用户态缓冲区,进程崩溃时未 `flush` 的数据直接丢失。with 退出或显式 `f.flush()` 才落盘;日志系统常用 `flush=True` 保证每条即时可见,数据库类写入则靠 `fsync` 兜底。理解"缓冲区存在"这个事实,排障时少走弯路。

误区:不指定 encoding。 Windows 默认用 GBK 打开文本文件,Linux 默认 UTF-8——同一份代码两平台行为不同。生产规范:所有文本 open 一律显式 `encoding="utf-8"`,写文件同理,否则读 UTF-8 文件在 Windows 上直接 `UnicodeDecodeError`。

7.2.1 BOM 与换行

Windows 记事本保存的 UTF-8 文件带 BOM(文件头三个字节 EF BB BF),Python 读它会多出一个不可见字符 `U+FEFF`。用 `encoding="utf-8-sig"` 读可自动剥掉 BOM;写文件想带 BOM 给别人用记事本看,也用 `utf-8-sig` 写。

```
with open("bom.txt", "rb") as f:
    head = f.read(3)          # 二进制读文件头三个字节
    print(head == b"\xef\xbb\xbf")  # True — BOM 确实存在

with open("bom.txt", encoding="utf-8-sig") as f:
    text = f.read()          # utf-8-sig 自动剥掉 BOM

print("BOM 已剥离:", "" not in text)
```

换行:open 默认开启通用换行模式,读时把 `\r\n` 统一转成 `\n`,写时按平台转换。若与别人交换的字节必须原样,用 `newline=""` 关闭转换(写 CSV 时这是标准操作)。

§7.3 流式读大文件:内存恒定

C 里处理大文件的正道是 `fgets` 逐行;Python 的对应物是直接迭代文件对象。文件对象是惰性的,一次只从磁盘读一个块,因此 10 GB 日志也能处理,内存占用与文件大小无关:

```
def count_levels(path: Path) -> dict:
    """统计日志中各级别出现次数,内存与文件大小无关"""
    stats = {"INFO": 0, "WARN": 0, "ERROR": 0}
    with open(path, encoding="utf-8") as f:
        for line in f:          # 逐行迭代,绝不 read() 全量
            for level in stats:
                if level in line:
                    stats[level] += 1
    return stats

print(count_levels(Path("data/app.log")))  # 例如 {'INFO': 42, ...}
```

C(流式)	Python(流式)
<code>while (fgets(buf, N, fp))</code>	<code>for line in f:</code>
buf 要自己开、自己管长度上限	迭代器按行切分,行长度无上限
行内再 <code>strtok</code> 拆分,小心改坏原串	<code>line.split()</code> 返回新列表,原行不动
忘记 <code>fclose</code> 泄漏句柄	<code>with</code> 退出自动 close

迭代文件对象还有两个实用细节:空行也会被迭代出来,需要时自行 `strip` 判断;行尾换行符保留着,做精确匹配先 `line.rstrip()`。若按"固定块"而非"行"处理(如解析二进制日志),用 `f.read(chunk_size)` 循环读固定字节数,同样内存恒定。

误区:大文件一把梭。 `f.read()` 把整个文件读进内存,100 MB 文件就是 100 MB 内存;若文件再大,进程直接 OOM。凡是"逐行处理"的场景都用迭代;只有文件确定很小(配置、模板)才用 `read()`。

§7.4 二进制文件:struct 打包与解包

C 用 `fwrite/fread` 加结构体直接怼文件,但内存布局依赖编译器与平台;Python 用 `struct` 模块按格式串精确控制字节序与字段宽度,跨平台可移植:

```
import struct

# 格式串 ">iif":大端(int32, int32, float32)
data = struct.pack(">iif", 7, 100, 3.14)
print(len(data))                # 12 字节

with open("bin.dat", "wb") as f:
    f.write(data)

with open("bin.dat", "rb") as f:
    raw = f.read()               # 小文件,一次读完没问题
    a, b, c = struct.unpack(">iif", raw) # (7, 100, 3.1400001049...)
    print(a, b, round(c, 2))
```

C(fwrite/fread)	Python(struct)
结构体直接写,布局依赖 ABI	格式串显式声明字节序与宽度
跨机器/跨编译器读不回原样	> 大端 / < 小端由你指定,结果可移植
填充字节(padding)不可控	= 与 > 控制是否对齐,字段宽度精确
读回要逐字段 memcpy 校验	unpack 一次解出,长度不匹配直接抛错

生产场景:与 C 服务通信的二进制协议、图片头解析、定长记录文件。记住格式串语法:整数用 `i`(4 字节)/`q`(8 字节),浮点用 `f`(4)/`d`(8),字符串用 `s` 前缀长度,如 `"10s"` 表示定长 10 字节字符串。

`struct` 格式串可以组合出复杂协议,比如 `">H2sH"` 表示无符号短整型、两字节字符串、无符号短整型共 6 字节。解析外部协议时先验证 `len(raw) == struct.calcsize(fmt)`,不匹配立刻报错而不是盲取——这正是 C 里缓冲区越界的前身,在 Python 里变成显式的长度检查。

§7.5 JSON:生产数据交换第一格式

JSON 是服务间交换数据的默认格式。Python 的 `json` 模块四个函数覆盖全部需求:`loads`(字符串→对象)、`dumps`(对象→字符串)、`load`(文件→对象)、`dump`(对象→文件):

```

import json
from pathlib import Path

order = {
    "order_id": "A1001",
    "customer": "李雷",
    "items": [{"sku": "P-001", "qty": 2}],
    "paid": True,
}

text = json.dumps(order, ensure_ascii=False, indent=2)
Path("order.json").write_text(text, encoding="utf-8")

loaded = json.loads(Path("order.json").read_text(encoding="utf-8"))
print(loaded["customer"])          # 李雷
print(loaded["items"][0]["qty"])   # 2

```

误区:中文变成 \uXXXX。 dumps 默认 ensure_ascii=True,所有非 ASCII 字符转成 \u 转义,中文文件变成天书——功能没错,但难读、体积大、diff 混乱。生产规范:写给人读或与业务方对齐的文件,一律 ensure_ascii=False 且文件编码 UTF-8。

类型映射要心里有数:JSON 的 true/false/null 对应 Python 的 True/False/None;数字只有一种,Python 的 int 读回来可能是 int 或 float。Python 的 set、datetime 不能直接序列化,需要自定义 default 函数转换——第10章的类型体系会再讲。

一个生产级细节:json.load 读入的 dict,访问不存在的 key 会抛 KeyError。对第三方接口的返回,先判断字段是否存在再取值,或用 dict.get(key, default)。同理,dumps 前确认数据里没有 set、bytes 这类不可序列化对象,否则运行到一半才抛 TypeError,殃及整批数据落盘。

§7.6 CSV:表格数据的最简载体

csv 模块处理带引号、带逗号、带换行的字段,比自己 split(",") 可靠得多。生产首选 DictReader——用表头当 key,代码可读且不怕列顺序变化:

```

import csv
from pathlib import Path

rows = [
    {"name": "李雷", "dept": "研发", "salary": 18000},
    {"name": "韩梅梅", "dept": "销售", "salary": 12000},
]

with open("staff.csv", "w", encoding="utf-8-sig",
        newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "dept", "salary"])
    writer.writeheader()
    writer.writerows(rows)

with open("staff.csv", encoding="utf-8-sig", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["dept"])    # 按表头取值

```

两个必知细节: `newline=""` 关闭换行转换,防止 Excel 打开出现空行——这是 csv 写操作的标准配置;读别人给的 CSV 先确认编码(utf-8 无 BOM / 带 BOM / GBK),BOM 用 utf-8-sig 自动剥离。Excel 生成的 CSV 常是 GBK,Windows 上读这类文件要 `encoding="gbk"`。

还有一类常见需求:表头缺失或列数不齐的脏 CSV。DictReader 遇到缺字段的行会得到缺 key 的 dict,取值时用 `row.get("col")` 配合默认值;列顺序则完全不敏感。若 CSV 来自外部且内容不可信,可先 `csv.Sniffer` 嗅探分隔符与引号风格,再决定参数。

§7.7 argv 与标准流:命令行管线

Unix 哲学"一个程序做好一件事,输出接给下一个程序"。Python 程序通过 `sys.argv` 拿参数、`sys.stdin` 读输入、`sys.stdout` 写输出,天然参与管道:

```
import sys

def main() -> None:
    if len(sys.argv) < 2:
        print("用法: python word_count.py 文件名", file=sys.stderr)
        sys.exit(1)
    path = sys.argv[1]
    total = 0
    with open(path, encoding="utf-8") as f:
        for line in f:
            total += len(line.split())
    print(total) # 标准输出,可被管道接走

if __name__ == "__main__":
    main()
```

C	Python
<code>int main(int argc, char *argv[])</code>	<code>sys.argv[0]</code> 是脚本名, <code>sys.argv[1:]</code> 是参数
参数个数靠 <code>argc</code> 判断	<code>len(sys.argv)</code> 判断
<code>printf</code> 打印,错误码 <code>return</code>	<code>print</code> 打印, <code>sys.exit(1)</code> 带状态码退出
<code>getchar/gets</code> 读 <code>stdin</code>	<code>for line in sys.stdin:</code> 直接迭代
错误输出 <code>stderr</code> 手动 <code>flush</code>	<code>print(..., file=sys.stderr)</code>

参数多了就上 `argparse` 标准库:自动生成帮助、类型转换、必选可选参数,比手写 `sys.argv` 解析稳健得多。第13章讲项目工程化时会用到。

`stdout` 的编码同样值得注意:Windows 控制台默认 GBK,print 中文一般没问题,但输出重定向到文件时按控制台编码写入,可能乱码或报错。生产脚本开头 `sys.stdout.reconfigure(encoding="utf-8")` 统一输出编码,是跨平台排障的常规动作。

§7.8 生产实践:临时文件与错误处理

写文件最容易半途而废:磁盘满、进程被杀,目标文件只剩一半。两条生产铁律:**原子替换**——先写临时文件,成功后 `rename` 覆盖;以及**用 `tempfile` 生成临时文件**,不自己拼路径:

```

import os
import tempfile
from pathlib import Path

def atomic_write(path: Path, content: str) -> None:
    """先写同目录临时文件,再原子替换,避免写一半的坏文件"""
    fd, tmp_name = tempfile.mkstemp(
        dir=path.parent, prefix=path.name + ".", suffix=".tmp")
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            f.write(content)
        os.replace(tmp_name, path) # 原子操作:要么全有,要么全无
    except BaseException:
        os.unlink(tmp_name)      # 出错清理,不留垃圾文件
        raise

atomic_write(Path("config.json"), '{"env": "prod"}')

```

工程建议 2:所有文件读写都放进 with;能预估会失败的解析(JSON/编码/struct 长度)用 try/except 包住并给出带文件名的错误信息: `except OSError as e: raise RuntimeError(f"读取 {path} 失败") from e` ——from 保留原始异常链,排障时能看到根因。

误区:直接覆盖用户文件,出错即毁灭。w 模式打开瞬间就把原内容清空,如果写入中途抛异常,原文件已不可恢复。写重要配置、报表、索引文件一律走"临时文件 + os.replace"的原子替换,这是生产文件写入的基本盘。

最后回顾本章主线:路径用对象、打开必带编码、读大文件走迭代、写重要文件走原子替换、交换格式统一 JSON/CSV。文件 I/O 没有玄学,只有"模式、编码、缓冲、原子性"四个词——每个词对应一个 C 时代的经典事故,也对应一条 Python 时代的纪律。

本章速查表

场景	写法
拼接路径	<code>Path("data") / "logs" / f.name</code>
读小文件	<code>Path(p).read_text(encoding="utf-8")</code>
写小文件	<code>Path(p).write_text(s, encoding="utf-8")</code>
流式逐行	<code>for line in open(p, encoding="utf-8"):</code>
追加写	<code>open(p, "a", encoding="utf-8")</code>
独占新建	<code>open(p, "x", ...)</code> ,已存在则报错
剥 BOM	<code>encoding="utf-8-sig"</code>
二进制打包	<code>struct.pack(">iif", 7, 100, 3.14)</code>
JSON 序列化	<code>json.dumps(obj, ensure_ascii=False, indent=2)</code>
CSV 读	<code>csv.DictReader(f)</code> ,取行用 <code>row["表头"]</code>
CSV 写	<code>csv.DictWriter + newline=""</code> + 统一编码
命令行参数	<code>sys.argv[1:]</code> ,参数多上 <code>argparse</code>
错误输出	<code>print(msg, file=sys.stderr)</code>
原子写文件	<code>tempfile + os.replace(tmp, target)</code>
递归找文件	<code>Path(root).glob("**/*.log")</code>
建目录	<code>p.mkdir(parents=True, exist_ok=True)</code>

最小必会清单

1. 能解释 w/a/x/r 四种模式的区别,并说出 w 的危险性与原子替换方案
2. 能说出为何所有文本 open 都要显式指定 encoding,以及 utf-8-sig 的用途
3. 能用 `for line in f` 流式统计 10 GB 文件,并解释为什么内存不涨
4. 能用 struct 打包/解包定长二进制记录,并理解字节序前缀的含义
5. 能用 json 读写中文文件(`ensure_ascii=False`)与 DictReader 处理带表头 CSV
6. 能写一个读 sys.argv、向 stdout 输出、错误走 stderr 的命令行工具

自测题

Q

Q1. `open("a.txt", "w")` 打开一个已有重要数据的文件会发生什么?正确的做法是什么?

答案:文件内容在打开瞬间被清空,后续写入失败数据也不可恢复。正确做法:先写同目录临时文件(`mkstemp`),全部成功后再 `os.replace` 原子替换;失败时删除临时文件。若只想追加内容,用 "a" 模式。

Q

Q2. 在 Windows 上执行 `open("f.txt").read()` 处理 UTF-8 文件,常报 `UnicodeDecodeError`,为什么?怎么修?

答案:未指定 encoding 时 Windows 默认用区域语言编码(GBK)解码,UTF-8 的中文字符序列在 GBK 下不合法。修复:显式 `encoding="utf-8"`;若文件带 BOM 用 `"utf-8-sig"`。生产代码所有文本 `open` 一律显式声明编码。

Q

Q3. 为什么说 `for line in f` 处理大文件内存恒定?如果写成 `f.read().splitlines()` 会怎样?

答案:文件对象是惰性迭代器,每次只从磁盘读一块并切出下一行,行处理完即释放,峰值内存与文件总大小无关。`f.read()` 一次性把全部内容装入内存,`splitlines()` 再复制一份列表,10 GB 文件直接 OOM。

Q

Q4. `struct.pack(">ii", 1, 2)` 与 `struct.pack("ii", 1, 2)` 的字节差异是什么?生产中选择哪种?

答案:前者显式大端,输出 `00 00 00 01 00 00 00 02`;后者不指定字节序,默认用本机字节序(小端机器上是 `01 00 00 00 02 00 00 00`)。生产与外部系统交换二进制一律显式指定字节序("`>`" 大端或 "`<`" 小端),避免跨平台解析错乱。

Q

Q5. `json.dumps(d)` 输出里中文全是 `\uXXXX`,影响排障与 diff,如何解决?还有哪些类型不能直接序列化?

答案:加 `ensure_ascii=False`,中文原样输出,文件再以 UTF-8 编码保存。不能直接序列化的类型:`set`、`datetime`、`bytes`、自定义类实例;解决方式是 `dumps` 的 `default` 参数提供转换函数,或先把数据转成可序列化的 `dict/list`。

Q

Q6. 用 `open("staff.csv", "w", encoding="utf-8", newline="")` 写 CSV 后,为什么必须带 `newline=""`?

答案:不带 `newline=""` 时,通用换行模式会把 `\n` 转成平台换行(Windows 为 `\r\n`),而 `csv` 模块内部已为字段内的换行加了引号与 `\r\n` 处理,双重转换导致 Excel 打开出现空行或错行。显式 `newline=""` 关闭转换,让 `csv` 模块全权负责换行。

章末实验题:日志解析与统计

实验 7.1 日志解析与统计工具(覆盖:\$7.1–\$7.7)

需求:① 接受命令行参数:输入日志路径(必选)、输出报告路径(可选,默认 `stdout`);② 按行流式解析形如 `[2026-08-01 10:00:03] [INFO] user=alice op=login` 的日志;③ 统计:各级别条数、活跃用户数、总行数、按小时分桶统计 `ERROR`;④ 结果输出为 CSV(表头 `time,level,count`)与一段摘要文本;⑤ 路径不存在或格式损坏时给出带文件名的清晰错误并返回非零退出码;⑥ 覆盖知识点自检清单写入报告文件(可选扩展)。

覆盖知识点:`pathlib` 路径处理(\$7.1)/`open` 模式与编码(\$7.2)/流式逐行读(\$7.3)/`csv` 输出与编码(\$7.6)/`sys.argv` 与 `stderr`(\$7.7)/异常与原子写(\$7.8)。

实验环境:Python 3.10+,单文件 `logstat.py`。

```

# logstat.py — 日志解析与统计(覆盖第7章全部知识点)
import csv
import re
import sys
from collections import Counter, defaultdict
from pathlib import Path

LINE_RE = re.compile(
    r"^\[(?P<ts>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]"
    r" \[(?P<level>\w+)\] user=(?P<user>\w+)"

def parse_line(line: str) -> dict | None:
    """解析一行日志;格式不对返回 None,不抛异常"""
    m = LINE_RE.match(line.strip())
    if not m:
        return None
    ts = m.group("ts")
    return {
        "hour": ts[11:13],          # 取 "10" 作为小时桶
        "level": m.group("level"),
        "user": m.group("user"),
    }

def main() -> None:
    if len(sys.argv) < 2:
        print("用法: python logstat.py 日志文件 [报告.csv]",
              file=sys.stderr)
        sys.exit(1)
    src = Path(sys.argv[1])
    if not src.is_file():
        print(f"错误: 文件不存在 {src}", file=sys.stderr)
        sys.exit(2)

    levels = Counter()
    hours = defaultdict(int)
    users = set()
    total = 0
    with open(src, encoding="utf-8") as f:
        for line in f:
            total += 1
            rec = parse_line(line)
            if rec is None:
                continue
            levels[rec["level"]] += 1
            hours[rec["hour"]] += 1
            users.add(rec["user"])

    print(f"总行数: {total} 活跃用户: {len(users)}")
    for level, n in levels.most_common():
        print(f" {level}: {n}")
    print("ERROR 高峰时段:", hours.most_common(3))

```

```
if len(sys.argv) > 2:
    write_report(Path(sys.argv[2]), hours)
```

第二段:报告输出函数。main 只负责统计,输出职责交给独立函数,便于单独测试。

```
def write_report(out: Path, hours: dict) -> None:
    """把小时桶统计写成 CSV;utf-8-sig 保证 Excel 打开不乱码"""
    with open(out, "w", encoding="utf-8-sig",
              newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["hour", "count"])
        for hour, n in sorted(hours.items()):
            writer.writerow([hour, n])
    print(f"报告已写入 {out}")

if __name__ == "__main__":
    main()
```

测试建议:先生成样例日志: `for i in $(seq 1000); do echo "[2026-08-01 10:$(date +%M):00] [INFO] user=alice op=login"; done > sample.log`,再运行 `python logstat.py sample.log report.csv`;故意删掉一行开头的 [验证容错;把文件编码改成 GBK 验证编码错误信息是否清晰。对照检查清单:路径用 pathlib(是)、显式编码(是)、流式读(是)、CSV 输出带 `newline=""`(是)、argv 与 stderr(是)、非零退出码(是)。

下一章预告:第8章 网络通信——socket 从 C 迁移、TCP/UDP 语义、粘包与长度前缀协议、requests 与 http.server 最小 Web 服务。数据开始跨越进程与机器。

第8章 网络通信

socket 迁移 · TCP/UDP · 粘包与协议设计 · requests · 最小 Web 服务

本章目标:以 C 的 socket API 为对照,讲透 Python 网络编程:TCP/UDP 客户端与服务器、粘包问题与长度前缀协议、超时与异常处理,再上层到 requests 调用 HTTP API、http.server 搭建最小 Web 服务。学完你能写出可靠、有超时、有重试的生产级网络程序。

直觉起点:C 里网络是"文件描述符 + 一串套接字函数";Python 的 socket 模块 API 与 C 几乎一一对应,迁移成本极低。真正的难点不在 API,而在网络本身的三条铁律:数据可能不来、可能迟到、可能不完整。所有生产实践——超时、重试、长度前缀——都是对这三条铁律的回应。

前置要求:第7章文件与编码(网络收发的本质是字节流)、第5章 with 与资源管理。

本章路线:第一阶段(1.5 小时)读 §8.1–§8.3,理解 socket 流程与 TCP 语义;第二阶段(1.5 小时)读 §8.4–§8.5,掌握协议设计与 UDP;第三阶段(2 小时)读 §8.6–§8.8,完成 HTTP 层并做章末实验。

本章知识导图

- └─ 8.1 socket 迁移 ── C 到 Python 的一一对应
- └─ 8.2 TCP 客户端 ── connect、收发与超时
- └─ 8.3 TCP 服务器 ── bind、listen、accept 循环
- └─ 8.4 粘包与协议 ── 长度前缀解决一切
- └─ 8.5 UDP ── 无连接的取舍
- └─ 8.6 HTTP 客户端 ── urllib 与 requests
- └─ 8.7 http.server ── 40 行内的 Web 服务
- └─ 8.8 生产实践 ── 超时、重试与异常

§8.1 socket 迁移:API 几乎一一对应

C 的 socket 编程流程:创建套接字、connect/bind、send/recv、close。Python 的 **socket** 模块就是这套 API 的直译,连常量名都一样。先看两端完整的 TCP 客户端对照:

C(socket + connect)	Python(socket)
socket(AF_INET, SOCK_STREAM, 0)	socket.socket(socket.AF_INET, socket.SOCK_STREAM)
struct sockaddr_in addr; ...	sock.connect(("host", 8080)) ──元组即地址
send(buf, len, 0) 返回字节数	sock.sendall(data) ──一次发完,内部循环
recv(buf, N, 0) 返回字节数	sock.recv(1024) 返回 bytes,空串=对端关闭
close(fd)	sock.close(),配 with 自动关闭
errno 判断错误类型	异常体系:ConnectionRefusedError 等

Python 的三个语法级改进值得记住:地址用元组 ("host", port),不用手填 sockaddr;发送用 **sendall** 而不是 send(后者可能只发一半);套接字支持 with,退出自动关闭。其余流程——connect、bind、listen、accept——与 C 完全同构。

迁移时的心理建设:Python 屏蔽了 C 里最磨人的部分——地址结构体、字节序转换、errno 错误分支——但网络语义(流、阻塞、半关闭)完全一致。你在 C 里学会的每一个网络概念,在 Python 里都能原样复用,差别只在代码量少了一个量级。

§8.2 TCP 客户端:连接、收发与超时

```
import socket

with socket.create_connection(("127.0.0.1", 9000), timeout=5) as s:
    # create_connection 自动完成 getaddrinfo + connect
    s.sendall(b"GET /time HTTP/1.0\r\nHost: localhost\r\n\r\n")

    chunks = []
    while True:
        data = s.recv(4096)
        if not data:                # 空 bytes:对端已关闭连接
            break
        chunks.append(data)
    print(b"".join(chunks).decode("utf-8", errors="replace"))
```

误区:recv 一次就想收完。 TCP 是字节流,recv 返回多少完全由网络与内核决定——可能只有 1 字节,也可能分批到达。recv(4096) 只保证"最多 4096",不保证"你要的都在里面"。收消息的唯一正确姿势是循环 recv,直到满足协议条件(定长、长度前缀或连接关闭)。

create_connection 是生产首选:它内部处理了 IPv4/IPv6 解析与自动重试,还接受 timeout 参数。超时是网络程序的第一安全阀:不设超时,服务端挂死时客户端会无限期阻塞,线程、连接全部耗尽。超时异常是 socket.timeout,记得捕获并转成业务错误。

超时单位是秒,可以是浮点数。设了超时后,阻塞的 recv/accept 到点抛 socket.timeout,连接并未失效——但生产上超时后重试更稳妥。调试网络程序时先把超时调大避免误判,再谈协议正确性;先确认连接通不通,再排查协议细节。

§8.3 TCP 服务器:bind、listen、accept

```
import socket

def handle(conn: socket.socket, addr: tuple) -> None:
    """处理单个客户端连接:回显收到的内容"""
    with conn:
        print(f"连接来自 {addr}")
        while True:
            data = conn.recv(4096)
            if not data:
                break
            conn.sendall(data)      # 原样回显

def main() -> None:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 9000))
        srv.listen(8)             # 等待队列长度
        print("监听 9000 端口...")
        while True:
            conn, addr = srv.accept()    # 阻塞等待新连接
            handle(conn, addr)           # 串行处理(演示用)

if __name__ == "__main__":
    main()
```

误区:accept 的连接忘了 close。 accept 返回的新连接是独立资源,用完不关会堆积,达到上限后新连接全部失败。用 with conn: 包裹处理逻辑即可自动关闭。另一个经典坑:服务器进程重启时 bind 报 Address already in use——setsockopt(SO_REUSEADDR, 1) 是服务器标配,否则调试期频繁被杀重启。

上面的服务器是串行的:一个客户端占着连接,其他客户端全部排队。生产要同时服务多个客户端,有两条路:每连接开线程(第9章讲),或事件循环(asyncio)。现在先理解串行版,它把 accept 循环讲清楚了。

理解 accept 的语义:它在监听队列上取一个已完成握手的连接;队列满时内核会丢弃新的握手请求。listen(n) 的 n 是等待队列深度,不是并发数。生产并发要靠线程或事件循环——第9章会把 threading 与 asyncio 讲透,届时回头看这个串行版,架构演进脉络就清楚了。

§8.4 粘包与协议设计:长度前缀

初学 TCP 最常见的困惑是"粘包":两次 send,对端一次 recv 全收到,以为消息黏在一起了。真相是 TCP 不保消息边界——它只保证字节有序到达。send("AB") 与 send("CD") 可能合并传输,也可能拆成三次。解决方案不是"避免粘包"(做不到),而是**设计协议,让接收方能确定消息边界**。

```
import struct

# 协议:4 字节长度前缀(大端) + 消息体
def send_msg(sock: socket.socket, payload: bytes) -> None:
    header = struct.pack(">I", len(payload)) # 定长 4 字节
    sock.sendall(header + payload)

def recv_exact(sock: socket.socket, n: int) -> bytes:
    """收满 n 字节为止—recv 可能分批到达"""
    buf = bytearray()
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("对端提前关闭")
        buf.extend(chunk)
    return bytes(buf)

def recv_msg(sock: socket.socket) -> bytes:
    header = recv_exact(sock, 4)
    (length,) = struct.unpack(">I", header)
    return recv_exact(sock, length)
```

C(手工组包)	Python(长度前缀)
htonl 转网络字节序,memcpy 进缓冲	struct.pack(">I", n) 一步完成
recv 循环自己写,容易漏边界	recv_exact 收满 n 字节,复用一次
缓冲区管理易越界	bytearray 自动扩容,长度即边界
每种协议重写一遍边界逻辑	send_msg/recv_msg 一对函数通用所有消息

长度前缀是二进制协议的基础设施,HTTP 的 Content-Length、WebSocket 的帧头、主流 RPC 框架都基于同一思想。上限检查别忘了:recv_exact 前先校验 length 不超约定上限(如 10 MB),否则恶意端一个 4 GB 的长度字段就能拖垮接收方内存。

协议设计三原则:长度字段定宽(4 字节足以表达 4 GB)、先验后收(长度超限直接拒绝)、预留版本字段(将来加字段不破坏旧客户端)。内部系统协议别贪多,够用即止——每多一个字段,就要多写一组兼容测试。

§8.5 UDP:无连接的取舍

UDP 无连接、不保证到达、不保证顺序,换来低延迟与无握手开销。Python 的用法与 TCP 对称但更简单——没有 accept,一个套接字收发都行:

```
import socket

# 服务器:绑定端口,循环收包
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    while True:
        data, addr = srv.recvfrom(4096)      # 一次收一个完整数据报
        print(f"{addr} 发来 {data}")
        srv.sendto(data, addr)              # 回给同一个地址

# 客户端:无需连接,直接发
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cli:
    cli.sendto(b"ping", ("127.0.0.1", 9001))
    reply, _ = cli.recvfrom(4096)
```

注意 `recvfrom` 返回 (数据, 对端地址), 一个数据报对应一次 `recvfrom`——UDP 天然保留消息边界, 没有粘包问题。代价是丢包与乱序: 生产 UDP 程序要自带序号、重传与超时确认(或者直接用 QUIC 这类成熟协议, 别自己造轮子)。

什么场景选 UDP: 实时音视频、游戏位置同步、DNS、日志采集——丢一帧无所谓, 延迟敏感。什么场景绝不选 UDP: 交易、订单、配置下发——凡是“丢一条消息会出事故”的业务, 老老实实用 TCP 或可靠传输协议, 这是网络选型的第一课。

§8.6 HTTP 客户端: `urllib` 与 `requests`

调 HTTP API 是网络编程的最高频场景。标准库 `urllib` 能干活但笨重; 生产事实标准是第三方库 `requests` (`pip install requests`)。它的三个核心能力: `get/post`、JSON 自动编解码、响应状态检查:

```
import requests

resp = requests.get(
    "https://api.example.com/v1/orders/A1001",
    timeout=(3.05, 10),          # (连接超时, 读取超时)
)
resp.raise_for_status()         # 4xx/5xx 抛 HTTPError

order = resp.json()             # 自动按 Content-Type 解析
print(order["status"])

resp2 = requests.post(
    "https://api.example.com/v1/orders",
    json={"sku": "P-001", "qty": 2}, # json= 自动序列化并带头
    timeout=10,
)
print(resp2.status_code, resp2.json())
```

误区: 请求不带 `timeout`。 不设 `timeout` 的 `requests` 请求可能永远挂着(默认无超时!), 线程池、任务队列、重试循环全部被一个慢服务拖死。生产规范: 连接超时 3 秒、读取超时按业务给 10–30 秒, 并捕获 `requests.Timeout` 单独处理——超时不该被当作普通网络错误。

`requests` 的其他生产要点: `params=` 传查询参数(自动 URL 编码)、`headers=` 传认证头、`verify=False` 仅测试环境用并配警告抑制、`Session` 复用连接池(大量请求时性能差距明显)。

requests 的响应对象是“一次性的”:resp.content 与 resp.text 读写的是同一块解码缓冲,大响应别反复取。流式大文件下载用 iter_content(chunk_size) 按块迭代,避免整包进内存——这是网络版的 for line in f,与第7章的大文件哲学一脉相承。

§8.7 http.server:40 行内的最小 Web 服务

标准库 http.server 可以在一分钟内部署一个能处理业务逻辑的 HTTP 服务——适合内部工具、测试桩、原型。核心是继承 BaseHTTPRequestHandler 覆写 do_GET/do_POST:

```
import json
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

ORDERS = {}                                # 内存存储,演示用

class Handler(BaseHTTPRequestHandler):
    def do_GET(self) -> None:
        if self.path == "/health":
            body = json.dumps({"status": "ok"}).encode("utf-8")
            self.send_response(200)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)
        else:
            self.send_error(404, "未找到资源")

    def do_POST(self) -> None:
        if self.path == "/orders":
            length = int(self.headers.get("Content-Length", 0))
            data = json.loads(self.rfile.read(length))
            oid = f"B{len(ORDERS) + 1:04d}"
            ORDERS[oid] = data
            body = json.dumps({"order_id": oid}).encode("utf-8")
            self.send_response(201)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)
        else:
            self.send_error(404)

if __name__ == "__main__":
    ThreadingHTTPServer(("127.0.0.1", 8080), Handler).serve_forever()
```

C(手写 HTTP 服务器)	Python(http.server)
解析请求行、头部、Content-Length	BaseHTTPRequestHandler 已解析,self.path/self.headers 就绪
手动拼响应头与状态行	send_response/send_header/end_headers 三连
线程模型自己管	ThreadingHTTPServer 每连接一线程
读请求体要自写循环	<code>self.rfile.read(length)</code> 按长度读取

BaseHTTPRequestHandler 的 self.path 是请求行里的原始路径,self.headers 是邮件头格式的头部容器;query string 要自己从 self.path 中解析。模板渲染、静态文件、表单解析这些能力标准库给不了——原型验证阶段手写够用,规模上来就换框架。

误区:把 http.server 当生产 Web 服务器。 它只监听一个端口、无 HTTPS、无并发调优,只适合内网工具与测试。生产服务请用 FastAPI/Flask 加成熟服务器,或直接交给云平台。认清边界,工具才不伤人。

§8.8 生产实践:超时、重试与异常

网络代码的三条铁律(可能不来、可能迟到、可能不完整)在生产中落到三条纪律:

```
import time
import requests
from requests.exceptions import RequestException

MAX_RETRIES = 3
BACKOFF = 1.0

def fetch_with_retry(url: str, timeout: float = 10) -> dict:
    """带超时与退避重试的 GET;指数退避避免雪崩"""
    for attempt in range(MAX_RETRIES):
        try:
            resp = requests.get(url, timeout=timeout)
            resp.raise_for_status()
            return resp.json()
        except (requests.Timeout, requests.ConnectionError):
            if attempt == MAX_RETRIES - 1:
                raise RuntimeError(f"请求 {url} 连续失败") from None
            time.sleep(BACKOFF * (2 ** attempt)) # 1s, 2s
        except requests.HTTPError as e:
            if resp.status_code in (429, 500, 502, 503, 504):
                time.sleep(BACKOFF * (2 ** attempt))
                continue
            raise RuntimeError(f"HTTP {resp.status_code}: {url}") from e
    raise RuntimeError("不可达") # 不会执行到,类型安全兜底
```

工程建议 1:重试只对“可重试”的错误生效:超时、连接拒绝、5xx、429 可以重试;4xx 业务错误重试也没用,直接失败并记日志。重试要有上限与退避,否则下游故障时你的重试风暴会把它彻底打垮——这是生产事故的经典剧本。

工程建议 2:网络异常永远带上下文:捕获后重新抛出时注明 URL、方法、耗时,如 `raise RuntimeError(f"GET {url} 超时") from e`。from 保留原始异常链,排障时一眼看到根因;绝不裸 except 吞掉异常——静默失败是分布式系统最贵的 bug。

总结网络编程的黄金三角:超时兜底(永不无限等待)、协议显式(边界、长度、版本)、异常可查(上下文、异常链、日志)。把这三条写进团队代码规范,网络事故率会直线下降——它们本质上是把"网络不可靠"这个事实内化成代码结构。

本章速查表

场景	写法
TCP 客户端	<code>socket.create_connection((host, port), timeout=5)</code>
发送全部字节	<code>sock.sendall(data)</code> ,不用 <code>send</code>
循环接收	<code>while True: d = sock.recv(4096); if not d: break</code>
TCP 服务器	<code>bind</code> → <code>listen</code> → <code>accept</code> ,with 管理连接
端口复用	<code>srv.setsockopt(SO_REUSEADDR, 1)</code>
定长收发	4 字节大端长度前缀 + <code>recv_exact</code>
UDP 收	<code>data, addr = srv.recvfrom(4096)</code>
UDP 发	<code>sock.sendto(data, addr)</code>
HTTP GET	<code>requests.get(url, timeout=(3, 10))</code>
HTTP POST	<code>requests.post(url, json={...}, timeout=10)</code>
状态检查	<code>resp.raise_for_status()</code>
最小 Web 服务	<code>BaseHTTPRequestHandler</code> + <code>ThreadingHTTPServer</code>
读请求体	<code>self.rfile.read(int(self.headers["Content-Length"]))</code>
超时处理	捕获 <code>requests.Timeout</code> ,单独分支
重试	指数退避,只重试超时/5xx/429
异常链	<code>raise RuntimeError("...") from e</code>

最小必会清单

1. 能默写 `create_connection` + `sendall` + 循环 `recv` 的 TCP 客户端骨架
2. 能写出 `bind/listen/accept` 的服务器骨架,并说出两个经典坑(忘 `close`、端口占用)
3. 能解释"粘包"的本质,并用长度前缀协议实现 `send_msg/recv_msg`
4. 能说出 UDP 与 TCP 的取舍,以及 `recvfrom/sendto` 的用法
5. 能用 `requests` 完成带超时、状态检查、JSON 解析的 API 调用
6. 能用 `http.server` 在 40 行内搭出 GET/POST 的最小服务

自测题

Q

Q1. 为什么 `sendall` 比 `send` 可靠? `recv` 一次返回 100 字节,能说明对端只发了 100 字节吗?

答案:send 只尝试发送,返回实际发送字节数,可能小于请求长度(发送缓冲满或中断);sendall 内部循环发送直到全部完成或出错。不能——TCP 是字节流,recv 一次返回值只代表"当前内核缓冲区可用数据",可能与发送方的单次 send 无关,必须按协议边界循环接收。

Q

Q2. 服务器运行中端口报 "Address already in use",可能原因与解法?

答案:上一进程的套接字处于 TIME_WAIT 状态,或另一进程占用该端口。解法:① 服务器创建时 setsockopt(SO_REUSEADDR, 1) 允许重用;② 确认旧进程已退出(或端口被其他程序占用,用 netstat/ss 排查)。SO_REUSEADDR 是服务器标配,客户端不需要。

Q

Q3. 长度前缀协议中,收方为什么要先 recv_exact(4) 再 recv_exact(length)?直接把两段合并 recv 不行吗?

答案:不行,因为 recv 不保证一次拿到任意指定长度。先收满固定的 4 字节头才能解出消息长度,再按该长度收满消息体;两个阶段都依赖"收满为止"的循环(recv_exact)。合并 recv 会出现头只到一半、体只到一半、头体一起来等各种情况,只有循环收满才能统一处理。

Q

Q4. requests.get 不设 timeout 会怎样?连接超时与读取超时分别指什么?

答案:默认无超时,请求可能无限期挂起,线程与连接被耗尽。连接超时指建立 TCP 连接的最长等待(如 3 秒);读取超时指连接建立后两次数据到达之间的最长间隔(如 10 秒)。requests 用元组 (connect, read) 分别设置,生产必须显式给出。

Q

Q5. 一次请求返回 500 后,直接原样重试 3 次,有什么风险?正确做法?

答案:风险:若服务端正在过载,你的重试会加大压力,且无退避的瞬时重试几乎必然再失败,形成重试风暴。正确做法:只对可重试错误(超时、连接错误、5xx、429)重试,指数退避(1s、2s、4s)并加随机抖动,设上限次数;4xx 业务错误不重试,立即失败并记录。

Q

Q6. BaseHTTPRequestHandler 里,为什么读请求体要用 Content-Length 而不是 recv 到空?

答案:HTTP 请求体是定长传输(Content-Length 声明)或分块传输;请求结束后连接可能复用,recv 到空要等对端关闭连接,而 keep-alive 下对端不会关闭——直接死等。正确姿势是读满 Content-Length 声明的字节数(与 recv_exact 同理),这也是 HTTP 语义下"长度前缀"的体现。

章末实验题:带协议的文件传输 + 统计 API

实验 8.1 长度前缀文件传输与 HTTP 统计接口(覆盖:\$8.2-\$8.7)

需求:① 服务器端:监听 9100,用长度前缀协议接收"文件名(短串)+文件内容",存到 receive/ 目录;② 客户端:把本地任意文件按协议发过去,打印传输字节数与耗时;③ 服务器端另开 8081 端口提供 HTTP API:GET /files 返回已收文件列表

JSON,GET /files/<名> 返回文件内容;④ 双方都设置超时与异常处理,服务器打印每笔传输的日志;⑤ 用 requests 写一个测试脚本调用 API 验证。

覆盖知识点:TCP 客户端/服务器(\$8.2、\$8.3)/长度前缀协议与 recv_exact(\$8.4)/http.server 最小服务(\$8.7)/超时与异常链(\$8.8)。

实验环境:Python 3.10+,两个终端分别运行服务端与客户端。

```

# file_server.py — 长度前缀文件接收 + HTTP 统计 API
import json
import socket
import struct

from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path

HOST, PORT = "127.0.0.1", 9100
SAVE_DIR = Path("receive")
SAVE_DIR.mkdir(exist_ok=True)
MAX_LEN = 10 * 1024 * 1024          # 协议上限 10 MB


def recv_exact(conn: socket.socket, n: int) -> bytes:
    buf = bytearray()
    while len(buf) < n:
        chunk = conn.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("对端提前关闭")
        buf.extend(chunk)
    return bytes(buf)


def recv_str(conn: socket.socket) -> str:
    (length,) = struct.unpack(">I", recv_exact(conn, 4))
    if length > 256 or length < 1:
        raise ValueError(f"非法文件名长度: {length}")
    return recv_exact(conn, length).decode("utf-8")


def serve_tcp() -> None:
    """协议: [4字节文件名长][文件名][4字节内容长][内容]"""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind((HOST, PORT))
        srv.listen(4)
        print(f"文件服务监听 {HOST}:{PORT}")
        while True:
            conn, addr = srv.accept()
            with conn:
                conn.settimeout(30)
                try:
                    name = recv_str(conn)
                    (size,) = struct.unpack(">I", recv_exact(conn, 4))
                    if size > MAX_LEN:
                        raise ValueError(f"文件过大: {size}")
                    content = recv_exact(conn, size)
                    (SAVE_DIR / name).write_bytes(content)
                    print(f"[{addr}] 收到 {name} ({size} 字节)")
                except (ConnectionError, ValueError, socket.timeout) as e:
                    print(f"[{addr}] 传输失败: {e}")

```

```

class FileAPI(BaseHTTPRequestHandler):
    """GET /files 列表;GET /files/<name> 取文件"""

    def _json(self, code: int, obj: dict) -> None:
        body = json.dumps(obj).encode("utf-8")
        self.send_response(code)
        self.send_header("Content-Type", "application/json; charset=utf-8")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self) -> None:
        if self.path == "/files":
            names = [p.name for p in SAVE_DIR.iterdir()]
            self._json(200, {"files": names})
            return
        if self.path.startswith("/files/"):
            name = self.path.removeprefix("/files/")
            target = SAVE_DIR / name
            if target.is_file():
                self._json(200, {"name": name,
                                "size": target.stat().st_size})
            else:
                self._json(404, {"error": "文件不存在"})
            return
        self._json(404, {"error": "未找到"})

    def log_message(self, fmt: str, *args) -> None:
        print(f"[http] {self.address_string()} {fmt % args}")

def main() -> None:
    import threading
    tcp = threading.Thread(target=serve_tcp, daemon=True)
    tcp.start()
    httpd = ThreadingHTTPServer(("127.0.0.1", 8081), FileAPI)
    print("HTTP API 监听 8081")
    httpd.serve_forever()

if __name__ == "__main__":
    main()

```

客户端与验证:客户端用 `create_connection` 连接 9100,按同一协议把文件名与内容发送出去;测试脚本用 `requests.get("http://127.0.0.1:8081/files")` 验证列表接口。对照检查清单:`sendall` 发送(是)、`recv_exact` 收满(是)、长度上限检查(是)、超时(连接与读取都有)、异常带上下文(是)、HTTP 按 `Content-Length` 读(是)、with 管理资源(是)。

下一章预告:第9章 异常处理与并发——try/except 完整体系、GIL 是什么与为什么、threading 与 queue、asyncio 入门。让程序在错误与并发面前保持优雅。

第9章 异常处理与并发

异常体系 · GIL 真相 · threading 与锁 · queue · asyncio 入门

本章目标:系统讲透两件事:Python 的异常处理体系(try/except/else/finally、raise from、自定义异常),以及并发编程(线程、GIL、锁与死锁、queue、多进程、asyncio 入门)。学完你能写出错误可查、并发安全的生产代码,并理解 Python 并发模型的边界与选型。

直觉起点:C 用返回码与 errno 报告错误,漏查就静默;Python 用异常,不处理就直接抛穿。并发侧,C 的 pthread 是"真并行";Python 的线程受 GIL 约束是"并发不并行"——理解 GIL 的成因,才能正确选型线程、进程与协程。

前置要求:第4章函数与作用域、第6章类与继承(异常也是类)、第8章网络(超时与重试已用到异常)。

本章路线:第一阶段(1.5 小时)读 §9.1–§9.2,掌握异常体系;第二阶段(2 小时)读 §9.3–§9.6,理解 GIL 并写出线程安全代码;第三阶段(2 小时)读 §9.7–§9.8,掌握多进程与 asyncio,完成章末实验。

本章知识导图

- └─ 9.1 异常体系 —— try/except/else/finally 全景
- └─ 9.2 抛异常 —— raise、raise from、自定义异常
- └─ 9.3 GIL 真相 —— 为什么有、影响了什么
- └─ 9.4 threading —— 线程的创建与生命周期
- └─ 9.5 锁与竞态 —— Lock、死锁、临界区
- └─ 9.6 queue —— 线程安全的生产者-消费者
- └─ 9.7 multiprocessing —— 绕开 GIL 的并行
- └─ 9.8 asyncio —— 事件循环与 async/await

§9.1 异常体系:try/except/else/finally 全景

异常是类,按继承树组织: `BaseException` 是根, `Exception` 是"可捕获的业务与系统错误"基类,再往下是 `ValueError`、`TypeError`、`KeyError`、`OSError` 等。捕获是按继承关系匹配的,所以捕获顺序要先子类后父类:

```
try:
    data = json.loads(raw)
    score = data["score"] / data["count"]
except KeyError:
    print("字段缺失")
except (TypeError, ValueError) as e:
    print(f"数据不合法: {e}")          # 多个异常共用一个分支
except ZeroDivisionError:
    print("count 为 0")
except Exception as e:
    print(f"未知错误: {e}")           # 兜底,但别滥用
else:
    print(f"无异常,正常处理 score={score}")
finally:
    print("无论如何都会执行:清理资源")
```

C(返回码 + errno)	Python(异常)
漏查返回值,错误被静默吞掉	不捕获异常就向上传播,不会被静默
错误路径与正常路径混在代码里	try 块聚焦正常逻辑,错误处理分离
errno 是全局变量,嵌套调用互相覆盖	异常对象携带完整上下文,链式传递
释放资源要手动检查每个分支	finally/with 保证清理必然执行

两个关键语义:else 分支"只有 try 无异常才执行"——把"预期成功后的动作"与"错误处理"分开;finally 无论 return 与否都执行,是资源清理的最后防线。注意 `except Exception` 捕获不了 `KeyboardInterrupt` 与 `SystemExit`(它们继承 `BaseException`),这是有意的设计。

误区:裸 `except:` 吞掉一切。 `except:` 连 `KeyboardInterrupt`、`SystemExit` 一起吞,程序 `Ctrl+C` 都退不出来;再配上空 `pass`,错误信息全部消失,排障靠猜。生产规范:捕获具体异常,处理不了的用 `raise` 原样重抛,或转成带上下文的业务异常。

§9.2 抛异常与自定义异常

主动抛异常是"前置校验"的 Python 方式: `raise ValueError(...)` 在入口处拦截非法参数,比让错误在深处爆发更容易定位。生产项目通常自定义异常基类,让调用方能按业务维度捕获:

```
class PaymentError(Exception):
    """支付业务异常基类:所有支付错误都继承它"""

class InsufficientBalanceError(PaymentError):
    """余额不足"""

class AccountLockedError(PaymentError):
    """账户被锁定"""

def pay(user: dict, amount: float) -> None:
    if user.get("locked"):
        raise AccountLockedError(f"账户 {user['name']} 已锁定")
    if user["balance"] < amount:
        raise InsufficientBalanceError(
            f"余额 {user['balance']} 不足 {amount}")
    user["balance"] -= amount

try:
    pay({"name": "李雷", "balance": 10, "locked": False}, 100)
except PaymentError as e:
    # 一个分支捕获整个业务族
    print(f"支付失败: {e}")
```

异常链 `raise ... from e` 保留"底层原因"——处理 `requests` 异常时尤其重要:

```

try:
    resp = requests.get(url, timeout=10)
    resp.raise_for_status()
    return resp.json()
except requests.Timeout as e:
    raise PaymentError(f"支付网关超时: {url}") from e
except requests.HTTPError as e:
    raise PaymentError(
        f"支付网关返回 {resp.status_code}") from e

```

输出里会显示完整的"直接原因"链,一眼看到最底层是连接超时还是解析失败。这是生产排障的核心工具,绝不写成裸 raise 丢上下文。

§9.3 GIL 真相:为什么有、影响了什么

GIL(全局解释器锁)是 CPython 的一个互斥锁:任何时刻只有一个线程在执行 Python 字节码。为什么存在?因为 CPython 的内存管理(引用计数)不是线程安全的,加一把全局锁让整个解释器原子化,代价是牺牲多核并行。它不影响"并发"——多个线程仍然轮流执行,只是不能同时推进。

C(pthread)	Python(threading)
多线程可同时跑在多核上,真并行	GIL 下字节码不并行,但 I/O 等待会让出锁
共享数据全靠自己加锁	同样要加锁;GIL 只保证字节码原子,不保证你代码原子
线程开销低,百万线程不现实	线程是系统线程,创建开销可观,数量控制在百级
计算密集任务靠线程加速	计算密集靠线程反而变慢(锁竞争),要用进程

GIL 对 I/O 密集任务几乎无感:网络请求、文件读写、数据库查询在等待期间会释放 GIL,其他线程照常运行。所以 web 服务、爬虫、API 调用用线程完全没问题;纯 CPU 计算(排序、图像处理)要并行,选 multiprocessing 或把热点交给 C 扩展。

误区:GIL 让我不需要加锁。 GIL 只保证单条字节码原子,不保证"读取-修改-写回"序列原子。count += 1 要经历取数、加一、写回三步,两个线程交错执行就会丢更新。共享可变数据的保护,Python 与 C 一样要靠锁,没有免费午餐。

§9.4 threading:线程的创建与生命周期

```
import threading
import time

def worker(name: str, delay: float) -> None:
    print(f"[{name}] 开始工作")
    time.sleep(delay)          # 模拟 I/O 等待
    print(f"[{name}] 完成")

threads = []
for i in range(3):
    t = threading.Thread(target=worker, args=(f"w{i}", 1.0))
    t.start()                  # 启动线程
    threads.append(t)

for t in threads:
    t.join()                   # 等待全部结束再退出
print("所有线程完成")
```

关键 API: `start()` 启动, `join()` 等待结束, `daemon=True` 的线程不阻塞进程退出(主线程结束即杀);线程间共享的是同一个进程内存,这正是竞态的温床。线程安全地收集结果,不要用全局 list 直接 `append`——用下面 §9.6 的 `queue`,或用 `concurrent.futures` 的线程池(生产首选)。线程函数的参数只能通过 `args` 元组传递,而且线程没有返回值通道——这正是 `queue` 与 `Future` 存在的意义,理解这一点,就理解了线程间通信为什么绕不开队列。

§9.5 锁与竞态:临界区、死锁

竞态(race condition)是并发程序的头号 bug:两个线程同时改一个变量,最终值取决于交错时机。标准解法是 **Lock**,把"读-改-写"包成临界区:

```
import threading

balance = 1000.0
lock = threading.Lock()

def transfer(amount: float) -> bool:
    """并发转账:先查后扣,临界区保护"""
    with lock:                  # 等价于 acquire/finally release
        if balance < amount:
            return False
        balance -= amount      # 读-判断-写 现在原子了
    return True
```

误区:锁的顺序不一致导致死锁。两个线程各持一把锁再等对方的锁,双方永远等下去——这就是死锁。防御规则:① 锁的获取顺序全局一致(如总是先锁 A 再锁 B);② 用 `with lock:` 而不是手动 `acquire/release`(异常时保证释放);③ 加锁范围最小化,不要在锁里做网络请求或 `sleep`;④ 实在复杂就用 `queue` 消除共享,无锁则无死锁。

另一个必懂概念:同一线程对同一把锁重复 with 会死锁——除非用 `threading.RLock` (可重入锁)。递归函数里加锁、锁里调用会再次加锁的同模块函数,都是 RLock 的典型场景。默认先写 Lock,出现重入需求再换。

§9.6 queue:线程安全的生产者-消费者

生产者-消费者是并发编程的基本架构:生产者产数据,消费者处理数据,中间用队列解耦。Python 的 `queue.Queue` 内部自带锁,是线程间传数据的标准容器——比"共享 list + 手写锁"少一半 bug:

```
import queue
import threading
import time

def producer(q: queue.Queue, n: int) -> None:
    for i in range(n):
        q.put(f"任务-{i}")          # 放任务,内部加锁
        time.sleep(0.05)

def consumer(q: queue.Queue, name: str) -> None:
    while True:
        item = q.get()              # 取任务,队列空则阻塞等待
        print(f"[{name}] 处理 {item}")
        q.task_done()              # 通知队列:一个任务完成

q = queue.Queue(maxsize=20)
threading.Thread(target=producer, args=(q, 10), daemon=True).start()
for i in range(2):
    threading.Thread(target=consumer, args=(q, f"c{i}"),
                      daemon=True).start()

q.join()                          # 等所有任务都被 task_done
print("全部消费完成")
```

三个方法组成了完整生命周期:put 放、get 取、task_done 报告完成,配套 join() 等待"所有已放任务完成"。队列满时 put 阻塞,队列空时 get 阻塞——生产里可以给超时(`q.get(timeout=5)`)或设置哨兵值(发 None 通知消费者退出)。

§9.7 multiprocessing:绕开 GIL 的并行

计算密集任务要真并行,用 `multiprocessing` 开多个进程——每个进程有自己的解释器与 GIL,多核真正同时计算。API 与 `threading` 高度相似,区别是内存不共享,数据要序列化后传递:

```
import multiprocessing as mp

def heavy(n: int) -> int:
    """模拟 CPU 密集:计算前 n 个数的平方和"""
    return sum(i * i for i in range(n))

if __name__ == "__main__":
    with mp.Pool(4) as pool:          # 4 个进程
        results = pool.map(heavy, [10_000_000] * 4)
    print(results)                    # 约 4 倍于单进程的耗时
```

C(fork/线程)	Python(进程选择)
I/O 密集:多线程	threading(受 GIL 影响小,轻量)
CPU 密集:多线程直接并行	multiprocessing.Pool(真并行,数据要可序列化)
共享内存传递结构体	进程间默认 pickle 传参,大对象开销高
进程启动代价可控	Python 进程启动重,池化复用是标配

注意三点:参数与返回值必须可 pickle(函数要模块级定义);Windows 上进程通过 spawn 创建,入口代码必须放 `if __name__ == "__main__":` 保护,否则无限递归;任务小而多时进程开销可能超过收益,先用 profile 数据说话。

§9.8 asyncio:事件循环与 async/await

asyncio 是单线程内的协作式并发:一个事件循环在多个协程之间切换,遇到 `await` 就切走,不阻塞也不占线程。适合海量 I/O 并发(万级连接)而线程做不到的场景:

```
import asyncio

async def fetch(name: str, delay: float) -> str:
    """协程:遇到 await 让出控制权,等待期间执行别的协程"""
    print(f"[{name}] 开始请求")
    await asyncio.sleep(delay)      # 模拟 I/O 等待(不占线程)
    print(f"[{name}] 响应到达")
    return name.upper()

async def main() -> None:
    tasks = [fetch("a", 1.0), fetch("b", 0.5), fetch("c", 0.2)]
    results = await asyncio.gather(*tasks)  # 并发跑三个协程
    print(results)                          # ['A', 'B', 'C']

if __name__ == "__main__":
    asyncio.run(main())                    # 创建事件循环并运行
```

理解要点:async def 定义协程;await 是让出点;asyncio.run 是入口;gather 并发收集结果。三个任务总耗时约 1 秒(最长者)而不是 1.7 秒——这就是并发的价值。写生产代码时,用 aiohttp 做 HTTP、asyncpg 连数据库,网络等待期间循环继续转。

误区:协程里混入阻塞调用。在 `async` 函数里直接 `requests.get` 或 `time.sleep`,会把整个事件循环卡死——所有协程一起等这一个阻塞调用。规则:`async` 世界里只能用异步库(`await` 版本);不得不用同步库时,丢到 `asyncio.to_thread` 里跑,别让它占住循环。

工程建议 1:并发选型口诀:I/O 密集且并发量适中(百级)用 `threading`;I/O 密集且海量(千级+)用 `asyncio`;CPU 密集用 `multiprocessing`;三者可以组合,但每层都要明确边界。

工程建议 2:所有并发代码都要回答三个问题:错误在哪捕获(线程里未捕获异常会静默消失,要包 `try/except` 并记录)、资源在哪释放(锁与连接用 `with`)、如何优雅停机(`join` 与 `sentinel` 值)。回答不清,上线必出事。

本章速查表

场景	写法
捕获异常	<code>except ValueError as e:</code> ,先子类后父类
多异常一支	<code>except (TypeError, ValueError):</code>
无异常才执行	<code>else:</code> 块
必然清理	<code>finally:</code> 或 <code>with</code>
抛出业务错误	<code>raise PaymentError("...")</code>
保留异常链	<code>raise X from e</code>
启动线程	<code>t = threading.Thread(target=f, args=()); t.start()</code>
等待线程	<code>t.join()</code>
临界区	<code>with lock:</code> 范围最小化
可重入锁	<code>threading.RLock()</code>
线程传数据	<code>queue.Queue</code> + <code>put/get/task_done</code>
CPU 并行	<code>mp.Pool(n).map(f, items)</code> ,须 <code>__main__</code> 保护
协程并发	<code>await asyncio.gather(*tasks)</code>
事件循环入口	<code>asyncio.run(main())</code>
<code>async</code> 里跑同步库	<code>await asyncio.to_thread(sync_fn, ...)</code>
线程内未捕获异常	不会打印!包 <code>try/except</code> 自己记录

最小必会清单

1. 能写出 `try/except/else/finally` 完整结构,并解释 `else` 与 `finally` 的语义差异
2. 能设计自定义异常层次(业务基类 + 具体子类),并用 `raise from` 保留根因
3. 能解释 GIL 是什么、为什么存在,以及它对 I/O 密集与 CPU 密集任务的不同影响
4. 能用 `with lock` 保护临界区,能识别死锁场景并说出防御规则
5. 能用 `queue.Queue` 实现生产者-消费者,解释 `put/get/task_done/join` 的作用
6. 能说出线程、进程、协程三种并发的适用场景,并写一个最小的 `async/await` 例子

自测题

Q

Q1. else 与 finally 的区别是什么?try 里 return 了,finally 还执行吗?

答案:else 只在 try 无异常时执行,是"成功路径"的延续;finally 无条件执行,用于清理。try 里 return 时,finally 先执行再返回——包括 return 表达式求值完成后、真正退出函数前。若 finally 里也有 return,它会覆盖 try 的返回值(生产上避免在 finally 写 return)。

Q

Q2. except: 裸捕获有什么危害?什么情况下用它合理?

答案:它连 KeyboardInterrupt、SystemExit、GeneratorExit 一起吞,程序无法被 Ctrl+C 停止;空 pass 更让错误彻底消失。合理场景:最外层"记录日志并优雅退出"的兜底,且必须记录完整异常(traceback.format_exc());除此之外一律捕获具体异常类型。

Q

Q3. 两个线程同时执行 count += 1 一万次,count 为什么不是 20000?GIL 为什么没保护它?

答案:count += 1 是"取数、加一、写回"三步字节码,GIL 只保证单条字节码不被其他线程插入,不保证这三步连续。线程 A 取到 100 后让出锁,线程 B 也取到 100 并写回 101,A 再写回 101——丢了一次更新。保护办法:with lock: 包住,或用 queue 转移所有权。

Q

Q4. 死锁的四个必要条件是什么?代码层面如何预防?

答案:互斥、持有并等待、不可抢占、循环等待。预防:① 全局一致的锁顺序(打破循环等待);② with lock 自动释放(不可抢占下的异常安全);③ 锁范围最小化,锁内不做 I/O;④ 用 queue 等结构消除共享,无共享则无锁;⑤ 复杂场景用锁超时(acquire(timeout=))并记录告警。

Q

Q5. 一个爬虫要并发抓 10000 个 URL,线程、进程、协程各有什么利弊?你会选哪个?

答案:线程:百级可行,万级线程创建与切换开销过大;进程:能真并行但进程数多时资源重、传参序列化开销大;协程:单线程内万级并发毫无压力,I/O 等待自动切换,是首选。选 asyncio + aiohttp,连接池控制并发上限(semaphore),超时与重试照章执行。

Q

Q6. 线程函数里抛了异常,主线程为什么没看到?怎么处理?

答案:线程异常只属于该线程,解释器不会把它传回主线程;线程静默死亡,主线程 join 返回,一切"正常"。处理:线程函数内 try/except 记录日志;需要结果的场景用 concurrent.futures.Future——future.result() 会把异常重新抛给调用方,异常与结果同路返回。

章末实验题:生产者-消费者任务流水线

实验 9.1 日志处理流水线(覆盖:\$9.3-\$9.6)

需求:① 生产者线程生成 200 条模拟日志(随机级别与耗时),放入有界队列;② 三个消费者线程从队列取日志,按级别统计到共享计数器(必须加锁),并模拟处理耗时;③ 队列满时 put 阻塞、空时 get 阻塞,用 task_done/join 保证"全部处理完才结束";④ 增加一个"异常注入"开关:某条日志处理时抛业务异常,消费者捕获并记录到错误队列,最后汇总打印;⑤ main 汇总输出各级别统计与错误数,验证并发正确性(加锁与不加锁的结果对比)。

覆盖知识点:GIL 与线程(\$9.3、\$9.4)/锁与临界区(\$9.5)/queue 生产消费(\$9.6)/异常处理(\$9.1、\$9.2)。

实验环境:Python 3.10+,单文件 `pipeline.py`。

```

# pipeline.py — 日志处理流水线(覆盖第9章全部知识点)
import queue
import random
import threading
import time

LEVELS = ("INFO", "WARN", "ERROR")
STATS_LOCKED = {lv: 0 for lv in LEVELS} # 加锁版本
STATS_RAW = {lv: 0 for lv in LEVELS} # 不加锁对照
stats_lock = threading.Lock()
error_q = queue.Queue() # 错误汇报通道
FAIL_EVERY = 17 # 每 17 条注入一次异常

def make_log(i: int) -> dict:
    """模拟一条日志:级别随机,耗时随机"""
    return {"id": i,
            "level": random.choice(LEVELS),
            "cost": random.uniform(0.005, 0.02)}

def producer(q: queue.Queue, total: int) -> None:
    """生产者:生成日志放入有界队列,放不下就阻塞"""
    for i in range(total):
        q.put(make_log(i)) # 队列满时自动阻塞
    q.put(None) # 哨兵:通知消费者退出

def consumer(q: queue.Queue, name: str, use_lock: bool) -> None:
    """消费者:取日志、统计、模拟处理;遇哨兵退出"""
    while True:
        item = q.get()
        if item is None: # 哨兵值,退出循环
            q.task_done()
            break
        try:
            if item["id"] % FAIL_EVERY == 0:
                raise ValueError(f"日志 {item['id']} 内容损坏")
            time.sleep(item["cost"]) # 模拟处理耗时
        except ValueError as e:
            error_q.put(str(e))
            q.task_done()
            continue
        if use_lock:
            with stats_lock: # 临界区:读-改-写原子化
                STATS_LOCKED[item["level"]] += 1
        else:
            STATS_RAW[item["level"]] += 1 # 竞态对照组
        q.task_done()

```

```

def run(use_lock: bool) -> tuple:
    """跑一轮:总耗时、统计结果、错误数"""
    q = queue.Queue(maxsize=30)          # 有界队列
    error_q.queue.clear()
    threads = []
    threads.append(threading.Thread(
        target=producer, args=(q, 200), daemon=True))
    for i in range(3):                  # 三个消费者
        threads.append(threading.Thread(
            target=consumer, args=(q, f"c{i}", use_lock),
            daemon=True))
    for t in threads:
        t.start()
    q.join()                            # 等所有任务完成
    for t in threads:
        t.join()
    stats = dict(STATS_LOCKED if use_lock else STATS_RAW)
    return stats, error_q.qsize()

def main() -> None:
    random.seed(42)
    locked_stats, locked_err = run(True)
    print("加锁统计:", locked_stats, "错误:", locked_err)
    raw_stats, _ = run(False)
    print("不加锁统计:", raw_stats)
    print("合计(应为 200 - 错误数):",
          sum(raw_stats.values()))

if __name__ == "__main__":
    main()

```

设计说明:① 有界队列 maxsize=30 让 put 阻塞可被观察,哨兵 None 让消费者干净退出;② 加锁与不加锁两轮对照,能直接看到统计丢失(两轮合计明显小于 200 减错误数);③ 异常注入走独立的错误队列,不污染主数据通道——这正是“错误与数据分离”的生产姿势;④ 每轮用独立 queue 与线程组,避免状态串扰;⑤ main 里 seed 固定,结果可复现,适合自动化验证。

下一章预告:第10章 类型标注与元编程——TypeVar 与 Generic、Protocol 结构化协议、元类与 `__init_subclass__`、描述符协议。让动态语言在大型项目里获得静态检查的护航。

第10章 类型标注与元编程

TypeVar 与 Generic · Protocol · 描述符 · 元类与 `__init_subclass__`

本章目标:讲透 Python 类型标注的进阶能力与"类的类"元编程:TypeVar/Generic 泛型、Protocol 结构化协议、描述符协议、元类与 `__init_subclass__`。学完你能写出既保持 Python 灵活、又能被 mypy 静态检查护航的生产代码,并理解装饰器、property 等语法糖背后的机制。

直觉起点:C 的类型在编译期决定一切;Python 的类型标注是"写给检查器与人看的承诺",运行期基本不检查。泛型之于 Python,类似 C 的宏之于类型——但更安全。元编程则是"创建类的代码"——class 语句本身就是一个可拦截的构造过程,Python 把 C 的编译期魔法搬到了运行期。

前置要求:第6章面向对象(类与继承)、第4章函数(标注语法)、第9章异常。

本章路线:第一阶段(1.5 小时)读 §10.1–§10.3,掌握泛型与 Protocol;第二阶段(1.5 小时)读 §10.4–§10.6,理解描述符与类创建钩子;第三阶段(2 小时)读 §10.7–§10.8,完成章末实验。

本章知识导图

- └─ 10.1 标注语法速览 — X | Y、内置泛型、3.8 兼容
- └─ 10.2 TypeVar 与 Generic — 泛型函数与容器
- └─ 10.3 Protocol — 鸭子类型的静态化
- └─ 10.4 描述符 — `__get__`/`__set__` 与 property 真相
- └─ 10.5 元类 — class 语句背后的构造过程
- └─ 10.6 `__init_subclass__` — 更亲民的子类钩子
- └─ 10.7 dataclass — 数据类的元编程红利
- └─ 10.8 生产实践 — 元编程的边界与检查工具

§10.1 标注语法速览:X | Y、内置泛型

标注只对检查器有意义,运行期几乎不校验——这是理解本章的前提。语法上记住四个点:内置容器可直接标注 (`list[int]`、`dict[str, int]`);可空用 `X | None` (3.10+)或 `Optional[X]`;联合用 `A | B` (3.10+)或 `Union[A, B]`;泛型参数给容器:

```
from typing import Optional, Union

# 3.10+ 新写法
def lookup(name: str) -> dict | None:
    ...

# 3.8 兼容写法:同一含义
def lookup_old(name: str) -> Optional[dict]:
    ...

# 3.8 里内置类型不能直接下标,要用 typing 版本
from typing import List, Dict, Tuple
def stats(scores: List[int]) -> Tuple[int, float]:
    ...
```


3.8 兼容注意: `list[int]` 这类内置泛型语法从 3.9 才有;3.8 必须写 `List[int]` 。若项目仍在支持 3.8,统一用 `typing` 版别称;项目已是 3.10+,全用新语法,代码更短。除了语法,还要记住 `->` 后的返回值标注里写 `"Point"` 字符串(前向引用)可以避免定义顺序问题,或直接用 `from __future__ import annotations` 延迟求值。

标注写错的代价不是崩溃而是误导:错误类型被检查器传给下游,错误像滚雪球。所以标注的纪律是"从入口到出口完整"——函数、类字段、返回值都要有,半标注比不标注更危险,因为它给了假的信心。

\$10.2 TypeVar 与 Generic:泛型函数与容器

TypeVar 声明一个"类型变量",让一个函数或类对多种类型保持类型安全——同一参数位置的类型关系被检查器追踪,不会像 C 的 `void*` 那样失去类型信息:

```
from typing import TypeVar, Generic

T = TypeVar("T")                # 任意类型
U = TypeVar("U", int, float)    # 只允许数值类型

def first(items: list[T]) -> T:
    """泛型函数:返回类型与元素类型一致"""
    return items[0]

def add_all(a: U, b: U) -> U:
    """类型变量同一位置要求同类型"""
    return a + b

class Stack(Generic[T]):        # 泛型类:Stack[str]、Stack[int]
    """后进先出栈,元素类型在实例化时确定"""

    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()

s = Stack[int]()
s.push(1)
n: int = s.pop()                # 检查器知道 pop 返回 int
```

C(void*/宏)	Python(泛型标注)
void* 传数据,类型信息丢失	T 被检查器全程追踪,返回类型精确
宏展开错误在编译期报,信息晦涩	类型错误在 mypy 运行时报,信息直白
强转(cast)全靠人肉保证	检查器验证类型关系,不一致即报错
运行期零开销	标注运行期零开销(不检查,但也不约束)

TypeVar 还有两个变体: `TypeVar("T", bound=Base)` 限制 T 必须是某基类或其子类; `TypeVar("T", covariant=True)` 用于"只读容器"的协变。日常 90% 用最朴素的 `TypeVar("T")` 就够。

泛型标注要避免过度设计:一个函数用三个 TypeVar 多半是设计出了问题。经验法则:类型变量出现在两个以上位置才有价值;如果 T 只出现一次,直接用具体类型或 Any 更诚实。

§10.3 Protocol:鸭子类型的静态化

第6章说"长得像就用",运行时这么写没问题,但检查器看不到"长得像"的契约。**Protocol** 把鸭子类型写成可检查的结构化协议——只要类型有这些方法,就被认定为协议实现者,无需继承:

```
from typing import Protocol

class Payable(Protocol):
    """结构化协议:有 pay() 方法即满足,无需继承"""

    def pay(self) -> float: ...

class Manager:
    # 完全不继承 Payable
    def pay(self) -> float:
        return 20000.0

def total_pay(staff: list[Payable]) -> float:
    return sum(p.pay() for p in staff)

print(total_pay([Manager()])) # mypy 通过,运行正常
```

误区:拿 Protocol 当运行期检查。 Protocol 是给 mypy 看的契约, `isinstance(obj, Payable)` 默认直接报错——除非加 `@runtime_checkable` 装饰器,且只能做"方法存在性"检查。生产姿势:类型检查靠 Protocol,运行期防御靠 EAFP(直接调用 + 捕获 `AttributeError`),两层各司其职。

Protocol 的威力在大型项目:接口定义在库层,调用方与实现方互不认识也能类型联动;改协议时检查器立刻列出所有违约者。这比 C 的虚函数接口更松、比鸭子类型更严,是"Pythonic 的接口"。注意 Protocol 成员的 `...` 占位写法,以及协议方法不要写实现。

§10.4 描述符: `__get__` / `__set__` 与 property 的真相

描述符是实现了 `__get__` / `__set__` 的对象,被赋值给类属性时,对实例属性的访问会转交给它。property、classmethod、staticmethod 全是描述符——理解了描述符,就理解了这些语法糖的底层:

```

class Positive:
    """字段描述符:拒绝负值,自动校验"""

    def __init__(self, name: str) -> None:
        self.name = name          # 存储用的属性名

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self           # 类上访问时返回描述符本身
        return obj.__dict__[self.name]

    def __set__(self, obj, value) -> None:
        if value < 0:
            raise ValueError(f"{self.name} 不能为负")
        obj.__dict__[self.name] = value

class Account:
    balance = Positive("balance")  # 描述符实例作为类属性

    def __init__(self, balance: float) -> None:
        self.balance = balance     # 触发 __set__ 校验

a = Account(1000)
a.balance = 500                  # 正常
a.balance = -5                   # ValueError: balance 不能为负

```

误区:描述符的存储属性与描述符名冲突。 `__set__` 里如果写成 `obj.balance = value`,会再次触发 `__set__` 无限递归。正确姿势:把值存进实例的 `__dict__`,key 用描述符自己的名字(如 `self.name="balance"`),这是描述符的标准存储模式。另一个坑:描述符只在"类属性"上生效,塞进实例属性就完全失灵。

看完这个例子就明白 `@property` 是什么:Python 内置的 `property` 描述符,把 `fget / fset` 两个函数包装成属性访问。你自己写描述符的场景通常是"重复的字段校验"(如上),与 C 里"每个字段手写 `setter`"对照,描述符一次写好、处处复用。

C(手写访问器)	Python(描述符)
每个字段一对 <code>getter/setter</code> ,复制粘贴	描述符写一次,挂在任意字段
校验逻辑散落在各字段函数里	校验集中在 <code>__set__</code> ,一处修改全局生效
新增字段忘了加访问器,数据裸奔	类属性上挂描述符,漏挂即属性不存在
访问器改名,调用方全部要改	实例属性语法不变,升级无感知

§10.5 元类:class 语句背后的构造过程

类也是对象,由**元类**创建——默认元类是 `type`。 `class Foo:` 在运行期等价于调用 `type("Foo", (object,), namespace)`。自定义元类继承 `type`,重写 `__new__` 拦截"类创建"这一刻,实现 AOP 式增强:

```

class TimedMeta(type):
    """元类:给所有方法自动包一层计时日志"""

    def __new__(mcls, name, bases, namespace):
        for key, value in list(namespace.items()):
            if callable(value) and not key.startswith("__"):
                namespace[key] = mcls._wrap(value)
        return super().__new__(mcls, name, bases, namespace)

    @staticmethod
    def _wrap(func):
        import time
        def wrapper(*args, **kwargs):
            start = time.perf_counter()
            result = func(*args, **kwargs)
            print(f"{func.__name__} 耗时 "
                  f"{{(time.perf_counter() - start) * 1000:.2f}} ms")
            return result
        return wrapper

class Worker(metaclass=TimedMeta):
    def job(self) -> None:
        for _ in range(1000000):
            pass

Worker().job()                                # 自动打印耗时,类体零改动

```

类创建三步:收集类体命名空间 → 调元类构造新类 → 绑定名字。自定义元类常用场景:框架层"注册所有子类"、"自动生成方法"、"统一接口校验"。代价是隐式魔法:新人看到 Worker 里没写计时,却输出了耗时——所以元类只该出现在库与框架层,业务代码慎用。

想判断一个自动化是否该用元类,问三个问题:收益是否覆盖所有使用者、失败模式是否可解释(报错信息是否指向问题根源)、新人能否在五分钟内看懂。三个都答是,才值得写。

§10.6 __init_subclass__:更亲民的子类钩子

Python 3.6+ 提供了元类场景 90% 的替代品: `__init_subclass__` 在"有人继承我"时被调用,直接写在普通类里,不需要元类。经典应用是插件注册表:

```

class Plugin:
    """插件基类:继承即注册,自动进入注册表"""
    registry: dict[str, type] = {}

    def __init_subclass__(cls, name: str = None, **kwargs) -> None:
        super().__init_subclass__(**kwargs)
        key = name or cls.__name__.lower()
        Plugin.registry[key] = cls    # 注册子类

class HttpChecker(Plugin, name="http"):    # 自定义注册名
    def run(self) -> str:
        return "检查 HTTP"

class DnsChecker(Plugin):
    def run(self) -> str:
        return "检查 DNS"

print(sorted(Plugin.registry))            # ['dns', 'http']
runner = Plugin.registry["http"]()
print(runner.run())                       # 检查 HTTP

```

插件系统、ORM 模型收集、策略注册——"继承即登记"让新增功能只写一个类,调度代码零改动。与 C 的注册表(手工登记函数指针数组)相比,Python 把登记动作自动化了。注意细节:关键字参数必须显式命名并处理(**kwargs 传给 super),否则子类自定义参数会报错。

C(手工注册)	Python(继承即注册)
新增插件要改注册表初始化函数	新增插件只写类,注册自动完成
漏登记直到运行期才暴露	注册发生在导入期,启动即报错
注册表与实现分离,易失同步	注册表与类定义同处一个模块,天然一致
函数指针数组的类型检查靠约定	检查器校验子类确实满足接口

误区:元类与 __init_subclass__ 混用踩坑。两者叠加时,执行顺序是"先元类构造类,再调 __init_subclass__";如果元类的 __new__ 里也做注册,会与 __init_subclass__ 重复执行或顺序颠倒。生产规则:同一份"子类收集"逻辑只选一个机制——优先 __init_subclass__(简单、可读),只有需要拦截"类创建过程本身"(改名、改基类、注入方法)才上元类。

§10.7 dataclass:数据类的元编程红利

dataclass 是标准库对"数据载体类"的元编程自动化:声明字段即自动生成 __init__、__repr__、__eq__,还能参与比较与不可变:

```

from dataclasses import dataclass, field

@dataclass(frozen=True)           # frozen:不可变,可哈希
class Product:
    sku: str
    name: str
    price: float = 0.0
    tags: list = field(default_factory=list) # 可变默认值必须 factory

    @property
    def total(self) -> float:
        return self.price

p = Product("P-001", "机械键盘", 399.0)
print(p)                          # Product(sku='P-001', ...)
print(p == Product("P-001", "机械键盘", 399.0)) # True
# p.price = 1                    # frozen=True 时报错

```

与 C 写"字段 + 初始化 + 打印 + 比较"的样板对照,dataclass 一次省掉几十行。三个要点:字段默认值直接写在标注后;可变默认值(list/dict)必须用 field(default_factory=) 否则所有实例共享同一对象(与第4章可变默认参数同源);frozen=True 让实例不可变从而可哈希,适合作为 dict 的 key。

dataclass 与字典的边界:临时打包几个值用 dict 够快,但"字段固定、需要方法、需要校验"的数据结构一律 dataclass——它把散落的键名拼接变成了拼写检查器能拦的字段访问,这是生产代码可维护性的关键差异。

§10.8 生产实践:元编程的边界

元编程是把双刃剑:自动化的另一面是隐式。三条生产纪律:① **只写"一次"与"多处收益"的自动化**——注册表、校验字段、统一日志值得;为省两行代码写元类不值。② 元编程代码必须有单元测试与文档,它的失败模式(属性不存在、注册遗漏)比普通代码难排查。③ 团队约定:业务模块禁用元类,框架层允许,__init_subclass__ 全项目可用。

工程建议 1:把 mypy 纳入 CI,标注覆盖率不用 100%,但新代码必须过检。Protocol + TypeVar + Generic 组合,让"运行期动态、检查期静态"成为项目常态——这是 Python 大型项目的唯一正解。

工程建议 2:数据类统一用 @dataclass,禁止手写 __init__/_repr__/_eq__ 样板;校验逻辑放 property 或 field 描述符,让非法数据在构造时失败——这比"运行到深处才爆"便宜一个数量级。

本章速查表

场景	写法
可空标注	<code>X None (3.10+) / Optional[X]</code>
联合类型	<code>A B (3.10+) / Union[A, B]</code>
内置泛型	<code>list[int] (3.9+); 3.8 用 List[int]</code>
泛型函数	<code>T = TypeVar("T"); def f(x: list[T]) -> T</code>
泛型类	<code>class Stack(Generic[T]):</code>
限制类型变量	<code>TypeVar("T", bound=Base)</code>
结构化接口	<code>class Payable(Protocol): def pay(self) -> float: ...</code>
运行期协议检查	<code>@runtime_checkable</code> + <code>isinstance</code> (仅方法存在性)
字段校验复用	自定义描述符(<code>__get__</code> / <code>__set__</code>)挂在类属性
类创建拦截	元类继承 <code>type</code> , 重写 <code>__new__</code>
继承即注册	<code>def __init_subclass__(cls, **kwargs) + registry</code>
数据类	<code>@dataclass(frozen=True)</code> + <code>field(default_factory=)</code>
前向引用	<code>from __future__ import annotations</code>
静态检查	<code>mypy src/</code> , 纳入 CI
元类纪律	仅框架层使用, 业务模块禁用
样板禁止	手写 <code>__init__</code> / <code>__repr__</code> 前先想 <code>dataclass</code>

最小必会清单

1. 能写出 3.10+ 与 3.8 两套兼容的标注(联合、可空、内置泛型)
2. 能用 `TypeVar` + `Generic` 写出泛型函数与泛型容器, 并解释 `T` 的追踪
3. 能用 `Protocol` 定义结构化接口, 并说明它与继承接口、鸭子类型的区别
4. 能解释描述符的 `__get__`/`__set__` 机制与存储模式, 并说明 `property` 的原理
5. 能用 `__init_subclass__` 实现"继承即注册", 并说清它与元类的分工
6. 能用 `@dataclass` 声明数据类, 并解释 `default_factory` 与 `frozen` 的用途

自测题

Q

Q1. 类型标注在运行期到底做什么?为什么说"标注只对检查器有意义"?

答案:运行期基本不做事:函数标注存进 `__annotations__` 属性供内省,但传参类型错误不会报错。校验由 `mypy` 等静态检查器在开发期完成。正因如此,标注写错不会崩,但检查器会立刻指出——所以标注的收益在"开发期拦截错误"与"文档化契约",不在运行期防御。

Q

Q2. `def first(items: list[T]) -> T` 中, `T` 与 `C` 的 `void*` 有什么区别?

答案:`void*` 在编译期丢失元素类型,取用要强转,转错运行期崩溃;`T` 由检查器按"同一位置类型一致"规则推导并全程追踪,`first(["a"])` 返回 `str`,`first([1])` 返回 `int`,传错类型在检查期就报错。运行期两者都不存在——Python 的 `T` 不生成任何代码。

Q

Q3. Protocol 与抽象基类(ABC)有什么区别?什么时候用哪个?

答案:ABC 是"继承式"契约:必须显式继承并实现抽象方法,是运行时强制;Protocol 是"结构化"契约:类型长得像即满足,无需继承,是检查期强制。选型:定义框架接口、需要运行期 `isinstance` 强制用 ABC;想让第三方类型零侵入地参与你的多态(鸭子类型静态化)用 Protocol。

Q

Q4. 描述符的 `__set__` 里写 `obj.xxx = value` 为什么递归?正确存值方式是什么?

答案:`obj.xxx` 触发属性查找,找到类上的描述符,再调 `__set__`,而 `__set__` 里又执行 `obj.xxx` 赋值.....无限递归直到 `RecursionError`。正确方式:把值存进 `obj.__dict__["xxx"]`,`__get__` 也读同一个 `key`;命名约定是描述符构造时接收存储名,与描述符自身的属性名一致。

Q

Q5. `__init_subclass__` 里为什么要 `super().__init_subclass__(**kwargs)`?不传会怎样?

答案:子类声明中的关键字参数(如 `name="http"`)先被 `class` 语句收集,传给 `__init_subclass__`; `super()` 调用把参数沿继承链继续传递,让父类的钩子也执行。若吞掉 `kwargs` 或漏调 `super`,深层继承时祖先钩子不执行,注册表缺类——这是"继承即注册"链断裂的头号原因。

Q

Q6. `@dataclass(frozen=True)` 的实例为什么能当 `dict` 的 `key`? `list` 字段为什么必须 `default_factory`?

答案:`frozen=True` 生成 `__setattr__` 拦截赋值,实例不可变,`dataclass` 自动生成基于字段的 `__hash__`,因此可哈希可做 `key`。`list` 是可变对象,若直接 `default=[]`,所有实例共享同一个列表(第4章可变默认参数坑的类级版本);`default_factory=list` 每次实例化调用一次工厂,生成独立列表。

章末实验题:类型安全的插件系统

实验 10•1 泛型检查器与插件注册表(覆盖:§10.2–§10.6)

需求:① 用 Protocol 定义 Checker 接口(方法 `check() -> Result`);② 用 `dataclass` 定义 `Result(code: int, message: str)`,支持比较;③ 用 `__init_subclass__` 实现"继承即注册"的插件注册表,支持自定义注册名;④ 实现至少两个插件(如 `HttpChecker`、`SqlChecker`),检查逻辑用正则与 `sleep` 模拟;⑤ 用 `TypeVar` + `Generic` 写一个泛型函数 `run_all(checkers: list[C]) -> list[Result]`,要求返回类型与传入类型对应;⑥ `main` 从注册表按名字取插件并全部运行,输出汇总;⑦ 跑一遍 `mypy` 确认零错误。

覆盖知识点:Protocol 结构化协议 (§10.3)/`TypeVar` 与 `Generic`(§10.2)/`__init_subclass__` 注册表 (§10.6)/`dataclass`(§10.7)。

实验环境:Python 3.10+,单文件 `plugins.py` 。

```

# plugins.py — 类型安全的插件注册表(覆盖第10章全部知识点)
import re
import time
from dataclasses import dataclass
from typing import Protocol, TypeVar

@dataclass(frozen=True)
class Result:
    """检查结果:不可变,可按 code 比较排序"""
    code: int # 0=通过,1=警告,2=失败
    message: str

    @property
    def ok(self) -> bool:
        return self.code == 0

class Checker(Protocol):
    """结构化协议:有 check() 即插件,无需继承"""

    def check(self) -> Result: ...

class Plugin:
    """插件基类:继承即自动注册,支持自定义注册名"""
    registry: dict[str, type] = {}

    def __init_subclass__(cls, name: str = None, **kwargs) -> None:
        super().__init_subclass__(**kwargs)
        key = name or cls.__name__.lower()
        if key in Plugin.registry:
            raise ValueError(f"插件名重复: {key}")
        Plugin.registry[key] = cls

class HttpChecker(Plugin, name="http"):
    """检查 HTTP 服务:模拟探测"""

    def __init__(self, url: str) -> None:
        self.url = url

    def check(self) -> Result:
        time.sleep(0.05) # 模拟网络探测
        if not re.match(r"^https?:/", self.url):
            return Result(2, f"URL 格式不合法: {self.url}")
        return Result(0, f"{self.url} 可达")

class SqlChecker(Plugin):

    def __init__(self, sql: str) -> None:
        self.sql = sql

    def check(self) -> Result:

```

```

    if re.search(r"drop\s+table|;\s*--", self.sql, re.I):
        return Result(2, "检测到危险 SQL 模式")
    if re.search(r"\bselect\b", self.sql, re.I):
        return Result(1, "SELECT 语句请走只读账号")
    return Result(0, "SQL 检查通过")

```

```

C = TypeVar("C", bound=Checker)

def run_all(checkers: list[C]) -> list[Result]:
    """泛型函数:接受任意 Checker 列表,返回对应结果"""
    return [c.check() for c in checkers]

def main() -> None:
    print("已注册插件:", sorted(Plugin.registry))
    http_cls = Plugin.registry["http"]          # 按名取类
    sql_cls = Plugin.registry["sqlchecker"]
    jobs = [http_cls("https://example.com"),    # 工厂式构造
            http_cls("ftp://bad.example.com"),
            sql_cls("select * from users; -- x"),
            sql_cls("update accounts set bal=0")]
    results = run_all(jobs)                      # 泛型调用
    for r in results:
        print(f"[{'通过' if r.ok else '失败'}] {r.message}")
    print(f"汇总: {sum(1 for r in results if r.ok)}/"
          f"{len(results)} 通过")

if __name__ == "__main__":
    main()

```

设计说明:① 注册名冲突在注册瞬间抛 ValueError,把"插件名重复"这类配置错误提前到导入期;② Result 用 frozen dataclass,不可变且自动带 `__eq__`/`__repr__`,正好呼应 §10.7;③ `run_all` 用 `TypeVar(bound=Checker)` 保证参数与返回的类型联动,乱传非 Checker 会被 mypy 拦截;④ Plugin 基类与业务插件分层,新增插件只写一个类——扩展系统不修改调度代码,这就是开闭原则的元编程实现;⑤ 最后在项目根执行 `python -m mypy plugins.py`,看到 Success 输出即类型安全达成。

下一章预告:第11章 反射与动态特性——`dir/inspect/type` 动态造类、`__getattr__` 与属性拦截、猴子补丁的双刃剑。让程序在运行期自我审视、自我改造。

第11章 反射与动态特性

内省 · 属性拦截 · 动态造类 · 猴子补丁

本章目标:讲透 Python 的反射能力:dir/type/inspect 程序自省、__getattr__/__setattr__ 属性拦截、type 动态造类、importlib 动态加载,以及猴子补丁的双刃剑。学完你能写出调试器、序列化器、插件加载器这类"元工具",并清楚反射的边界与风险。

直觉起点:C 程序在编译后只是一堆指令与符号表,程序无法问自己"我有哪些函数";Python 的对象带着完整元信息运行,任何对象都能被追问"你是什么、你有什么、你能做什么"。反射就是这种自我审视的能力——它在 C 里几乎不存在,是 Python 动态性的核心。

前置要求:第6章面向对象(属性与类)、第10章类型与元编程(类创建机制)。

本章路线:第一阶段(1 小时)读 §11.1–§11.2,掌握内省工具;第二阶段(2 小时)读 §11.3–§11.5,理解属性拦截与动态构造;第三阶段(2 小时)读 §11.6–§11.8,认识猴子补丁并完成章末实验。

本章知识导图

- └─ 11.1 内省工具 ── dir / type / vars / getattr
- └─ 11.2 inspect ── 签名、源码、类层次
- └─ 11.3 属性拦截 ── __getattr__ / __setattr__
- └─ 11.4 动态造类 ── type() 与运行时方法
- └─ 11.5 动态导入 ── importlib 与插件加载
- └─ 11.6 猴子补丁 ── 威力与代价
- └─ 11.7 反射序列化 ── 通用对象转字典
- └─ 11.8 生产实践 ── 反射的边界

§11.1 内省工具:dir / type / vars / getattr

内省四件套覆盖"看有什么、是什么、取什么"的全部需求:dir(obj) 列出所有属性名;type(obj) 得类型;vars(obj) 得实例的 __dict__;动态取属性用 getattr(obj, name),动态设属性用 setattr(obj, name, value):

```

class Config:
    def __init__(self) -> None:
        self.env = "prod"
        self.debug = False

    def reload(self) -> None:
        print("重新加载配置")

cfg = Config()
print(dir(cfg))          # ['__class__', ..., 'debug', 'env', 'reload']
print(vars(cfg))         # {'env': 'prod', 'debug': False}

# 按字符串动态取属性:配置驱动
key = "debug"
print(getattr(cfg, key)) # False
setattr(cfg, key, True)  # 等价于 cfg.debug = True
print(cfg.debug)         # True

# 不存在时给默认值,避免 AttributeError
print(getattr(cfg, "missing", "无此属性")) # 无此属性

```

C(静态内省)	Python(运行期内省)
sizeof/offsetof 只给内存布局	dir/vars 给出完整成员清单与当前值
符号表只在调试器里可见	程序自己就能枚举方法并调用
类型信息编译期固定	type() 运行期可得,且能判断 isinstance
字段增减要改结构体并重编译	setattr 随时加属性,无需重编译

getattr 的第三个参数是反射的标配动作——"可能不存在"的访问永远带默认值或 try/except,这是反射代码的第一纪律。dir 的输出注意区分:实例属性、类属性、双下划线魔法属性混在一起,过滤 `startswith("__")` 是常规操作。

内省的第一步永远是"看",第二步才是"取":先 dir 确认存在,再 getattr 读取——但生产上不要先查再取(两步之间有竞态),直接用带默认值的 getattr 一步到位。遍历对象的通用工具(调试器、序列化器)要把"实例属性、类属性、继承属性"三层分开处理,vars 只看实例层,dir 看全部。

§11.2 inspect:签名、源码与类层次

inspect 是标准库的深度内省模块:拿函数签名、看实现源码、遍历类层次,还能判断"这个对象是不是类/方法/协程"。调试器、文档生成器、测试框架都建立在它之上:

```
import inspect

def connect(host: str, port: int = 8080, *, timeout: float = 5) -> None:
    """建立连接"""
    ...

sig = inspect.signature(connect)
print(sig)                    # (host: 'str', port: 'int' = 8080, *, timeout: 'float' = 5)
for name, param in sig.parameters.items():
    print(name, param.kind, param.default)
# host POSITIONAL_OR_KEYWORD
# port POSITIONAL_OR_KEYWORD 8080

print(inspect.getsource(connect))    # 打印函数源码
print(inspect.isclass(Config), inspect.isfunction(connect))
```

生产中最常用的三个能力:signature 做参数校验与自动文档;getsource 排查"这个函数到底是谁的"(尤其装饰器包裹后);ismethod/isfunction 区分绑定方法、静态方法与普通函数——框架层统一处理"可调用对象"时必须用它们兜底。

inspect 还有一个高频用途:协程与生成器的判断(iscoroutine/isgenerator)。异步框架调度任务前必须分清"普通函数、生成器、协程",传错类型会在 await 处爆出难解的报错;用 inspect 预检,错误提前到调度点,信息也清晰得多。

§11.3 属性拦截: __getattr__ 与 __setattr__

Python 允许类拦截属性访问: __getattr__ 只在"正常查找失败"时调用(懒加载的天然实现); __setattr__ 在每次属性赋值时调用(校验与只读的强制实现); __getattribute__ 拦截一切读取(危险,慎用):

```
class LazyConfig:
    """懒加载配置:首次访问才真正读取文件"""

    def __init__(self, path: str) -> None:
        self._path = path
        self._loaded = False
        self._data = {}

    def __getattr__(self, name: str):
        """只在正常查找失败时触发:惰性加载"""
        if name == "data" and not self._loaded:
            self._data = {"env": "prod", "workers": 4} # 模拟读文件
            self._loaded = True
        return self._data[name] # 返回配置值

c = LazyConfig("config.json")
print(c.workers)                # 4 — 首次访问触发加载
print(c.env)                    # prod
print(c.workers)                # 4 — 已缓存,不再加载
```

误区: __setattr__ 里写 self.x = value 无限递归。__setattr__ 拦截一切赋值,在它内部再给 self.x 赋值会再次进入 __setattr__ 直到 RecursionError。正确姿势:用 object.__setattr__(self, name, value) 或操作

`self.__dict__` (与第10章描述符的存储模式同源)。同理, `__getattr__` 里访问 `self.xxx` 也可能绕回自身,先检查 `__dict__` 或使用前缀保护。

```
class Immutable:
    """ __setattr__ 强制只读:赋值一律拒绝"""

    def __init__(self, value: int) -> None:
        object.__setattr__(self, "value", value)

    def __setattr__(self, name: str, value) -> None:
        raise AttributeError(f"{name} 是只读属性")

obj = Immutable(42)
print(obj.value)           # 42
obj.value = 1              # AttributeError: value 是只读属性
```

三种拦截的选型:只做"缺属性时的兜底"用 `__getattr__` (幂等、安全);要强制校验或只读用 `__setattr__`; `__getattribute__` 拦截所有读取,性能与复杂度代价都高,99% 的场景不需要它。代理对象、mock、ORM 懒加载字段是这三个钩子的经典生产场景。

属性拦截的调试技巧:在 `__getattr__`/`__setattr__` 里加一行 `print(name, value)` 或日志,立刻看到"谁在读写什么"——排查魔法属性意外触发、框架注入字段这类玄学问题时,这个手段比断点还快。生产上把日志级别设为 `debug`,平时零成本,出问题有抓手。

§11.4 动态造类: `type()` 与运行时方法

`class` 语句在运行期等价于 `type(name, bases, namespace)` 调用(第10章元类已见);反过来,我们可以在运行期凭空构造一个类,再给实例动态绑定方法:

```

# 方式一:type 三参数动态造类
Point = type("Point", (object,), {
    "__init__": lambda self, x, y: (setattr(self, "x", x),
                                     setattr(self, "y", y)),
    "norm": lambda self: (self.x ** 2 + self.y ** 2) ** 0.5,
})
p = Point(3, 4)
print(p.norm())           # 5.0

# 方式二:给实例动态绑方法(需要 MethodType 绑定 self)
import types

class Calc:
    pass

def square(self, n: int) -> int:
    return n * n

c = Calc()
c.square = types.MethodType(square, c)  # 绑定后 self 自动传入
print(c.square(6))                     # 36

```

动态造类的生产场景:数据驱动生成 DTO、按配置文件构造策略对象、测试里快速造替身。注意纪律:动态生成的类没有静态检查护航,mypy 对它们基本失明;能写静态类就写静态类,动态造类只用于"数量不可预知"的场景。

动态造类与静态类的关系,类似 C 里宏与函数的关系:宏(动态)灵活但不可调试,函数(静态)可读可查。一个实用的中间态是工厂函数:逻辑写在静态类里,工厂按配置选择并实例化——既有静态类的可读性,又有动态分发的灵活性,多数"动态需求"用工厂就够。

§11.5 动态导入:importlib 与插件加载

插件系统的另一条腿是"按名字加载模块"。importlib.import_module 把字符串变成可用的模块对象,配合 getattr 取其中的类——第10章的注册表与这里连成完整闭环:


```

import importlib
from pathlib import Path

def load_plugin(module_name: str, class_name: str):
    """从模块路径动态加载插件类"""
    module = importlib.import_module(module_name)
    cls = getattr(module, class_name)
    return cls()

def load_all_from(dir_path: Path) -> list:
    """扫描目录下所有 check_*.py,加载其中 main()"""
    plugins = []
    for f in dir_path.glob("check_*.py"):
        mod = importlib.import_module(f"plugins.{f.stem}")
        plugins.append(mod.main)
    return plugins

```

误区:动态加载失控。 import_module 的字符串来自配置文件或用户输入时,等于给了任意代码执行入口——恶意插件路径能加载任意模块。生产纪律:① 插件目录固定且受控,白名单校验模块名(正则 `^[a-z_]+$`);② 加载失败 (ImportError/AttributeError)要捕获并给出"哪个插件、缺什么"的清晰报错;③ 反射加载的代码视同第三方代码,一样要过评审与测试。

动态导入的另一面是"导入期副作用":模块顶层代码(注册、连库、打印)在 import 时就执行。插件系统要约定"插件模块只定义,不执行",main/run 由框架显式调用——把"加载"与"执行"分离,是插件架构的稳定性基石。

§11.6 猴子补丁:威力与代价

猴子补丁是运行期替换对象的属性/方法——给第三方库打补丁、测试替身、A/B 灰度都靠它。经典用例:修复第三方库 bug、给库方法加缓存、测试中替换网络层:

```

import time
import requests

# 场景:给 requests.get 统一加日志与超时兜底
_original_get = requests.get          # 先存原函数!

def patched_get(url, **kwargs):
    kwargs.setdefault("timeout", 10) # 没传超时的请求统一兜底
    print(f"[patched] GET {url}")
    return _original_get(url, **kwargs)

requests.get = patched_get             # 打补丁:全局生效

resp = requests.get("https://example.com")

```

C(函数覆盖)	Python(猴子补丁)
弱符号覆盖在链接期,静态、全局	运行期任意对象任意属性,可恢复可撤销
dlsym 拿符号表地址,晦涩	直接赋值 <code>mod.func = new</code> ,一行完成
覆盖后原函数丢了(需手动保存地址)	先存原引用即可随时还原(如上面的 <code>_original_get</code>)
生效范围:整个进程	生效范围:整个进程——同样全局,且难以追溯

误区:猴子补丁当常规手段。它全局生效、隐式修改、难以调试——被补丁的模块在别处可能依赖原行为,测试间的补丁还会互相污染。纪律:① 只补第三方库,不补自己的代码(自己的代码直接改);② 补丁代码集中在一个模块,写明"补什么、为什么";③ 测试替身优先用 `unittest.mock.patch`(自动还原),不用手写赋值;④ 生产补丁必须有日志与监控,升级库后第一时间复查补丁是否还必要。

补丁的"保质期"问题常被忽略:库升级后补丁可能已被官方修复,旧补丁反而引入双重重试、日志噪音等新 bug。生产规范:每个补丁文件头写明"创建日期、上游 issue 链接、移除条件",升级依赖时把补丁清单过一遍,该删就删。

§11.7 反射序列化:通用对象转字典

把任意对象转成 dict/JSON(序列化器、日志、调试工具的核心)是反射最实用的场景之一:遍历实例的 `__dict__`,递归转换基本类型,处理循环引用:

```
def to_dict(obj, seen: set | None = None) -> object:
    """把任意对象递归转成可 JSON 序列化的结构"""
    if seen is None:
        seen = set()
    if isinstance(obj, (str, int, float, bool)) or obj is None:
        return obj
    if isinstance(obj, (list, tuple, set)):
        return [to_dict(x, seen) for x in obj]
    if isinstance(obj, dict):
        return {str(k): to_dict(v, seen) for k, v in obj.items()}
    if id(obj) in seen:
        # 循环引用兜底:返回引用标记
        return {"__ref__": id(obj)}
    seen.add(id(obj))
    result = {}
    for key, value in vars(obj).items():
        # 反射:取所有实例字段
        result[key] = to_dict(value, seen)
    return result

class User:
    def __init__(self, name: str, tags: list) -> None:
        self.name = name
        self.tags = tags
        self.friend = None

u = User("李雷", ["vip", "cn"])
u.friend = User("韩梅梅", ["vip"])
import json
print(json.dumps(to_dict(u), ensure_ascii=False, indent=1))
```

这段代码一次覆盖了本章多数主题:isinstance 分派、vars 内省、seen 集合防循环引用。生产级序列化器(jsonable 库、pydantic)在其上增加了类型白名单与 __reduce__ 协议,但骨架就是反射——理解它,你就理解了 ORM 与 RPC 框架的一半。

§11.8 生产实践:反射的边界

反射是"元工具"的燃料,但有三条不可逾越的边界:① **可读性边界**——反射代码无法静态搜索"谁调用了我",IDE 跳转失效,项目里反射使用率超过 5% 就要警惕;② **性能边界**——getattr 动态查找比直接属性访问慢一个量级,热路径禁止反射;③ **安全边界**——字符串驱动的反射(getattr(obj, user_input))与 eval/exec 一样是注入面,永远白名单。

工程建议 1:反射的调试神器是 __repr__ 与 inspect:给所有业务类写 __repr__,排查"这个对象到底是什么"时一个 print 全清楚;怀疑方法来源时 inspect.getsource 直接看源码,比猜快得多。

工程建议 2:优先用"标准库提供的反射"而不是自己写:dataclasses.fields、inspect.signature、typing.get_type_hints 都是官方打磨过的实现;自己遍历 __dict__ 做参数匹配,边界情况(继承、描述符、插槽类)会让你踩不完。

本章速查表

场景	写法
列出成员	dir(obj) 过滤 startswith("__")
取实例字段	vars(obj) / obj.__dict__
动态取属性	getattr(obj, name, default)
动态设属性	setattr(obj, name, value)
函数签名	inspect.signature(func)
函数源码	inspect.getsource(func)
缺属性兜底	def __getattr__(self, name):
赋值拦截	def __setattr__(self, name, value): ,内部用 object.__setattr__
动态造类	type(name, bases, namespace)
实例绑方法	types.MethodType(func, instance)
按名加载模块	importlib.import_module("pkg.mod")
打补丁	先存原引用,再 mod.func = new ,用后还原
测试替身	unittest.mock.patch("pkg.mod.func") ,自动还原
防循环引用	seen 集合记录 id,命中返回标记
反射纪律	热路径禁用;字符串驱动必须白名单
优先现成	dataclasses.fields / typing.get_type_hints 优先于手写

最小必会清单

1. 能用 dir/vars/getattr/setattr 完成"配置驱动"的动态属性读写

2. 能用 `inspect` 拿到函数签名与源码,并判断对象的类别
3. 能写出不递归的 `__getattr__` (懒加载)与 `__setattr__` (只读/校验)
4. 能用 `type()` 动态造类、`MethodType` 给实例绑方法,并说出适用边界
5. 能用 `importlib` 实现插件加载,并说出两个安全纪律
6. 能解释猴子补丁的原理、典型场景与三条使用纪律

自测题

Q

Q1. `getattr(obj, "x")` 与 `obj.x` 有什么区别?什么场景必须用 `getattr`?

答案:功能相同,区别在属性名是否已知:`obj.x` 的属性名写死在代码里,`getattr` 接受字符串变量。必须用 `getattr` 的场景:属性名来自配置/数据(反射)、需要默认值(`getattr(obj, "x", 0)`)、遍历未知结构。已知属性名直接用点号——更可读、更快、可被静态检查。

Q

Q2. `__getattr__` 与 `__getattribute__` 的区别?为什么说后者危险?

答案:`__getattr__` 只在正常查找失败后调用(兜底、幂等);`__getattribute__` 对每次属性读取无条件调用,包括类属性、方法、魔法属性——内部任何 `self.xxx` 访问都再次触发它,极易无限递归,且性能开销大。生产 99% 场景用 `__getattr__`,只有实现代理、安全拦截等少数场景才碰 `__getattribute__`。

Q

Q3. `__setattr__` 里 `self.name = value` 为什么 `RecursionError`?怎么写才对?

答案:`__setattr__` 拦截一切赋值,`self.name = value` 又一次触发自身,无限递归。正确写法:`object.__setattr__(self, name, value)` 绕过拦截直接设置,或写 `self.__dict__[name] = value`。`__init__` 里的首次赋值同样要走这两条路,否则初始化即爆栈。

Q

Q4. 猴子补丁的典型场景有哪些?为什么"先保存原函数引用"是好习惯?

答案:典型场景:给第三方库打 bug 修复、统一包装(加超时/日志/缓存)、测试替身。保存原引用可随时还原补丁(尤其测试结束)、可在包装函数里调用原实现(装饰器模式)、可对比新旧行为。不保存则补丁不可逆,排障时无法对照,是生产事故隐患。

Q

Q5. 为什么说"字符串驱动的反射"是安全风险?如何防护?

答案:`getattr(obj, user_input)` 或 `import module(user_input)` 把用户输入变成代码路径/模块加载——恶意输入可访问任意属性、加载任意模块,等价于代码注入。防护:① 输入白名单校验(正则/集合匹配);② 用映射表翻译"允许的键"而非直接透传字符串;③ 反射加载失败时只暴露受控的错误信息,不泄露内部路径。

Q

Q6. 写通用序列化器时,为什么需要 `seen` 集合?不加会怎样?

答案:对象图可能有环(双向引用、自引用):`A.friend = B` 且 `B.friend = A`,递归遍历将无限循环直到 `RecursionError`。`seen` 集合记录已访问对象的 `id`,再次遇到时返回引用标记(如 `{"_ref_": id}`)或直接跳过,把环变成有限结构。真实序列化器(JSON 默认)遇到环会抛错,定制时也要显式决定环的表示。

章末实验题:通用遍历器与反射序列化

实验 11.1 对象遍历打印器与反射序列化(覆盖:\$11.1–\$11.7)

需求:① 写一个 `dump(obj, indent=0)` 函数:递归遍历任意对象的属性,以缩进树状打印"属性名:值",基本类型直接打印,容器逐元素,自定义对象进一层缩进;② 处理循环引用(`seen` 集合,打印 [循环引用] 标记);③ 只打印公开成员(跳过 `__` 开头,跳过方法/类);④ 写一个 `to_jsonable(obj)`:把任意对象转成可 JSON 序列化的结构(复用 \$11.7 思路,支持 list/dict/自定义类/基本类型);⑤ 用 `User/Order` 两层对象验证输出,并对比 `json.dumps` 结果;⑥ 用 `__repr__` 配合 `inspect` 输出对象摘要。

覆盖知识点:内省工具 `vars/dir/getattr`(\$11.1)/`inspect`(\$11.2)/属性拦截(\$11.3)/反射序列化(\$11.7)。

实验环境:Python 3.10+,单文件 `objtool.py`。

```

# objtool.py — 对象遍历打印机与反射序列化(覆盖第11章全部知识点)
import inspect
import json

def _is_public_member(value) -> bool:
    """跳过方法、魔法属性与私有成员"""
    if inspect.ismethod(value) or inspect.isfunction(value):
        return False
    return True

def dump(obj, indent: int = 0) -> None:
    """树状打印对象结构:属性名: 值"""
    prefix = " " * indent
    if isinstance(obj, (str, int, float, bool)) or obj is None:
        print(f"{prefix}{obj!r}")
        return
    if isinstance(obj, (list, tuple)):
        print(f"{prefix}[{type(obj).__name__}]")
        for item in obj:
            dump(item, indent + 1)
        return
    if isinstance(obj, dict):
        print(f"{prefix}{{dict}}")
        for k, v in obj.items():
            print(f"{prefix} {k}: ", end="")
            dump(v, indent + 2)
        return
    print(f"{prefix}<{type(obj).__name__}>")
    for name, value in vars(obj).items():
        if name.startswith("__"):
            continue
        print(f"{prefix} {name}: ", end="")
        dump(value, indent + 2)

def to_jsonable(obj, seen: set | None = None) -> object:
    """任意对象转可 JSON 序列化结构,循环引用安全"""
    if seen is None:
        seen = set()
    if isinstance(obj, (str, int, float, bool)) or obj is None:
        return obj
    if isinstance(obj, (list, tuple)):
        return [to_jsonable(x, seen) for x in obj]
    if isinstance(obj, dict):
        return {str(k): to_jsonable(v, seen) for k, v in obj.items()}
    if id(obj) in seen:
        return {"__ref__": id(obj)}
    seen.add(id(obj))
    result = {}
    for name, value in vars(obj).items():
        if name.startswith("__"):
            continue

```

```
    result[name] = to_jsonable(value, seen)
return result
```

```
class User:
    def __init__(self, name: str, age: int) -> None:
        self.name = name
        self.age = age
        self.orders = []

    def __repr__(self) -> str:
        return f"User({self.name}, {self.age})"

class Order:
    def __init__(self, oid: str, amount: float) -> None:
        self.oid = oid
        self.amount = amount

def main() -> None:
    alice = User("alice", 30)
    alice.orders = [Order("A001", 199.0), Order("A002", 58.5)]
    alice.friend = alice          # 自引用:验证循环引用处理

    print("== dump 输出 ==")
    dump(alice)

    print("== to_jsonable 输出 ==")
    print(json.dumps(to_jsonable(alice),
                     ensure_ascii=False, indent=1))

    print("== inspect 摘要 ==")
    print(f"alice 类型: {type(alice).__name__}")
    print(f"User 方法: {[m for m in dir(User) if not m.startswith('_')]}")

if __name__ == "__main__":
    main()
```

设计说明:① dump 与 to_jsonable 共用"vars + 过滤魔法成员"的遍历骨架,一个面向人读、一个面向机器读,体现反射工具的分工;② 循环引用统一用 seen 集合拦截,打印与序列化都给出可见标记;③ 过滤规则用 inspect 判断可调用对象,避免把方法当字段打印;④ 自引用示例(alice.friend = alice)直接验证防环逻辑;⑤ 输出均可通过测试断言校验(如 to_jsonable 结果包含 friend 的 __ref__ 标记)。

下一章预告:第12章 装饰器与函数式编程——装饰器本质、functools.wraps、带参数与类装饰器、contextmanager 与 lru_cache、自写计时/重试/缓存。让横切逻辑从代码里蒸发。

第12章 装饰器与函数式编程

装饰器本质 · wraps · 带参数装饰器 · contextmanager · lru_cache

本章目标:讲透装饰器:从"函数是对象"的本质出发,到 @ 语法糖、functools.wraps、带参数装饰器、类装饰器,再到 lru_cache/contextmanager 等内置宝库,最后手写生产级计时、重试、缓存装饰器。学完你能把日志、鉴权、超时、重试这类横切逻辑从业务代码里彻底蒸发。

直觉起点:C 里"给所有函数加日志"要改每个函数,或造宏、函数指针表;Python 的装饰器把"包装函数"变成一行语法: `@log` 等价于 `func = log(func)`。本质很简单——函数是对象,装饰器是"接收函数、返回新函数"的函数——威力来自它可以叠加、可以参数化、可以作用于类。

前置要求:第4章闭包与高阶函数、第9章异常与重试、第11章内省与元数据。

本章路线:第一阶段(1.5 小时)读 §12.1–§12.3,理解本质并写出第一个装饰器;第二阶段(1.5 小时)读 §12.4–§12.5,掌握内置宝库与类装饰器;第三阶段(2 小时)读 §12.6–§12.8,完成生产装饰器并做章末实验。

本章知识导图

- └─ 12.1 本质 —— 函数是对象, @ 就是赋值
- └─ 12.2 functools.wraps —— 保住函数元数据
- └─ 12.3 带参数装饰器 —— 三层嵌套的由来
- └─ 12.4 内置宝库 —— lru_cache / contextmanager
- └─ 12.5 类装饰器 —— 装饰类与装饰方法
- └─ 12.6 生产三件套 —— 计时 / 重试 / 缓存
- └─ 12.7 组合与顺序 —— 装饰器的叠加规则
- └─ 12.8 生产实践 —— 横切关注点的边界

§12.1 本质:函数是对象,@ 就是赋值

函数在 Python 里是一等公民:可以赋给变量、放进列表、作为参数传递。装饰器就是利用这一点——@deco 加在 def 前,等价于 `func = deco(func)`。手写一个"给函数打日志"的装饰器,过程一目了然:


```
import time

def logged(func):
    """装饰器:打印函数名与耗时,然后调用原函数"""
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} 耗时 "
              f"{{(time.perf_counter() - start) * 1000:.2f}} ms")
        return result
    return wrapper

@logged
def fetch_data(url: str) -> str:
    return f"数据来自 {url}"

print(fetch_data("https://example.com"))
# fetch_data 耗时 0.01 ms
# 数据来自 https://example.com
```

C(函数指针包装)	Python(装饰器)
包装函数要手写 wrapper 并手动覆盖	@deco 一行完成"函数 = 包装(函数)"
宏包装有类型与作用域限制	装饰器是普通函数,任意函数/方法/类通用
包装链要逐层手动调用	装饰器可叠加,顺序即层级
包装后原符号丢失,难还原	保留原函数引用,随时可还原(f.__wrapped__)

wrapper 用 `*args`, `**kwargs` 透传一切参数,这是装饰器的通用骨架:外面包一层,里面调原函数,前后插入横切逻辑。注意原函数在 `def` 时就执行了包装——装饰器代码只运行一次(导入期),wrapper 才在每次调用时运行。

§12.2 functools.wraps:保住函数元数据

上面的写法有个问题:装饰后 `fetch_data.__name__` 变成了 "wrapper",help/签名/文档全丢,调试与文档全部失真。functools.wraps 把原函数的元数据复制到 wrapper 上,一行修复:

```

from functools import wraps

def logged(func):
    @wraps(func)                # 复制 __name__ / __doc__ / __wrapped__ 等
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} 耗时 "
              f"{{(time.perf_counter() - start) * 1000:.2f}} ms")
        return result
    return wrapper

@logged
def fetch_data(url: str) -> str:
    """按 URL 拉取数据"""
    return f"数据来自 {url}"

print(fetch_data.__name__)      # fetch_data(而不是 wrapper)
print(fetch_data.__doc__)      # 按 URL 拉取数据
help(fetch_data)               # 签名、文档全部正常

```

误区:装饰器不写 @wraps。 不带 wraps 的装饰器会让被装饰函数的名字、文档、签名全部变成 wrapper 的——堆栈跟踪里全是 wrapper, help() 显示错误文档, 依赖签名内省的工具(第11章的 inspect.signature、参数校验框架)全部失灵。生产规范:自定义装饰器一律第一行 @wraps(func), 这是装饰器的"安全带"。

§12.3 带参数装饰器:三层嵌套的由来

@retry(times=3) 这类"带参数的装饰器"比无参版多一层:装饰器工厂(接收配置)返回真正的装饰器(接收函数)返回 wrapper。三层嵌套是必然结构,别嫌绕,理解层级即可:

```

import time
from functools import wraps

def retry(times: int = 3, delay: float = 0.5):
    """装饰器工厂:返回真正装饰器,失败重试 times 次"""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last = None
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    last = e
                    time.sleep(delay * (attempt + 1))
            raise last # 全部失败,抛最后一次异常
        return wrapper
    return decorator

@retry(times=3, delay=0.2) # 工厂 → 装饰器 → wrapper
def unstable_api() -> str:
    ...

```

记忆方法:配置层(工厂)→ 函数层(装饰器)→ 调用层(wrapper)。如果只想记一个,记住"最外层拿参数,中间层拿函数,最内层拿调用"。带参数装饰器比无参版更常见,因为生产配置(次数、超时、白名单)总要参数化。

§12.4 内置宝库:lru_cache 与 contextmanager

标准库已经给你写好了一批生产级装饰器,先认识最常用的两个: `functools.lru_cache` 给纯函数加内存缓存——同样的入参直接返回上次结果,递归与频繁重复计算场景立省一个量级:

```

from functools import lru_cache

@lru_cache(maxsize=128) # 缓存最近 128 个入参的结果
def fib(n: int) -> int:
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(50)) # 瞬间返回,不再指数爆炸
print(fib.cache_info()) # CacheInfo(hits=..., misses=...)
fib.cache_clear() # 清空缓存

```

`@contextmanager` 把生成器变成上下文管理器——第5章手写的类版,这里一行搞定,是"with 逻辑"的标准生产写法:

```

from contextlib import contextmanager

@contextmanager
def timer(label: str):
    """用生成器实现上下文管理器:yield 前后是进入/退出逻辑"""
    start = time.perf_counter()
    try:
        yield
    finally:
        print(f"[{label}] 耗时 {(time.perf_counter() - start) * 1000:.2f} ms")

with timer("查询订单"):
    for _ in range(100000):
        pass

```

误区:给有副作用或可变入参的函数加 lru_cache。 lru_cache 靠入参哈希做缓存键,两个陷阱:① 入参含 list/dict 会直接 TypeError(unhashable);② 函数有副作用(打印、改全局、读时间)时,缓存会跳过副作用——结果对而行为错。纪律:lru_cache 只用于纯函数;需要复杂键用 cache_key 函数转成可哈希元组。

§12.5 类装饰器:装饰类与装饰方法

装饰器同样作用于类:接收类,返回类(或包装对象)。经典场景是"给类的所有方法统一加日志/鉴权",配合第10章的元编程思路:

```

from functools import wraps

def log_methods(cls):
    """类装饰器:给类中所有公开方法包上调用日志"""
    for name, value in list(vars(cls).items()):
        if callable(value) and not name.startswith("__"):
            @wraps(value)
            def wrapped(*args, _f=value, _n=name, **kwargs):
                print(f"调用 {cls.__name__}.{_n}")
                return _f(*args, **kwargs)
            setattr(cls, name, wrapped)
    return cls

@log_methods
class Service:
    def start(self) -> None:
        print("启动服务")

    def stop(self) -> None:
        print("停止服务")

Service().start() # 调用 Service.start / 启动服务

```

细节:循环里用 `_f=value`, `_n=name` 默认参数绑定当前值——否则闭包捕获的是循环变量,所有方法都调用最后一个(第4章晚期绑定坑的实战版)。类装饰器与元类能力重叠,但更直白;规则照旧:简单场景用类装饰器,需要拦截"创建过程"才用元类。

§12.6 生产三件套:计时、重试、缓存

把本章知识组装成生产可用的三个装饰器。先看计时与重试的完整版(支持 `async` 函数用 `asyncio` 版,这里展示同步版):

```
import functools
import time

def timed(func):
    """计时装饰器:记录耗时,支持自定义日志函数"""
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            cost = (time.perf_counter() - start) * 1000
            print(f"{func.__name__} 耗时 {cost:.2f} ms")
    return wrapper

def retry(times=3, delay=0.5, on=(Exception,)):
    """重试装饰器:只重试 on 指定的异常,指数退避"""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except on as e:
                    if attempt == times - 1:
                        raise
                    time.sleep(delay * (2 ** attempt))
            return wrapper
        return decorator
```

重试装饰器的生产要点: `on=(Exception,)` 让调用方显式声明"哪些异常值得重试"——默认全捕获是危险的(业务错误重试3次纯属浪费);退避用 `delay * 2**attempt` 指数增长,避免重试风暴。`timed` 用 `finally` 保证异常路径也计时——这是"计时必须覆盖失败路径"的细节。

再看缓存装饰器——缓存键的设计是灵魂:默认用 `repr` 元组,可变参数先转成可哈希形式:

```

import functools

def memoize(func):
    """手写缓存:入参转可哈希键,命中直接返回"""
    cache: dict = {}
    MISS = object()

    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        try:
            key = (args, tuple(sorted(kwargs.items())))
        except TypeError:
            return func(*args, **kwargs)  # 参数不可哈希,跳过缓存
        result = cache.get(key, MISS)
        if result is MISS:
            result = func(*args, **kwargs)
            cache[key] = result
        return result
    return wrapper

@memoize
def query(user_id: int) -> dict:
    """模拟查询:同一用户只查一次"""
    return {"id": user_id, "name": "user" + str(user_id)}

print(query(1) is query(1))          # True — 第二次命中缓存

```

与 `lru_cache` 相比,手写版展示了缓存三要素:键构造、命中检查、存储。生产直接用 `lru_cache`;手写它的意义在于理解——当需要"按自定义键缓存""分环境缓存""缓存失效策略"时,你会知道自己要改哪里。

§12.7 组合与顺序:装饰器的叠加规则

多个装饰器叠加时,执行顺序是从下往上(离函数最近的先包装)。看例子:

```

@timed                                # 外层:最后包装,最先执行外层逻辑
@retry(times=2, delay=0.1)
def fragile_call() -> str:
    import random
    if random.random() < 0.5:
        raise ConnectionError("偶发失败")
    return "成功"

print(fragile_call())
# 调用链:timed.wrapper -> retry.wrapper -> fragile_call
# 重试发生在 timed 计时内部,一次计时的总耗时包含所有重试

```

误区:装饰器顺序想当然。顺序错了行为就错:把 `retry` 放外层、`timed` 放内层,则每次重试都被单独计时,总耗时被拆成多段;把缓存放外层、校验放内层,则未校验的数据可能已进缓存。规则:横切关注点按"越通用越在外"排列(缓存在最外、重试次之、业务校验最内),并对照调用链验证。

§12.8 生产实践:横切关注点的边界

装饰器是横切关注点(日志、鉴权、限流、缓存、重试)的标准载体。判断"该不该写成装饰器"的三个标准:① 多函数共享(两三个函数用就别抽象);② 行为正交(与业务逻辑无关,可独立测试);③ 参数化需求明确(次数、白名单、阈值)。

C(手工横切)	Python(装饰器)
每个函数重复写超时/重试样板	@retry(3) 一行,配置显式
横切代码与业务代码交织,改一处动全身	横切逻辑独立成装饰器,业务函数干净
加一个横切点要动 N 个函数	加一个 @ 符号即可,回滚同样一行
横切逻辑难以单独测试	装饰器可独立单测(配假函数)

工程建议 1:装饰器库单独放一个模块(decorators.py),每个装饰器配单元测试:用假函数验证包装行为、异常路径、参数透传;文档里写清"配置含义、适用场景、顺序要求"——装饰器是隐式逻辑,文档就是它的安全网。

工程建议 2:装饰器里不要吞异常、不要改签名、不要改返回值类型——它应该"透明"。排障时用 functools 的 `__wrapped__` 链(`fragile_call.__wrapped__`)直达原函数,这条链要保留,别在装饰器里覆盖它。

本章速查表

场景	写法
无参装饰器	<code>def deco(func): @wraps(func) def w(*a, **k): ...; return w</code>
带参装饰器	<code>def deco(conf): def d(func): ...; return d</code> 三层嵌套
保元数据	wrapper 第一行 <code>@functools.wraps(func)</code>
函数缓存	<code>@lru_cache(maxsize=128)</code> ,仅纯函数
清缓存	<code>fib.cache_clear()</code> ,查看 <code>fib.cache_info()</code>
生成器上下文	<code>@contextmanager</code> + <code>yield</code> 前后
类统一包装	类装饰器遍历 <code>vars(cls)</code> ,绑定默认参防晚期绑定
重试配置	<code>on=(异常,)</code> 显式声明,指数退避
计时含异常	<code>finally</code> 里算耗时
缓存键	<code>args + sorted(kwargs)</code> 转元组;不可哈希则跳过缓存
叠加顺序	从下往上包装,越通用越在外
还原原函数	<code>func.__wrapped__</code> (wraps 自动设置)
透明原则	不吞异常、不改签名、不改返回类型
使用标准	多函数共享 + 行为正交 + 参数化
测试	配假函数独立单测,异常路径必测
文档	写清配置含义、适用场景、顺序要求

最小必会清单

1. 能手写无参装饰器骨架,并解释 `@deco` 与 `func = deco(func)` 的等价关系
2. 能说出 `wraps` 的作用与缺失后果,并始终在装饰器里使用它
3. 能写出带参数装饰器(工厂→装饰器→wrapper 三层),并配置重试/超时
4. 能说出 `lru_cache` 的适用条件(纯函数)与两个陷阱(不可哈希、副作用)
5. 能用 `@contextmanager` 把生成器变成上下文管理器
6. 能解释装饰器叠加顺序,并用调用链验证行为

自测题

Q

Q1. `@deco` 到底做了什么?装饰器代码什么时候运行,wrapper 什么时候运行?

答案:`@deco` 等价于 `func = deco(func):deco` 在定义语句执行时(导入期)被调用一次,接收原函数、返回 wrapper,名字 `func` 绑定到 `wrapper`。之后每次调用 `func()` 实际执行 `wrapper`。所以装饰器工厂逻辑在导入期运行一次,包装逻辑每次调用运行——用这个区分,就能判断装饰器里哪些代码可以放函数外(初始化缓存、编译正则)。

Q

Q2. 不带 `@wraps` 的装饰器会带来哪些具体问题?

答案:① `__name__` 变成 `wrapper`,堆栈跟踪与日志里全是 `wrapper` 层,定位困难;② `__doc__` 丢失,`help()` 与文档生成器显示错误内容;③ `inspect.signature` 拿到 `wrapper` 的 `(*args, **kwargs)`,依赖签名的框架(参数校验、自动文档)失灵;④ `__wrapped__` 链不存在,无法还原原函数。`@wraps` 一行全部修复。

Q

Q3. 为什么带参数装饰器需要三层?能少一层吗?

答案:因为 `@retry(times=3)` 的求值过程:`retry(times=3)` 先执行返回装饰器,装饰器再接收函数返回 `wrapper`。三层分别对应:配置参数层(`retry`)、函数层(`decorator`)、调用层(`wrapper`)。无参装饰器两层即可(装饰器+`wrapper`);带参版少一层就没地方放配置。要一眼看懂,记住"最外层拿参数、中间拿函数、最内拿调用"。

Q

Q4. `lru_cache` 装饰一个"打印日志并返回当前时间"的函数,为什么结果可疑?

答案:该函数有副作用(打印)且结果依赖时间(非纯函数)。`lru_cache` 命中缓存时直接返回上次结果,不执行函数体——副作用被跳过、时间结果过时。缓存只适合纯函数:同样的入参永远产生同样的结果、无外部影响。非纯函数要么不用缓存,要么把"时间""环境"也纳入缓存键。

Q

Q5. 类装饰器循环里为什么用 `_f=value, _n=name` 默认参数?

答案:闭包捕获的是变量而非值:循环结束后 `value/name` 指向最后一个方法,所有 `wrapper` 实际都调用它(第4章晚期绑定坑)。默认参数在定义时求值,把当前迭代的值"钉"进 `wrapper`,每个 `wrapper` 调用各自的方法。这是装饰器与循

环组合的头号陷阱,任何“循环里造闭包”的场景都适用。



Q6. @timed 与 @retry 叠加时,顺序不同会怎样?如何决定顺序?

答案:@timed 在上、@retry 在下:timed 包住 retry,一次计时包含所有重试(总耗时)。反过来则每次重试被单独计时。规则:通用性高的在外——缓存最外(命中就不进内层)、重试其次、业务逻辑最内;验证方式:从内到外列出调用链,检查每个横切点是否在期望的边界上执行。

章末实验题:装饰器工具箱

实验 12•1 计时与重试装饰器库(覆盖:\$12.1–\$12.6)

需求:① 实现 timed 装饰器:支持自定义日志函数(默认 print),记录名称、耗时(ms)、成功/失败标记;② 实现 retry 装饰器:支持次数、退避基数、可重试异常元组、最大总时长上限(超限立即放弃);③ 实现 cached 装饰器:支持 TTL 过期时间(到期的键自动失效重算);④ 三个装饰器可叠加,顺序按"cached 最外、retry 次之、timed 最内";⑤ 用一个模拟的不稳定服务(按概率失败 + 随机耗时)验证:重试后成功、缓存命中不再调用、计时包含全部重试;⑥ 输出验证报告并写单元测试(用假函数断言调用次数)。

覆盖知识点:装饰器本质与 wraps(\$12.1、\$12.2)/带参数装饰器(\$12.3)/缓存键与 TTL(\$12.6)/叠加顺序(\$12.7)。

实验环境:Python 3.10+,单文件 `decorators_lab.py`。

```

# decorators_lab.py — 生产装饰器工具箱(覆盖第12章全部知识点)
import functools
import random
import sys
import time

def timed(logger=print):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            start = time.perf_counter()
            try:
                return func(*args, **kwargs)
            finally:
                ok = sys.exc_info()[0] is None
                logger(f"[timed] {func.__name__} "
                    f"{{(time.perf_counter() - start) * 1000:.2f}} ms({{'成功' if ok else '异常'}})")
        return wrapper
    return decorator

def retry(times=3, backoff=0.2, on=(Exception,), max_total=10.0):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            deadline = time.monotonic() + max_total
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except on:
                    if attempt == times - 1:
                        raise
                    if time.monotonic() >= deadline:
                        raise
                    time.sleep(backoff * (2 ** attempt))
            return wrapper
    return decorator

def cached(ttl: float = 60.0):
    def decorator(func):
        cache: dict = {}
        store = {"hits": 0, "misses": 0}
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            try:
                key = (args, tuple(sorted(kwargs.items())))
            except TypeError:
                return func(*args, **kwargs)
            entry = cache.get(key)
            if entry and time.monotonic() - entry[1] < ttl:
                store["hits"] += 1
                return entry[0]
            store["misses"] += 1
            result = func(*args, **kwargs)

```

```

        cache[key] = (result, time.monotonic())
    return result
    wrapper.stats = lambda: dict(store)
    return wrapper
return decorator

```

```

@cached(ttl=1.0)
@retry(times=3, backoff=0.1)
@timed()
def unstable_service(key: str) -> str:
    """模拟不稳定服务:30% 概率失败,成功返回带时间戳结果"""
    if random.random() < 0.3:
        raise ConnectionError("服务偶发不可用")
    time.sleep(random.uniform(0.01, 0.05))
    return f"{key}-{time.time():.3f}"

def main() -> None:
    random.seed(7)
    print("== 首次调用(可能触发重试)==")
    r1 = unstable_service("order")
    print("结果:", r1)

    print("== 缓存命中(1 秒内再调,不执行原函数)==")
    for _ in range(3):
        r2 = unstable_service("order")
        assert r2 == r1, "缓存未命中,结果不一致!"
    print("命中统计:", unstable_service.stats())

    print("== TTL 过期后重算 ==")
    time.sleep(1.2)
    r3 = unstable_service("order")
    print("过期后结果:", r3, "(应不同于首次)")

    print("== 断言与验证 ==")
    assert unstable_service.stats()["hits"] >= 3
    assert unstable_service.stats()["misses"] >= 2
    print("全部验证通过")

if __name__ == "__main__":
    main()

```

设计说明:① 装饰顺序 cached 最外:缓存命中时重试与计时都不执行,行为正确;② retry 的 max_total 用 time.monotonic()(不受系统时间调整影响)与"预计退避是否超限"双保险,绝不无限拖延;③ cached 用 (args, sorted(kwargs)) 做键、monotonic 时间戳做 TTL,miss/hit 统计挂在 wrapper 上便于断言;④ 实验用 assert 自动验证,加随机种子保证可复现;⑤ 三个装饰器都可独立单测——配一个假函数断言调用次数与参数透传即可。

下一章预告:第13章 模块与工程化——import 机制、sys.path 与 __init__.py、venv 与 pip、pyproject.toml、项目布局与发布。让代码从"能跑"走向"能交付"。

第13章 模块与工程化

import 机制 · sys.path · venv 与 pip · pyproject.toml

本章目标:讲透 Python 的模块系统与工程化:import 的搜索与缓存机制、包与 `__init__.py`、绝对/相对导入、循环导入陷阱,再到 venv 虚拟环境、pip 依赖管理、pyproject.toml 项目布局与 PyPI 发布。学完你能组织出一个多文件、可测试、可打包、可交付的规范项目。

直觉起点:C 的 `#include` 在编译期把头文件文本粘贴进来,链接期解决符号;Python 的 `import` 在运行期执行"模块文件本身",并缓存到 `sys.modules`。模块即"执行一次的文件",包即"带 `__init__.py` 的目录"——理解这两句话,import 的一切行为都有了解释。

前置要求:第4章函数与作用域、第11章动态导入(importlib)、第12章装饰器(项目常配装饰器库)。

本章路线:第一阶段(1.5 小时)读 §13.1–§13.4,理解 import 机制;第二阶段(1.5 小时)读 §13.5–§13.6,掌握 venv 与 pip;第三阶段(2 小时)读 §13.7–§13.8,完成项目布局与章末实验。

本章知识导图

- └─ 13.1 import 机制 — 执行一次、sys.modules 缓存
- └─ 13.2 sys.path 与包 — 搜索路径与 `__init__.py`
- └─ 13.3 导入风格 — 绝对导入与相对导入
- └─ 13.4 `__name__` 守卫 — 脚本与模块双模式
- └─ 13.5 venv — 虚拟环境隔离依赖
- └─ 13.6 pip — 安装、锁定与升级
- └─ 13.7 pyproject.toml — 现代项目布局
- └─ 13.8 发布与交付 — PyPI 与版本语义

§13.1 import 机制:执行一次,缓存永久

`import json` 在运行期做的事:① 在 `sys.path` 里找 `json.py`;② 首次导入时执行整个文件(模块顶层代码全部运行);③ 把结果放进 `sys.modules["json"]` 缓存;④ 以后每次 `import` 直接取缓存,不再执行。因此"模块顶层代码只执行一次",副作用(注册、打印)也只发生一次:

```
# db.py — 一个模块文件
print("db.py 被导入")                # 整个项目只打印一次

CONNECTIONS = []                     # 模块级共享状态

def connect(name: str) -> None:
    CONNECTIONS.append(name)

# main.py — 另一个文件
import db                             # 执行 db.py, 缓存
import db                             # 直接命中缓存, 不执行
db.connect("orders")
print(db.CONNECTIONS)                # ['orders']
```

C(#include)	Python(import)
编译期文本粘贴,重复包含要 #pragma once	运行期执行一次,缓存复用,天然防重
头文件与实现分离(.h/.c)	一个 .py 既是声明也是实现
编译单元间靠链接器结合	import 即链接,依赖关系运行时可见
头文件变化触发全量重编译	模块变化只需重跑程序

模块级共享状态(CONNECTIONS)是“模块即单例”的体现:任何 import 它的文件看到的是同一个列表。生产上常利用这点做配置中心、连接池、注册表——但注意线程安全(第9章)。`import db` 后访问 `db.xxx`,与 `from db import connect` 的区别:后者把名字直接绑进当前命名空间,改的是“副本绑定”,模块里的原对象不受影响。

与 C 的头文件不同,import 建立的是运行时对象关系:db.CONNECTIONS 是同一个列表对象,任何一处 append,全项目可见。这个特性让“模块级单例”成为 Python 组织共享状态的主要方式——配置、连接池、注册表都习惯放在模块顶层。生产提醒:模块级可变状态要与第9章线程安全结合起来考虑,必要时加锁或用不可变数据。

§13.2 sys.path 与包: __init__.py 的职责

import 在 `sys.path` 的目录列表里按顺序找模块:当前脚本目录、环境变量 PYTHONPATH、标准库、site-packages。把 import 失败当线索:先 `print(sys.path)` 看搜索路径,再看模块是否真的躺在某个路径下:

```
import sys
print(sys.path)
# ['项目根', ..., 'python312\\Lib\\site-packages', ...]

# 包 = 带 __init__.py 的目录;包内模块用点号路径导入
# project/
#   app/
#     __init__.py    ← 有此文件,app 才是包
#     main.py
#     utils/
#       __init__.py
#       format.py
import app.utils.format as fmt    # 点号路径即目录层级
```

`__init__.py` 有两个职责:① 标记目录为包(没有它,同名的普通目录不会被当作包导入);② 收纳“对外 API”——在 `__init__.py` 里 `from .utils.format import fmt_time`,使用方就能 `from app import fmt_time`,内部结构调整不影响调用方。这是包设计的核心手法:内部自由重组,对外保持稳定。

判断一个目录是否真是包,就看 `__init__.py` 是否存在。Python 3.3 引入的命名空间包允许无 `__init__.py` 的目录被导入,但那只是隐式机制;生产项目一律显式写 `__init__.py`,把“包边界”写清楚——既能收敛 API,也能防止两个同名模块被意外合并。

§13.3 导入风格:绝对导入与相对导入

绝对导入从包根写全路径(`from app.utils import format`),清晰但写起来长;**相对导入**从当前包出发(`from .utils import format`、`from ..common import x`),重构时跟随包移动。生产规范:包内互引用一律相对导入,包外一律绝对导入:

```
# app/utils/format.py
from ..common.constants import MAX_LEN    # 相对导入:上一级包的模块

# app/common/constants.py
MAX_LEN = 100

# 顶层入口 app/main.py
from app.utils.format import fmt_time    # 绝对导入(以包根为锚)
```

误区:循环导入。 a.py 顶部 import b,而 b.py 顶部 import a——a 还没执行完就回头找 b,而 b 正在等 a,必然 ImportError(或拿到半初始化模块)。常见诱因:把“运行时才需要的导入”提到模块顶部、两个模块互相依赖对方常量。解法:① 依赖方向理成单向(公共代码下沉到 common);② 把导入挪进函数体内(延迟到调用时,此时两边都已加载完);③ 用 __init__.py 做组装层解耦。

相对导入的边界:被 `python -m` 运行或被导入时,相对导入才有效;直接 `python app/utils/format.py` 运行会报“attempted relative import”。所以包内脚本一律从项目根用 `python -m app.main` 启动——这也是工程化第一步。

§13.4 __name__ 守卫:脚本与模块双模式

一个 .py 文件被直接运行时 `__name__ == "__main__"`,被导入时 `__name__ == "模块路径"`。利用这一点,同一个文件既能当脚本跑,也能被导入复用,且导入时不会误执行演示代码:

```
"""订单工具:可导入,也可命令行直接运行"""

def total(prices: list) -> float:
    return sum(prices)

if __name__ == "__main__":
    # 只有直接运行时才执行;被 import 时跳过
    print(total([1.0, 2.5, 3.0]))    # 6.5
```

这是工程化的基础纪律:所有可执行入口文件都包 `__main__` 守卫,并习惯用 `python -m` 运行包内模块(它会把包根放进 `sys.path`,相对导入可用)。第9章的进程池、第8章的服务器入口都依赖这个守卫,忘了写会在导入时触发整段逻辑。

一个常见反例:工具脚本在文件末尾直接写测试代码,被他人 import 时测试逻辑全量执行,轻则打印污染输出,重则执行删除等危险操作。养成习惯:模块文件的顶层只放定义与常量,一切“运行时才该发生的事”放进函数,再由 `__main__` 守卫决定是否调用。

§13.5 venv:虚拟环境隔离依赖

不同项目需要不同版本的第三方库,装进系统全局会互相打架——**venv** 为每个项目建独立环境:自己的 python、自己的 site-packages。创建与激活:

```
# 创建虚拟环境(在项目根,一次)
python -m venv .venv

# 激活(每次新终端)
# Windows(CMD): .venv\Scripts\activate
# Windows(PowerShell): .venv\Scripts\Activate.ps1
# Linux/macOS: source .venv/bin/activate

# 激活后,终端提示符出现 (.venv),pip 安装全部进入本项目
python -m pip install requests
python -c "import requests; print(requests.__version__)"
```

C(系统库)	Python(venv)
库装进系统,版本全局唯一	每项目独立 site-packages,版本互不干扰
升级系统库影响所有程序	升级只影响当前虚拟环境
依赖冲突只能靠改名/静态链接	环境隔离从源头消除冲突
部署要匹配目标机库版本	requirements 锁定 + 重建环境,可复现

误区:依赖装进了全局。忘了激活虚拟环境就 pip install,包落进系统 Python——换个环境程序就崩,排查时"我明明装了"却 ImportError。生产规范:每个项目第一件事创建 .venv 并激活;pip 一律用 `python -m pip` (避免"系统 pip 与当前 python 不一致");.venv 目录加入 .gitignore 绝不提交。

§13.6 pip:安装、锁定与升级

pip 的日常四件事:安装、锁定、按锁定安装、升级。锁定用 `pip freeze > requirements.txt` 导出当前环境全部依赖的精确版本,换机器 `pip install -r requirements.txt` 一键复现:

```
# 安装指定版本与范围
python -m pip install requests==2.31.0
python -m pip install "flask>=3.0,<4"      # 版本范围

# 锁定当前环境(记录精确版本)
python -m pip freeze > requirements.txt

# 按锁定文件重建环境
python -m pip install -r requirements.txt

# 升级 pip 自身与某依赖
python -m pip install --upgrade pip
python -m pip install --upgrade requests
```

误区:requirements.txt 不锁版本或随手删。写 `requests` 不写版本,今天装 2.31、下周装 3.x,行为漂移;删掉某行,别人复现环境时报错也查不出。规范:requirements.txt 由 freeze 生成、含精确版本、随代码入库;生产环境锁"平台无关的纯版本",必要处注释说明约束原因(如 "flask==3.0.3 # 4.x 改 API")。

升级依赖后的第一动作是跑全量测试;升级大版本(3.x → 4.x)前先读 release notes,别让 pip 替你决定命运。锁版本的精确度要分层:开发用范围,部署用精确——第14章会讲配套的版本策略。

§13.7 pyproject.toml:现代项目布局

Python 3.11+ 的现代项目用 **pyproject.toml** 声明一切:元数据、依赖、构建系统、工具配置。推荐布局——src 布局把"安装后的包"与"源码"分开,杜绝"测试能过、装完就崩"的经典事故:

```
# 项目布局(名字假设为 orderlib)
# orderlib/
#   pyproject.toml
#   README.md
#   src/
#       orderlib/
#           __init__.py
#           pricing.py
#           checkout.py
#   tests/
#       test_pricing.py
#       test_checkout.py

# pyproject.toml 核心内容
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "orderlib"
version = "0.1.0"
description = "订单计算库"
requires-python = ">=3.10"
dependencies = ["requests>=2.31"]

[project.optional-dependencies]
dev = ["pytest", "mypy", "ruff"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

src 布局的收益:测试从项目根运行时 import 到的是"安装后的 orderlib"而不是源码目录——安装行为与开发行为一致;构建工具只打包 src/orderlib,不会把 tests、文档意外塞进发行物。 `pip install -e ".[dev]"` 可编辑安装项目本体并带上开发工具,是本地开发的标配命令。

§13.8 发布与交付:PyPI 与版本语义

让项目可被 `pip install` 安装,是交付的最后一公里。构建 sdist/轮子后上传到 PyPI(公共)或私有源(企业):


```
# 构建发行物(项目根)
python -m pip install --upgrade build
python -m build # 生成 dist/*.tar.gz 与 *.whl

# 检查与上传(需要 PyPI 账号与令牌)
python -m pip install --upgrade twine
python -m twine check dist/* # 校验元数据与文件名
python -m twine upload dist/* # 上传 PyPI

# 其他人安装
python -m pip install orderlib
```

C(库分发)	Python(轮子)
源码包 + 编译,目标机要匹配编译器	wheel 是预打包格式,平台无关的纯 Python 轮子直接装
头文件/静态库/动态库三种形态	sdist 源码 + wheel 二进制(纯 Python 或含 C 扩展)
make install 手动管理	pip 自动解析依赖与版本,一键安装
ABI 不兼容要重新编译	cp310 等标签明确 Python 版本兼容范围

版本号用语义化版本(MAJOR.MINOR.PATCH):破坏性变更升 MAJOR,加功能升 MINOR,修 bug 升 PATCH。发布前过一遍质量清单:测试全绿、mypy 干净、README 更新、CHANGELOG 记录、版本号已升——**发布是不可逆操作,宁可多检查十分钟,不要发一个坏版本让全员踩坑。**

工程建议 1:新项目从第一天就用 pyproject.toml + src 布局 + venv + requirements 锁定,别等代码多了再迁移——迁移成本随文件数线性增长,而"规范骨架"只需十分钟。

工程建议 2:CI(持续集成)里加一个"干净环境安装"步骤:全新 venv + pip install -r requirements.txt + 跑测试,专门抓"本地能跑、别人装不上"的问题。这是工程化投入产出比最高的一个检查。

本章速查表

场景	写法
导入模块	<code>import db / from db import connect</code>
导入一次	<code>sys.modules</code> 缓存,重复 <code>import</code> 不重执行
看搜索路径	<code>import sys; print(sys.path)</code>
包内互引	相对导入 <code>from .utils import fmt</code>
包外引用	绝对导入 <code>from app.utils import fmt</code>
脚本/模块双模式	<code>if __name__ == "__main__":</code> 守卫
运行包内模块	<code>python -m app.main</code> (相对导入可用)
建虚拟环境	<code>python -m venv .venv</code> + 激活
装依赖	<code>python -m pip install requests==2.31.0</code>
锁定依赖	<code>pip freeze > requirements.txt</code>
复现环境	<code>pip install -r requirements.txt</code>
项目元数据	pyproject.toml 的 [project] 段
开发安装	<code>pip install -e ".[dev]"</code>
构建发行物	<code>python -m build</code> ,产物在 <code>dist/</code>
上传 PyPI	<code>python -m twine upload dist/*</code>
版本规则	语义化版本:MAJOR.MINOR.PATCH

最小必会清单

1. 能解释 `import` 的三步机制与 `sys.modules` 缓存的作用
2. 能说出 `__init__.py` 的两个职责,并组织一个三层包结构
3. 能区分绝对/相对导入的适用场景,并识别与修复循环导入
4. 能解释 `__name__ == "__main__"` 的语义并正确使用守卫
5. 能创建并激活 `venv`,用 `pip` 安装与锁定依赖
6. 能搭出 `pyproject.toml` + `src` 布局的规范项目,并说出构建发布的流程

自测题

Q

Q1. `import` 一个模块时,为什么顶层 `print` 只执行一次?强制重新执行怎么做?

答案:首次导入执行模块文件后,结果缓存进 `sys.modules`,后续 `import` 直接取缓存。强制重载:`importlib.reload(module)`——但 `reload` 不改写已绑定的引用(`from` 导入的名字不会更新),生产上几乎不用,调试热更新偶尔用。

Q

Q2. 循环导入为什么会失败?给出一个生产中最常见的诱因与两种解法。

答案:A 顶部 import B 时 B 还没执行完,而 B 顶部又在等 A——一方拿到未初始化模块即失败。常见诱因:两个模块互相引用对方的常量/函数。解法:① 把公共依赖下沉到第三模块,依赖变单向;② 延迟导入——把 import 移进函数体内,调用时才执行,此时两边模块都已加载完。

Q

Q3. 直接运行 `python app/utils/format.py` 报 "attempted relative import",为什么?正确做法?

答案:直接运行时,Python 把该文件当作顶层脚本, `__package__` 为空,相对导入(点号路径)没有锚点。正确做法:从项目根运行 `python -m app.utils.format`,此时解释器按包语义加载,相对导入成立。因此工程规范是"包内一切入口用 `-m` 运行"。

Q

Q4. `venv` 解决了什么问题?为什么说 "pip install 装进了全局" 是常见事故?

答案:`venv` 为项目提供独立 Python 与 `site-packages`,解决多项目依赖版本冲突。未激活 `venv` 时 `pip` 装进系统 Python,换环境(部署机、同事机器)程序就 `ImportError`,且系统环境被污染、权限与安全风险并存。规范:先激活再安装,永远 `python -m pip`,环境信息随项目文档记录。

Q

Q5. `requirements.txt` 为什么不锁版本会 "行为漂移"?`lock` 文件与范围约束各用在哪?

答案:不锁版本时,`pip` 按当前最新解析,今天装 2.x、下周同一命令装 3.x——测试过的组合被悄悄替换,行为漂移且难复现。策略:开发依赖用范围(`flask>=3,<4`)保留升级空间;部署与 CI 用精确锁定(`pip freeze` 或 `lock` 文件),保证 "测试过的版本=线上版本"。

Q

Q6. `src` 布局比 "包直接在项目根" 好在哪?为什么它能防 "测试过、装不上"?

答案:根布局下,测试 `import` 到的是源码目录,而安装后包的路径/内容可能不同(漏包、包含测试文件),出现 "测试全绿、生产崩" 的落差。`src` 布局让测试必须经安装后的路径 `import`,安装行为即测试行为;同时构建只打包 `src` 下的包,发行物干净。代价是安装前无法直接 `import`,属刻意设计。

章末实验题:多文件订单项目

实验 13.1 从零组织可交付项目(覆盖:§13.1–§13.8)

需求:① 按 `src` 布局建项目 `orderlib:src/orderlib/{__init__.py, pricing.py, checkout.py},tests/test_pricing.py`;② `pricing.py` 提供计算逻辑(折扣、税费、总额),`checkout.py` 依赖 `pricing` 并做结算与日志,二者用相对导入;③ `__init__.py` 导出对外 API(`pricing` 与 `checkout` 的核心函数),保证 `from orderlib import calc_total` 可用;④ 所有入口包 `__main__` 守卫,支持 `python -m orderlib.checkout` 直接运行演示;⑤ 写 `pytest` 测试(折扣边界、税额、空订单);⑥ 创建 `venv`、安装 `pytest` 与可编辑安装,跑通测试;⑦ 编写 `pyproject.toml`(元数据、dependencies、dev 依赖、`pytest` 配置),验证 `python -m build` 能产出发行物。

覆盖知识点:import 与包 (§13.1、§13.2)/相对导入与 __name__ 守卫 (§13.3、§13.4)/venv 与 pip (§13.5、§13.6)/pyproject 与构建 (§13.7、§13.8)。

实验环境:Python 3.10+,可联网安装 pytest。

```
# src/orderlib/pricing.py — 计价逻辑(无外部依赖,纯函数)
"""折扣、税额与总额计算"""

DISCOUNT_THRESHOLD = 1000.0
DISCOUNT_RATE = 0.9          # 满 1000 打九折
TAX_RATE = 0.06

def apply_discount(subtotal: float) -> float:
    """满额打折;负值入参直接抛错(入口校验)"""
    if subtotal < 0:
        raise ValueError(f"小计不能为负: {subtotal}")
    if subtotal >= DISCOUNT_THRESHOLD:
        return subtotal * DISCOUNT_RATE
    return subtotal

def calc_tax(amount: float) -> float:
    """按税率计算税额,结果保留两位小数"""
    return round(amount * TAX_RATE, 2)

def calc_total(prices: list) -> dict:
    """入参为单价列表,返回完整账单结构"""
    subtotal = sum(prices)
    discounted = apply_discount(subtotal)
    tax = calc_tax(discounted)
    return {
        "subtotal": round(subtotal, 2),
        "discounted": round(discounted, 2),
        "tax": tax,
        "total": round(discounted + tax, 2),
    }
```

```

# src/orderlib/checkout.py — 结算流程(消费 pricing,演示相对导入)
"""结算入口:组装账单并打印明细"""
from .pricing import calc_total

def checkout(items: list) -> dict:
    """结算:items 为商品对象列表,取 price 字段计价"""
    bill = calc_total([item["price"] for item in items])
    print(f"商品数 {len(items)} | 小计 {bill['subtotal']} | "
          f"折后 {bill['discounted']} | 税 {bill['tax']} | "
          f"总计 {bill['total']}")
    return bill

def main() -> None:
    goods = [{"name": "键盘", "price": 599},
             {"name": "鼠标", "price": 499}]
    checkout(goods)

if __name__ == "__main__":
    main()                                # python -m orderlib.checkout 运行

# src/orderlib/__init__.py — 对外 API 组装
"""orderlib:订单计价与结算库"""
from .pricing import calc_total, calc_tax
from .checkout import checkout

__all__ = ["calc_total", "calc_tax", "checkout"]

# tests/test_pricing.py — 单元测试
import pytest

from orderlib import calc_total

def test_discount_threshold() -> None:
    bill = calc_total([600, 500])          # 1100 触发九折
    assert bill["discounted"] == 990.0

def test_no_discount_below_threshold() -> None:
    bill = calc_total([100, 200])
    assert bill["discounted"] == 300.0

def test_empty_order() -> None:
    bill = calc_total([])
    assert bill["total"] == 0.0

def test_negative_price_rejected() -> None:

```

```
with pytest.raises(ValueError):  
    calc_total([-1])
```

工程清单:① pyproject.toml 按 §13.7 编写,name=orderlib、requires-python=>=3.10、[project.optional-dependencies] dev=["pytest"];② `python -m venv .venv` 并激活;③ `python -m pip install -e ".[dev]"` 可编辑安装;④ `python -m pytest` 全绿;⑤ `python -m build` 产出 dist 下两个发行物;⑥ `python -m pip install dist/orderlib-0.1.0-py3-none-any.whl` 装进干净环境验证"安装即用"。逐条核对:相对导入(是)、__main__ 守卫(是)、__init__ 收敛 API(是)、src 布局(是)、锁定与测试(是)。

下一章预告:第14章 类型系统进阶与生产实践——类型推导与检查、深浅拷贝、None 与 Optional、Union 与 match、dataclass 深度、静态与动态的平衡。全书收官的最后一讲。

第14章 类型系统进阶与生产实践

深浅拷贝 · Optional 与 Union · dataclass · mypy 静态检查

本章目标:全书收官。把动态类型哲学推向生产:值语义与引用语义的边界、深浅拷贝陷阱、None 与 Optional 的工程语义、Union 与 match 的窄化、dataclass 深度用法,以及 mypy 静态检查如何把"动态的自由"变成"静态的保障"。学完你能写出"标注完整、可静态检查、行为可预期"的生产级代码。

直觉起点:C 的类型系统在编译期把关,Python 的类型系统在运行期自由。两者不是对错,而是时机:mypy 把类型检查提前到开发期,运行时保留动态灵活性——"渐进式类型"让你在自由与安全之间自选刻度,这正是 Python 进生产环境的钥匙。

前置要求:第4章传参语义、第10章类型标注基础(TypeVar/Protocol)、第13章工程化流程。本章是全书综合应用。

本章路线:第一阶段(1.5 小时)读 §14.1–§14.3,吃透对象语义与拷贝;第二阶段(1.5 小时)读 §14.4–§14.6,掌握 None/Union/dataclass;第三阶段(2 小时)读 §14.7–§14.8 并完成章末实验,配 mypy 实战。

本章知识导图

- └─ 14.1 类型推导 —— 显式优于隐式,动态≠无类型
- └─ 14.2 值语义与引用语义 —— 可变性的边界
- └─ 14.3 深浅拷贝 —— copy 模块与嵌套陷阱
- └─ 14.4 None 与 Optional —— 空值的工程语义
- └─ 14.5 Union 与 match —— 类型的窄化
- └─ 14.6 dataclass 深度 —— frozen/slots/field
- └─ 14.7 mypy 静态检查 —— 标注兑现为保障
- └─ 14.8 静态与动态的平衡 —— 渐进式类型决策

§14.1 类型推导:显式优于隐式

Python 不需要声明变量类型,但**标注**让意图显式化。类型推导(annotation inference)由 mypy 等检查器完成:它顺着赋值与调用推导每个变量的类型,再与标注比对,不一致就报错——检查发生在开发期,运行期零开销:

```
def greet(name: str, times: int = 1) -> str:
    return f"你好,{name}!" * times

count = 3                # mypy 推导 count: int
count = "三"             # 错误:不能把 str 赋给 int

name = "张三"            # 推导为 str
print(greet(name, count)) # 类型全部吻合,检查通过
```

C(编译期类型)	Python(标注+mypy)
类型强制,无标注概念	标注是文档,检查器强制
类型错误在编译期拦截	mypy 在 CI 中拦截,运行时不受限
泛型模板写起来重	TypeVar/Protocol 轻量表达形状
类型与实现强耦合	渐进式:可只给关键接口加标注

生产规范:公共函数与模块边界**必须有标注**(入参、返回值),内部实现可省略;新代码从第一行就写标注,存量代码按模块逐步迁移。第10章的 TypeVar/Generic/Protocol 是标注的进阶词汇,本章在此之上解决"对象语义"与"空值"两大生产痛点。

§14.2 值语义与引用语义:可变性的边界

一切皆对象,但对象分两类:不可变(int、float、str、tuple、frozenset)与可变(list、dict、set、自定义实例)。int 赋值像 C 的值传递(复制),list 赋值是引用传递(共享)——第4章的传参语义在此深化:可变对象传进函数,函数内修改会"逃逸"出去:

```
def add_item(bag: list, item: str) -> None:
    bag.append(item)           # 直接改共享对象

my_bag = ["水"]
add_item(my_bag, "面包")
print(my_bag)                 # ['水', '面包'] — 外部被改了!

def add_item_safe(bag: list, item: str) -> list:
    return bag + [item]       # 返回新列表,不改原对象
```

生产纪律:**函数默认不修改入参**,需要产出新值就返回新对象(纯函数风格,便于测试与并发);确实要原地修改(排序、批量更新)就把意图写进函数名与 docstring。不可变对象的"值语义"让它们天然可作 dict 的 key 与 set 的元素——可变对象不可哈希,硬塞会 TypeError。

§14.3 深浅拷贝:嵌套结构的复制陷阱

`copy.copy` (浅拷贝)只复制最外层,内层还是共享引用; `copy.deepcopy` (深拷贝)递归复制全部。生产事故高发区:你以为复制了一份,实际内层列表被两边共享修改:

```
import copy

menu = {"soup": ["蛋花汤", "酸辣汤"], "price": 18}
shallow = copy.copy(menu)      # 浅拷贝:内层列表仍共享
deep = copy.deepcopy(menu)     # 深拷贝:完全独立

shallow["soup"].append("番茄汤")
print(menu["soup"])            # ['蛋花汤', '酸辣汤', '番茄汤'] ← 被改了!

deep["soup"].append("菌菇汤")
print(menu["soup"])            # 不变,深拷贝完全隔离
```


误区:浅拷贝当深拷贝用。列表推导、list()、切片、dict.copy 都是浅拷贝——嵌套结构改内层,原对象跟着变,排查时百思不得其解。判据:数据结构只有一层用浅拷贝,多层嵌套(字典套列表、列表套字典)一律 deepcopy。注意 deepcopy 有代价(递归+缓存),大对象循环复用时要评估性能。

与 C 对照:浅拷贝相当于"指针复制",深拷贝相当于"结构体整体复制"。Python 的默认是引用,深拷贝需要显式表达——这也再次印证"Python 是引用语义的 C",理解这一点,所有拷贝问题都有了坐标。

§14.4 None 与 Optional:空值的工程语义

C 用 NULL 表示空,Python 用 None——但 None 是一个**对象**,可以到处传递。标注 Optional[int] (等价 int | None) 声明"可能没有值",而调用方必须处理这种可能。mypy 会用"None 检查"强制你窄化,否则拒绝通过:

```
def find_user(uid: int) -> dict | None:
    """按 id 查用户;查不到返回 None"""
    if uid > 100:
        return {"id": uid, "name": "未知"}
    return None

user = find_user(7)
print(user["name"])          # mypy 报错:user 可能是 None

if user is not None:         # 窄化后,user 确定为 dict
    print(user["name"])      # 通过

name = (user or {}).get("name", "缺省")  # 或用 or 兜底
```

误区:标注 Optional 却裸调属性。返回值标注了 dict | None,调用方直接 user["name"],运行时 AttributeError,标注形同虚设。规范:① 返回值可能是 None 就标注 Optional;② 调用方一律先判空(is not None / or 兜底 / match);③ 内部工具函数可声明"永不返回 None"(直接返回 dict),把判空责任收敛到边界。

None 还承担"哨兵"职责:dict.get 找不到返回 None、正则不匹配返回 None、队列取空返回 None——统一约定"失败给 None",配合 Optional 标注,失败路径变得可检查、可测试。这与 C 的错误码哲学不同:Python 更倾向"异常抛错 + None 表示空值",二者分工在第9章已讲。

另一个常被忽视的点:None 与布尔值的关系。None 在 if 判断中为假,但“为假”不等同于 None——0、空字符串、空列表都为假。判空一律写 is None 而不是 not x,否则空列表被误判为“查询失败”,业务语义被布尔逻辑偷换,这是空值相关的第二类常见 bug。

§14.5 Union 与 match:类型的窄化

int | str 声明一个值可能是多种类型,match 语句按类型分支并窄化——比一堆 isinstance 更可读、更类型安全。Python 3.10+ 的 match 与第3章的 if/else 不同,它更适合"按类型或结构分派"的场景:

```
def render(value: int | str | float) -> str:
    match value:
        case int() as n:
            return f"整数:{n}"
        case str() as s:
            return f"字符串:{s}"
        case float() as f:
            return f"浮点:{f:.2f}"
        case _:
            # 兜底分支:标注之外的形状
            return f"未知:{value!r}"

print(render(42))          # 整数:42
print(render("hi"))       # 字符串:hi
print(render(3.14159))    # 浮点:3.14
```

Union 标注配合 match,mypy 能推导每个 case 分支里 value 的具体类型(类型窄化):case int() 分支里 n 就是 int,可直接做算术。这比手写 isinstance 链的优势:结构清晰、兜底强制、检查器可验证"所有分支都覆盖"。生产上还常用 dict | None、list[str] | None 这类组合,判断与处理逻辑统一走"判空→窄化→使用"三拍。

§14.6 dataclass 深度:frozen 与 slots

第10章见过 dataclass 基础,这里补齐生产三件套:**frozen=True**(不可变,防意外修改)、**field(default_factory=...)**(可变默认值)、**slots=True**(省内存、禁动态加属性):

```
from dataclasses import dataclass, field

@dataclass(frozen=True, slots=True)
class Order:
    order_id: int
    items: list[str] = field(default_factory=list)  # 新列表,不共享
    status: str = field(default="created", kw_only=True)

    def add_item(self, name: str) -> None:
        # frozen 不能改字段,但可变字段内部仍可改
        self.items.append(name)

o1 = Order(1, items=["键盘"])
o2 = Order(1)                # items 是全新的空列表
o2.items.append("鼠标")      # 不影响 o1

o1.status = "paid"           # 报错:frozen 实例禁止改字段
```

误区:可变默认值直接写 = []。dataclass 的 =[] 与函数的默认参数一样,只创建一次,所有实例共享同一个列表——给 A 加数据,B 的列表里也出现。必须用 field(default_factory=list) 让每个实例执行一次工厂函数。同理 dict 用 default_factory=dict。这是面试与评审的经典陷阱,也是线上 bug 的常见源头。

frozen + slots 的组合等于"C 风格的结构体":字段固定、内存紧凑、不可被随意改写——非常适合配置对象、领域模型、跨线程传递的不可变消息。需要"修改"时,用 dataclasses.replace(o, status="paid") 生成新实例,保持不可变语义。可哈希需求再加 eq=True、frozen 实例天然可哈希,可直接进 set 与 dict 的 key。

至此回顾全书:第1章的变量绑定、第4章的传参、第5章的引用计数、本章的深浅拷贝与 None 语义——一条“对象与引用”的主线贯穿始终。能把这个模型讲清楚,Python 的绝大多数行为都有了解释;剩下的是熟练度与工程习惯。

§14.7 mypy 静态检查:把标注兑现为保障

标注本身不检查,**mypy** 负责兑现。接入流程:pyproject.toml 配置 + 命令行运行 + CI 强制。mypy 发现“类型对不上”就报错,相当于把 C 编译器的类型检查搬到 Python 开发期:

```
# pyproject.toml 中的 mypy 配置
[tool.mypy]
python_version = "3.10"
strict = true
warn_return_any = true
check_untyped_defs = true

# 命令行运行
# python -m mypy src/

# 常见检查结果:
# error: Argument 1 to "greet" has incompatible type "int"; expected "str"
# error: Item "None" of "dict | None" has no attribute "get"
```

```
# before.py — 未标注,错误潜伏到运行时
def total(prices):
    return sum(prices)

# after.py — 标注后,mypy 在开发期拦截错误调用
def total(prices: list[float]) -> float:
    return sum(prices)

# 别处调用 total([1, "a"]) → mypy 报错:int 与 str 混入列表
# 而运行时 sum 也会 TypeError — 检查器把故障提前了
```

mypy 的 strict 模式把遗漏标注、Any 泄漏都当错误——初上手会觉得“烦”,但坚持两周后,改代码的胆量明显变大:重构有检查器托底,删字段有引用报错,协作时接口契约一目了然。生产实践:① 新项目从 strict 起步;② 存量项目先开 check_untyped_defs 再逐步收严;③ CI 里 mypy 与 pytest 并列,不过不放行——动态语言的自由,交给运行时;类型的安全,交给检查器。

§14.8 静态与动态的平衡:渐进式类型决策

本书最后谈“度”。Python 的生产哲学不是“静态或动态”,而是按边界选策略:

场景	策略
核心业务模型、公共库 API	全量标注 + mypy strict
内部实现、一次性脚本	轻量标注,类型靠 docstring
与外部系统交互(JSON、数据库行)	边界转 typed dict / dataclass
性能热点	运行时少做类型检查,靠测试保证
测试代码	标注可省,断言即契约

动态特性(第11章)与类型检查并不矛盾:getattr、鸭子类型在标注时代仍有用武之地,只是**要圈在明确边界内**——内部动态,边界静态,是 Python 生产化的黄金比例。一个可复用的经验:凡是"别人会调用"的函数,标注;凡是"只有自己调用"的函数,随心。标注是写给下一个维护者的,而下一个维护者大概率就是三个月后的你。

工程建议 1:给团队定一份"标注契约":函数签名必须完整、Optional 必须被处理、公共模型用 frozen dataclass、新代码禁止裸 dict 出入接口。契约比工具更能防住"标注了却没用"的形式主义。

工程建议 2:用 dataclasses.replace 与不可变模型实现"事件溯源"式修改流:每次变更生成新对象、记录变更日志,排查线上问题时重放即真相——这是类型安全与可观测性的双赢组合。

本章速查表

场景	写法
函数标注	<code>def f(x: int) -> str:</code>
可能为空的返回值	<code>dict None</code> (等价 <code>Optional[dict]</code>)
多类型参数	<code>int str float</code> (3.10+)
判空后再用	<code>if x is not None: ...</code>
多类型分派	<code>match x: case int() as n: ...</code>
浅拷贝	<code>copy.copy(x)</code> (仅一层)
深拷贝	<code>copy.deepcopy(x)</code> (嵌套结构)
不可变数据类	<code>@dataclass(frozen=True)</code>
省内存数据类	<code>@dataclass(slots=True)</code>
可变默认值	<code>field(default_factory=list)</code>
生成修改版	<code>dataclasses.replace(o, status="paid")</code>
静态检查	<code>python -m mypy src/ (strict)</code>
可哈希数据类	<code>frozen=True + eq=True</code> (天然可哈希)
引用传递铁律	函数不修改入参,要新值返回新对象

最小必会清单

- 1. 能说出 Python 可变/不可变对象的引用语义,并解释函数修改入参的逃逸后果
- 2. 能判断何时用浅拷贝、何时用深拷贝,并说出三种"假拷贝"写法

3. 能解释 None 的工程语义,写出判空窄化的规范代码
4. 能用 Union + match 做类型分派,并说清每个分支的窄化结果
5. 能写出 frozen/slots/default_factory 齐全的 dataclass,并解释各自作用
6. 能配置并运行 mypy strict,读懂其报错并修复
7. 能按场景制定"静态/动态"边界策略,说出渐进式类型的取舍

自测题

Q

Q1. 浅拷贝与深拷贝的本质区别?为什么嵌套结构必须 deepcopy?

答案:浅拷贝只复制最外层容器,内层仍是共享引用;深拷贝递归复制所有层。嵌套结构浅拷贝后,修改内层元素会同时影响原对象(共享),表现为"复制了却互相干扰"。判据:单层结构浅拷贝,多层嵌套深拷贝。

Q

Q2. 为什么 dataclass 的可变默认值必须用 field(default_factory=...)?直接 =[] 会发生什么?

答案:默认值表达式在类定义时求值一次,=[] 让所有实例共享同一个列表,给 A 实例添加元素,B 实例的"默认列表"也出现该元素,状态互相泄漏。default_factory 在每个实例创建时执行工厂函数,得到独立对象。这是可变默认值陷阱(第4章函数参数同理)在 dataclass 的复现。

Q

Q3. 标注 dict | None 后,mypy 为什么拒绝 user["name"] 直接访问?两种合规写法?

答案:user 可能是 None,None 上没有下标访问,运行时必炸;mypy 要求先窄化。合规写法:① if user is not None: 分支内访问;② (user or {}).get("name", 缺省) 兜底。更彻底的做法是边界函数保证非 None,把判空责任上移。

Q

Q4. match 与 isinstance 链相比,类型安全上的优势是什么?

答案:match 的每个 case 分支都会由检查器推导窄化后的具体类型,分支内直接用窄化类型(如 case int() 分支内 n 是 int)而无需断言;结构清晰、必须有兜底分支、遗漏类型时更易发现。isinstance 链则每个分支都要自己保证类型与逻辑一致,分支多了难维护。

Q

Q5. frozen=True 的数据类还能"修改"吗?怎么表达变更?

答案:字段层面不能原地赋值(报错),但可:① 可变字段(如 list)内部仍可改;② 用 dataclasses.replace(o, status="paid") 生成新实例;③ 需要彻底不可变时,连 items 也用 tuple 存储。不可变模型配合 replace,天然形成"变更即新对象"的日志式数据流。

Q

Q6. mypy 与运行时类型检查(第10章 isinstance)分工是什么?为什么说"标注是给下一个维护者的"?

答案:mypy 在开发期静态检查,零运行时开销,抓"类型写错、忘判 None、传错参数";isinstance 在运行期保护真实数据,用于边界(反序列化、外部输入)。标注本身不执行,它的价值是契约与文档——下一个维护者(常是未来的自己)靠签名理解接口,靠检查器保证改动的安全。

章末实验题:类型安全领域模型

实验 14•1 订单系统领域模型(覆盖:\$14.1–\$14.8)

需求:① 用 frozen+slots dataclass 定义 User(不可变)与 Order(含可变 items),items 用 default_factory;② 定义 Coupon 类型(Union:满减券或折扣券),用 match 计算优惠并验证窄化;③ 结算函数返回 dict | None,含判空窄化与 None 兜底;④ 用 deepcopy 实现"订单快照",验证与原订单完全隔离;⑤ 全部函数标注完整,运行 mypy --strict 零错误;⑥ 写 pytest 覆盖:优惠计算、None 分支、快照隔离、replace 变更。

覆盖知识点:引用语义与拷贝(\$14.2、\$14.3)/None 与 Optional(\$14.4)/Union 与 match(\$14.5)/dataclass 深度(\$14.6)/mypy(\$14.7)/边界策略(\$14.8)。

实验环境:Python 3.10+,安装 mypy 与 pytest。

```

# domain.py — 领域模型与结算逻辑
"""类型安全订单模型:不可变主体 + 显式拷贝 + 静态检查"""
from dataclasses import dataclass, field, replace
from copy import deepcopy

@dataclass(frozen=True, slots=True)
class User:
    uid: int
    name: str

@dataclass(frozen=True, slots=True)
class Order:
    order_id: int
    user: User
    items: list[str] = field(default_factory=list)

    def add_item(self, name: str) -> "Order":
        return replace(self, items=self.items + [name])

@dataclass(frozen=True)
class FullCut:
    """满减券:满 threshold 减 discount"""
    threshold: float
    discount: float

@dataclass(frozen=True)
class RateCoupon:
    """折扣券:打 rate 折(0.8 = 八折)"""
    rate: float

Coupon = FullCut | RateCoupon      # 类型别名:两种券的并集

def calc_price(subtotal: float, coupon: Coupon | None) -> dict:
    """结算:返回账单;coupon 为 None 时不优惠"""
    if coupon is None:
        return {"subtotal": subtotal, "pay": subtotal}

    match coupon:
        case FullCut(threshold=t, discount=d) if subtotal >= t:
            pay = subtotal - d
        case RateCoupon(rate=r):
            pay = subtotal * r
        case _:
            pay = subtotal          # 满减未达门槛,原价
    return {"subtotal": subtotal, "pay": round(pay, 2)}

def find_order(oid: int) -> Order | None:
    """模拟查询:id 为偶数返回订单,奇数返回 None"""

```

```
if oid % 2 == 0:
    return Order(oid, User(1, "张三"), ["键盘"])
return None
```



```

# test_domain.py — 测试与 mypy 验收
"""对 domain.py 的行为验证"""
from copy import deepcopy

import pytest

from domain import FullCut, Order, RateCoupon, User, calc_price, find_order


def test_fullcut_above_threshold() -> None:
    bill = calc_price(300.0, FullCut(200, 30))
    assert bill["pay"] == 270.0


def test_fullcut_below_threshold() -> None:
    bill = calc_price(100.0, FullCut(200, 30))
    assert bill["pay"] == 100.0


def test_rate_coupon() -> None:
    bill = calc_price(200.0, RateCoupon(0.8))
    assert bill["pay"] == 160.0


def test_no_coupon() -> None:
    bill = calc_price(88.0, None)
    assert bill["pay"] == 88.0


def test_frozen_user_rejected() -> None:
    user = User(1, "张三")
    with pytest.raises(Exception):
        user.name = "李四"          # frozen 禁止赋值


def test_replace_creates_new_order() -> None:
    order = Order(1, User(1, "张三"), ["键盘"])
    grown = order.add_item("鼠标")
    assert order.items == ["键盘"]   # 原订单不变
    assert grown.items == ["键盘", "鼠标"]


def test_deepcopy_isolated() -> None:
    order = Order(2, User(2, "李四"), ["书"])
    snap = deepcopy(order)
    snap.items.append("笔")
    assert order.items == ["书"]     # 快照修改不影响原订单


def test_find_order_none() -> None:
    order = find_order(7)             # 奇数 id 返回 None
    assert order is None
    if order is not None:             # 窄化:此分支内 order 是 Order
        assert order.user.name == "张三"

```

验收清单:① `python -m mypy --strict domain.py` 输出 Success,零错误(若有 Any 泄漏逐一消除);② `python -m pytest test_domain.py -v` 全绿;③ 检查代码中每处 dict | None 都有判空或兜底;④ 尝试把 `add_item` 改成 `self.items.append(name)` 会得到 frozen 报错,确认 `replace` 是正确姿势;⑤ 用 `typing.reveal_type` 在结算函数里确认 `match` 各分支的窄化类型。全部通过,即完成全书的最后一次实践——带着"标注、检查、测试"三件套进入生产。

全书完。从 C 到 Python,从语法到哲学,从自由到纪律——愿你写出优雅、安全、可交付的 Python 代码。